STUDENT MANUAL

FLUTTER

for
Practical App Development

Written by :
Tin Mai Zaw



2026 Edition

NORTHERN CITY

NORTHERN CITY

FLUTTER

MOBILE APPLICATION DEVELOPMENT

Student Manual

By

Tin Mai Zaw



Contents

CHAPTER 1 ..... PAGE 3
Flutter Development Environment Setup
CHAPTER 2 ..... PAGE 18
Creating Your First Flutter Application
CHAPTER 3 ..... PAGE 29
Dart Programming Fundamentals
CHAPTER 4 ..... PAGE 40
Flutter widgets and application layouts
CHAPTER 5 ..... PAGE 54
UI Design, Themes and Responsive Layouts
CHAPTER 6 ..... PAGE 69
Navigation and Routing
CHAPTER 7 ..... PAGE 78
Forms and User Input Validation
CHAPTER 8 ..... PAGE 92
State Management with setState and Callbacks
CHAPTER 9 ..... PAGE 104
Lists, GridViews and CRUD Operations
CHAPTER 10 ..... PAGE 119
Advanced State Management with Provider / Riverpod
CHAPTER 11 ..... PAGE 133
Local Database with SQLite
CHAPTER 12 ..... PAGE 151
REST API Integration with HTTP and JSON
CHAPTER 13 ..... PAGE 165
Firebase Authentication and User Login
CHAPTER 14 ..... PAGE 181
Building a Complete Flutter Application
CHAPTER 15 ..... PAGE 198
Publishing Flutter Apps to Google Play Store
References ..... PAGE 217

CHAPTER 1

Flutter Development Environment Setup



ဒီ Chapter မှာ Flutter ကို အသုံးပြုပြီး Android Mobile Application တွေ စတင်ရေးသားနိုင်ဖို့ လိုအပ်တဲ့ Development Environment ကို အဆင့်ဆင့် တပ်ဆင်သွားမှာ ဖြစ်ပါတယ်။ နောက်ပိုင်း Shopping Application Project မှာ Product CRUD, User Authentication, Database, Shopping Cart နဲ့ Google Play Store Deployment တို့ကို လက်တွေ့လုပ်ဆောင်နိုင်ဖို့ ဒီအဆင့်တွေကို မှန်ကန်စွာ ပြင်ဆင်ထားဖို့ လိုအပ်ပါတယ်။

1.1. သင်ခန်းစာရည်ရွယ်ချက်များ

သင်ခန်းစာပြီးဆုံးတဲ့အခါ အောက်ပါအကြောင်းအရာတွေကို လုပ်ဆောင်နိုင်မှာ ဖြစ်ပါတယ်။

- Flutter SDK ကို Download လုပ်ပြီး တပ်ဆင်နိုင်ခြင်း။
- Dart SDK နဲ့ Flutter ရဲ့ဆက်စပ်မှုကို နားလည်ခြင်း။
- Android Studio နဲ့ Android SDK ကို ပြင်ဆင်ခြင်း။
- VS Code မှာ Flutter နဲ့ Dart Extensions တပ်ဆင်ခြင်း။
- Windows Environment Variables မှာ `PATH` ထည့်သွင်းခြင်း။
- `flutter doctor` နဲ့ Installation ပြဿနာတွေကို စစ်ဆေးခြင်း။
- Android Emulator သို့မဟုတ် Physical Android Phone မှာ App Run လုပ်ဖို့ ပြင်ဆင်ခြင်း။

1.2. လိုအပ်သော Software များ



Flutter SDK
Flutter Framework နဲ့ command-line tools တွေ ပါဝင်ပြီး Android Application တွေ တည်ဆောက်ဖို့ အသုံးပြုပါတယ်။
Official website: Flutter for Windows



Dart SDK
Flutter Application တွေကို ရေးသားရာမှာ အသုံးပြုတဲ့ Programming Language ဖြစ်ပါတယ်။ Flutter SDK ထဲမှာ Dart SDK ပါဝင်ပြီးသားဖြစ်တဲ့အတွက် Dart ကို သီးခြားတပ်ဆင်ရန် ပုံမှန်အားဖြင့် မလိုအပ်ပါဘူး။
Official website: Get the Dart SDK



Android Studio
Android SDK, Android Emulator နဲ့ Android development tools တွေကို တပ်ဆင်စီမံဖို့ အသုံးပြုပါတယ်။
Official website: Download Android Studio



Visual Studio Code (VS Code)
Flutter Project ရဲ့ Source Code တွေကို ရေးသား၊ ပြင်ဆင်၊ Debug လုပ်ဖို့ အသုံးပြုပါတယ်။
Official website: Download VS Code

အရေးကြီးချက်: Flutter ကို Windows မှာ အသုံးပြုဖို့ Android Studio နဲ့ VS Code နှစ်ခုစလုံးကို မဖြစ်မနေ ဖွင့်ထားရမှာ မဟုတ်ပါဘူး။ Android SDK နဲ့ Flutter/Dart development tools တွေကို ပြင်ဆင်ပြီးရင် VS Code တစ်ခုတည်းကနေ Coding လုပ်နိုင်ပါတယ်။

1.3. Git for Windows တပ်ဆင်ခြင်း

Flutter SDK ကို Download လုပ်ပြီး အသုံးပြုရာမှာ Git လိုအပ်တဲ့အတွက် Git ကို အရင်တပ်ဆင်ထားပါမယ်။

Download: Git for Windows

တပ်ဆင်နည်း -

1. Git website ကို ဖွင့်ပါ။
2. Windows အတွက် Installer ကို Download လုပ်ပါ။
3. Installer ကို Run လုပ်ပြီး Default Settings အတိုင်း `Next` နှိပ်သွားနိုင်ပါတယ်။
4. Installation ပြီးပါက CMD သို့မဟုတ် PowerShell ကို ဖွင့်ပါ။
5. အောက်ပါ Command ကို ရိုက်ထည့်ပါ။

```
git --version
```

အောင်မြင်စွာ တပ်ဆင်ပြီးပါက `git version 2.x.x.windows.x` ကဲ့သို့ Version Number ကို မြင်ရပါမယ်။

1-4. Flutter SDK တပ်ဆင်ခြင်း

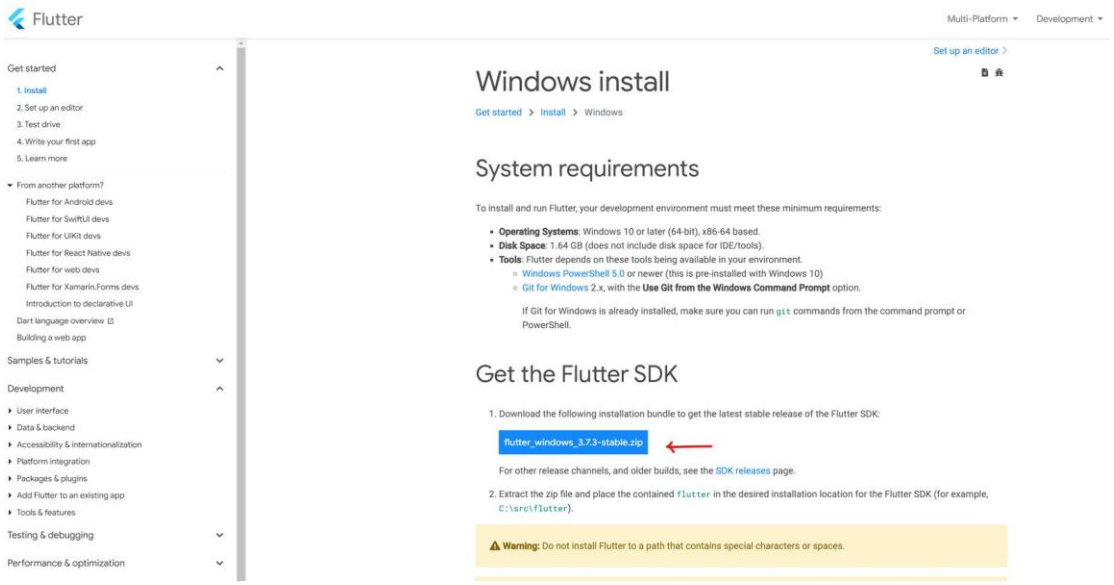


Figure 1.1. Flutter Download Link

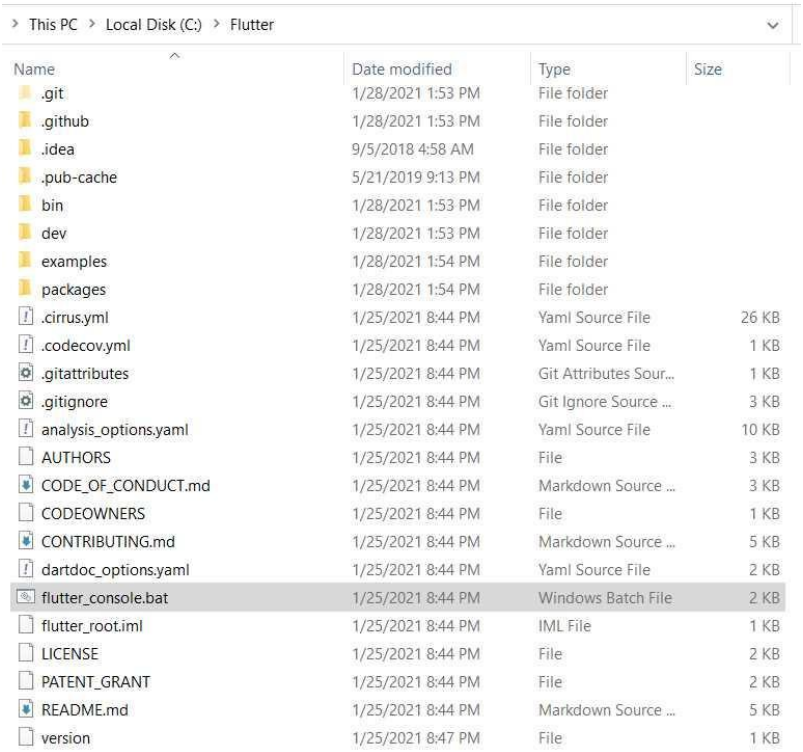


Figure 1.2. Local Flutter Folder

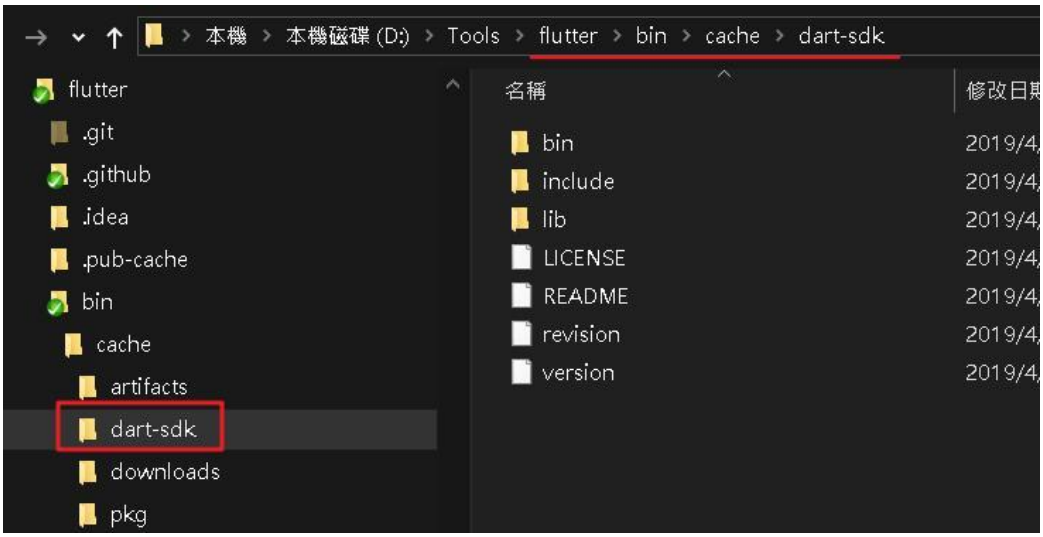


Figure 1.3. Flutter Cache Folder

Flutter SDK ကို တပ်ဆင်ရန် နည်းလမ်းနှစ်မျိုးရှိပါတယ်။ VS Code ကနေ တိုက်ရိုက် Download လုပ်နိုင်သလို Manual နည်းလမ်းနဲ့ ZIP File ကို Download လုပ်ပြီး Extract လုပ်နိုင်ပါတယ်။ ဒီသင်ခန်းစာမှာ နားလည်ရလွယ်တဲ့ Manual နည်းလမ်းကို အသုံးပြုပါမယ်။

Step 1 — Flutter SDK Download လုပ်ခြင်း

Official guide: Install Flutter manually

Flutter SDK ZIP File ကို Download လုပ်ပြီး Extract လုပ်ပါ။ ဥပမာ -

```
C:\src\flutter
```

ဒီ Folder ထဲမှာ အောက်ပါ Structure ကို တွေ့ရပါမယ်။

```
C:\src\flutter\
    bin\
        flutter.bat
        dart.bat
    cache\
    packages\
    version
```

`flutter.bat` File ရှိနေခြင်းက SDK Folder ကို မှန်ကန်စွာ Extract လုပ်ထားကြောင်း စစ်ဆေးနိုင်တဲ့ အချက်တစ်ခုဖြစ်ပါတယ်။

သတိပြုရန်: Flutter SDK ကို `C:\Program Files\` ကဲ့သို့ Permission ကန့်သတ်ချက်များရှိနိုင်တဲ့ Folder ထက် `C:\src\flutter` ကဲ့သို့ ကိုယ်ပိုင် Development Folder ထဲမှာထားတာ ပိုမိုအဆင်ပြေပါတယ်။

1.5. Windows Environment Variables သတ်မှတ်ခြင်း

Flutter ကို CMD, PowerShell နဲ့ VS Code Terminal တို့ကနေ ဘယ်နေရာမှာမဆို အသုံးပြုနိုင်ဖို့ `PATH` ထဲမှာ Flutter SDK ရဲ့ `bin` Folder ကို ထည့်ပေးရပါမယ်။

Step 1 — Environment Variables ဖွင့်ခြင်း

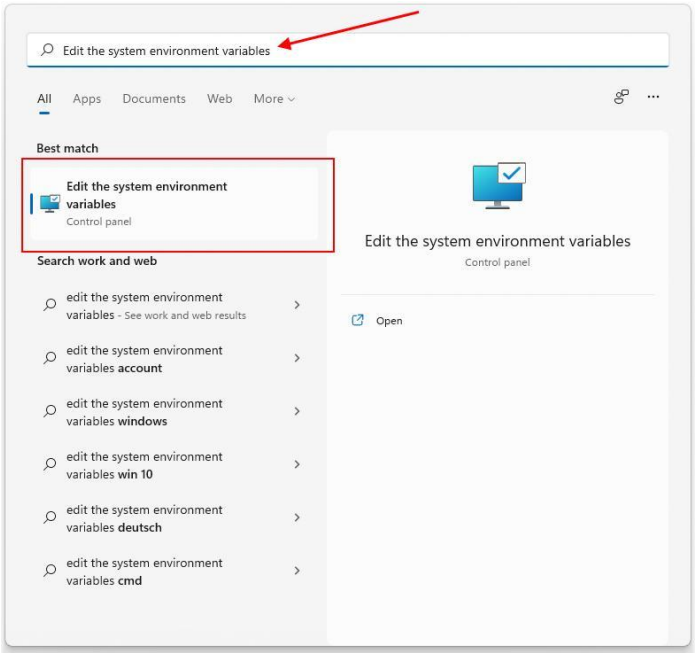


Figure 1.4. Search System Environment Variables

၁။ Windows Search မှာ `environment variables` လို့ ရိုက်ရှာပါ။

၂။ `Edit the system environment variables` ကို ဖွင့်ပါ။

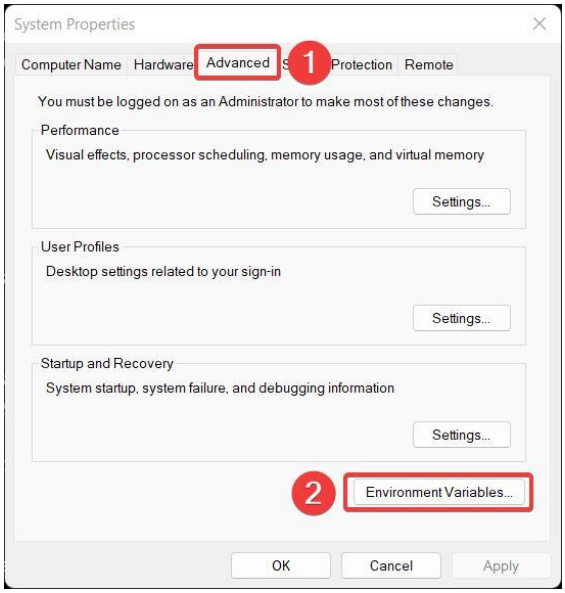


Figure 1.5. Config Environment Variables

၃။ System Properties Window မှာ `Advanced` Tab ကို ရွေးပါ။

၄။ `Environment Variables...` ကို နှိပ်ပါ။

Step 2 — Flutter PATH ထည့်ခြင်း

`User variables` အောက်က `Path` ကို ရွေးပြီး `Edit` နှိပ်ပါ။

`New` ကို နှိပ်ပြီး အောက်ပါ Path ကို ထည့်ပါ။

```
C:\src\flutter\bin
```

ပြီးရင် `OK` ကို အဆင့်ဆင့် နှိပ်ပြီး Window အားလုံးကို ပိတ်ပါ။

Flutter ကို တခြား Folder မှာ Extract လုပ်ထားတယ်ဆိုရင် ကိုယ့် SDK ရဲ့တကယ့် Path ကို အသုံးပြုရပါမယ်။

Step 3 — Flutter နဲ့ Dart အလုပ်လုပ်မလုပ် စစ်ဆေးခြင်း

PowerShell အသစ်တစ်ခု ဖွင့်ပြီး အောက်ပါ Command တွေကို Run ပါ။

```
flutter --version
dart --version
```

အောင်မြင်ရင် Flutter နဲ့ Dart တို့ရဲ့ Version Information တွေကို မြင်ရပါမယ်။

Command မတွေ့ဘူးဆိုရင် Terminal အဟောင်းတွေကို ပိတ်ပြီး အသစ်ပြန်ဖွင့်ပါ။ မရသေးရင် `PATH` ကို ပြန်စစ်ပါ။

1.6. Android Studio နဲ့ Android SDK တပ်ဆင်ခြင်း

Android Studio က Flutter App တွေကို Android Emulator သို့မဟုတ် Physical Android Phone ပေါ်မှာ Run လုပ်ဖို့ လိုအပ်တဲ့ Android Development Tools တွေကို ပြင်ဆင်ပေးပါတယ်။

Download: Android Studio Official Website

Step 1 — Android Studio Installation

1. Android Studio Installer ကို Download လုပ်ပါ။
2. Installer ကို Run လုပ်ပါ။
3. Android Studio နဲ့ Android Virtual Device (AVD) ဆိုင်ရာ Components တွေကို ရွေးချယ်ပါ။
4. Installation ပြီးရင် Android Studio ကို ဖွင့်ပါ။
5. Setup Wizard က တောင်းဆိုတဲ့ Android SDK နဲ့ လိုအပ်တဲ့ Components တွေကို Download လုပ်ပါ။

Step 2 — Android SDK Components စစ်ဆေးခြင်း

Android Studio ထဲမှာ `More Actions` သို့မဟုတ် `Tools` Menu ကနေ `SDK Manager` ကို ဖွင့်ပါ။ Version အလိုက် Menu နေရာ အနည်းငယ်ကွာနိုင်ပါတယ်။

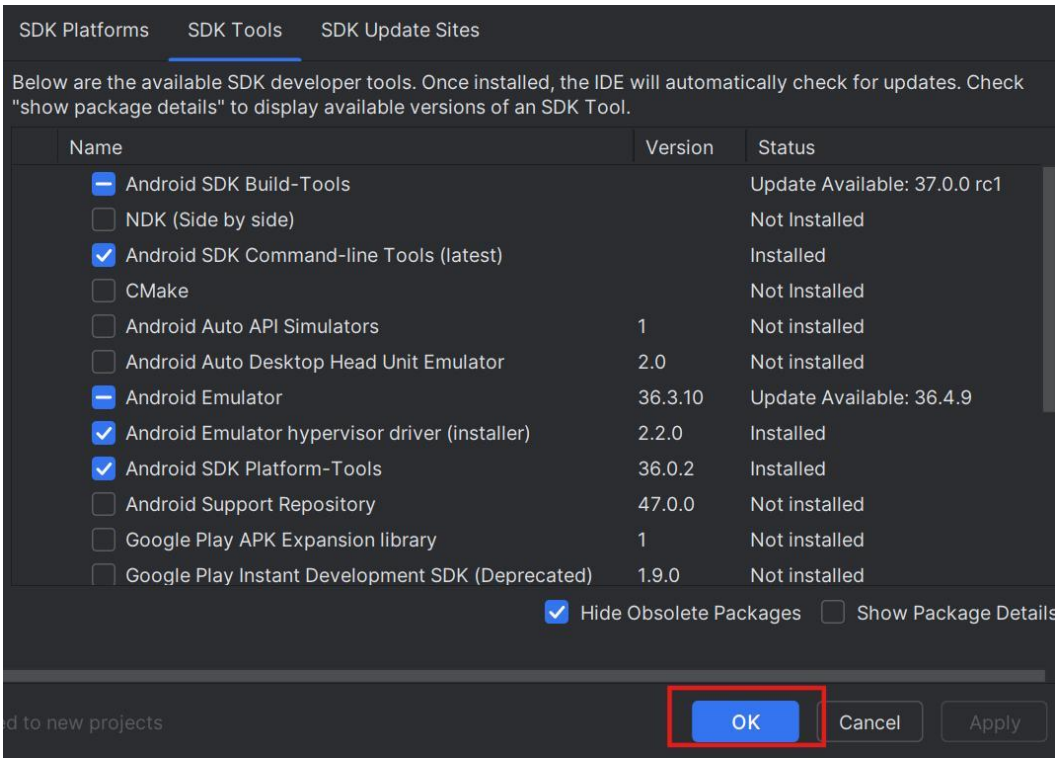


Figure 1.6. Installation of SDK Tools

`SDK Platforms` Tab မှာ လက်ရှိ Flutter Project အတွက် သင့်လျော်တဲ့ Android SDK Platform ကို ရွေးပါ။

`SDK Tools` Tab မှာ အောက်ပါ Tools တွေကို စစ်ဆေးပါ။

- Android SDK Build-Tools
- Android SDK Platform-Tools
- Android SDK Command-line Tools (latest)
- Android Emulator (Emulator အသုံးပြုမယ်ဆိုရင်)

`Apply` ကို နှိပ်ပြီး လိုအပ်တဲ့ Packages တွေ Download လုပ်ပါ။

Step 3 — Android SDK Path စစ်ဆေးခြင်း

Windows မှာ Android SDK ရဲ့ပုံမှန်တည်နေရာက အောက်ပါအတိုင်း ဖြစ်နိုင်ပါတယ်။

```
%LOCALAPPDATA%\Android\Sdk
```

ဥပမာ -

```
C:\Users\YourUserName\AppData\Local\Android\Sdk
```

Android Studio ရဲ့ SDK Manager ထိပ်ပိုင်းမှာလည်း တကယ့် SDK Location ကို ကြည့်နိုင်ပါတယ်။

1.7. Android SDK Environment Variable ထည့်ခြင်း

Android SDK Path ကို Command Line ကနေ တိုက်ရိုက်အသုံးပြုချင်ရင် Environment Variables မှာ သတ်မှတ်နိုင်ပါတယ်။

`User variables` အောက်မှာ `New` ကို နှိပ်ပြီး -

Variable name:

```
ANDROID_HOME
```

Variable value:

```
%LOCALAPPDATA%\Android\Sdk
```

ထို့နောက် `Path` ထဲမှာ အောက်ပါ Entry တွေကို ထည့်နိုင်ပါတယ်။

```
%ANDROID_HOME%\platform-tools
%ANDROID_HOME%\cmdline-tools\latest\bin
```

SDK Location ကို ကိုယ့်စက်ရဲ့တကယ့်တည်နေရာနဲ့ ကိုက်ညီအောင် ပြင်ဆင်ပါ။ `cmdline-tools\latest\bin` မရှိသေးရင် SDK Manager ကနေ Android SDK Command-line Tools (latest) ကို တပ်ဆင်ပါ။

1.8. VS Code နဲ့ Flutter Extension တပ်ဆင်ခြင်း

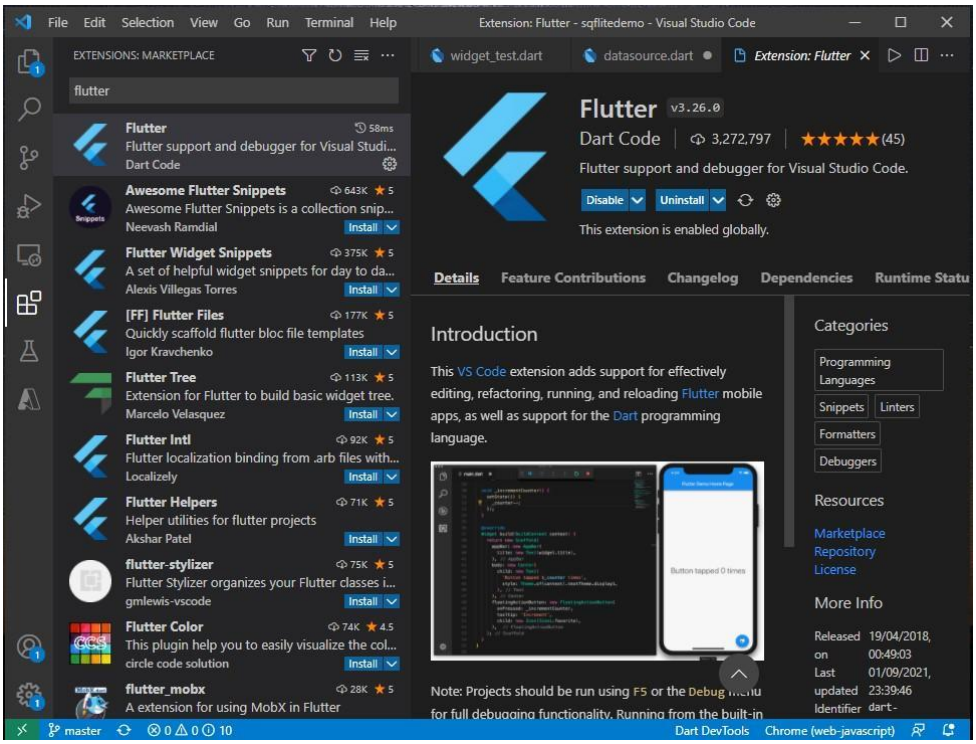


Figure 1.7. VSCode Extension for Flutter

Download: Visual Studio Code

VS Code ကို ဖွင့်ပြီး `Ctrl + Shift + X` ကို နှိပ်ပါ။ Extensions Marketplace မှာ အောက်ပါ Extensions တွေကို ရှာဖွေတပ်ဆင်ပါ။

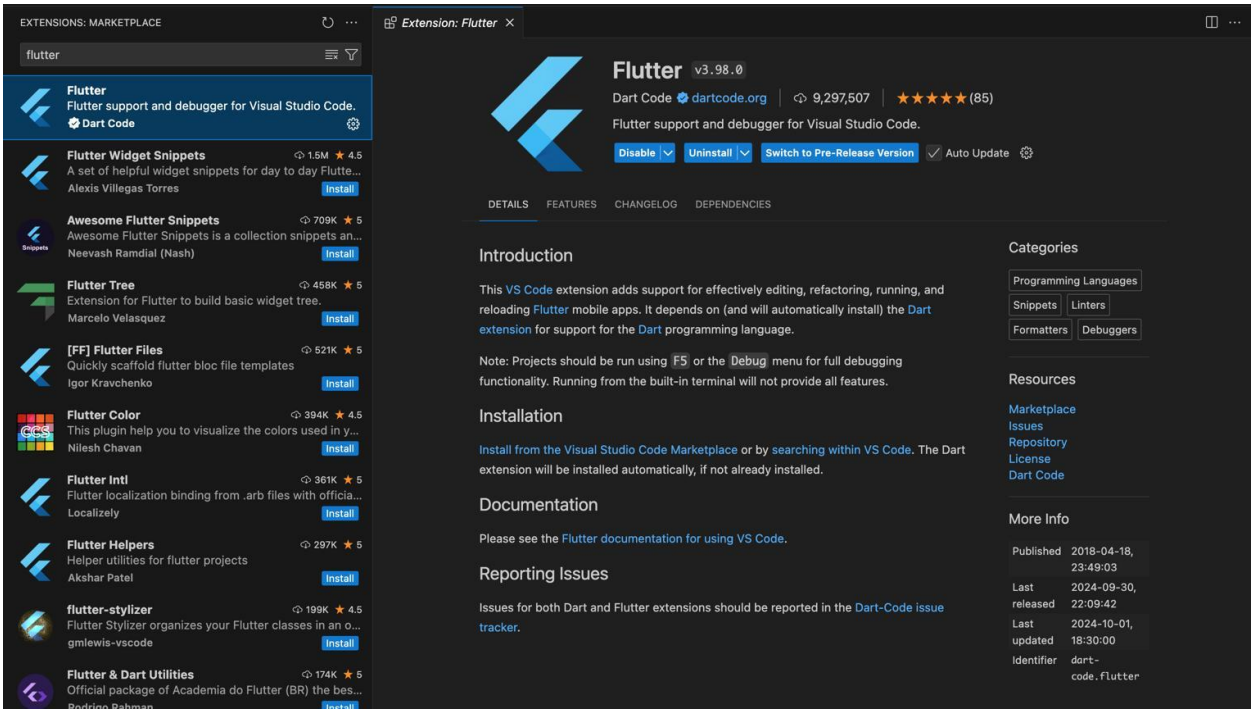


Figure 1.8. VS Code Extension: Flutter



Flutter

Publisher: Dart Code

Flutter Project ဖန်တီးခြင်း၊ Run/Debug လုပ်ခြင်းနဲ့ Hot Reload အသုံးပြုခြင်းတို့ကို ပံ့ပိုးပေးပါတယ်။



Dart

Publisher: Dart Code

Dart Syntax Highlighting, Code Completion နဲ့ Error Detection တို့ကို ပံ့ပိုးပေးပါတယ်။

Flutter Extension ကို တပ်ဆင်တဲ့အခါ Dart Extension ကိုလည်း အလိုအလျောက် ထည့်သွင်းပေးနိုင်ပါတယ်။

1.9. Flutter Doctor ဖြင့် Installation စစ်ဆေးခြင်း

ဒီအဆင့်က သင်ခန်းစာရဲ့အရေးကြီးဆုံးအပိုင်းဖြစ်ပါတယ်။ `flutter doctor` က Flutter Development Environment မှာ လိုအပ်နေတဲ့ Tools နဲ့ Configuration ပြဿနာတွေကို စစ်ဆေးပေးပါတယ်။

VS Code Terminal သို့မဟုတ် PowerShell ကို ဖွင့်ပြီး -

```
flutter doctor
```

အသေးစိတ် စစ်ဆေးလိုပါက -

```
flutter doctor -v
```

Android SDK License တွေကို လက်ခံဖို့ လိုအပ်ရင် အောက်ပါ Command ကို Run ပါ။

```
flutter doctor --android-licenses
```

ပြသလာတဲ့ License Terms တွေကို ဖတ်ရှုပြီး လက်ခံမယ်ဆိုရင် `y` နဲ့ အတည်ပြုပါ။

Flutter Doctor ရလဒ်ကို ဖတ်နည်း

[✓] Flutter
Flutter SDK ကို မှန်ကန်စွာ တွေ့ရှိပြီး အသုံးပြုနိုင်ပါတယ်။

[✓] Android toolchain
Android SDK နဲ့ Android Build Tools တွေကို အသုံးပြုနိုင်ပါတယ်။

[✓] Chrome / VS Code
စစ်ဆေးထားတဲ့ Browser သို့မဟုတ် Editor ကို တွေ့ရှိထားပါတယ်။

[!] သို့မဟုတ် [✗]

သက်ဆိုင်ရာ Component မှာ Warning သို့မဟုတ် Error ရှိနေပါတယ်။ Error Message အလိုက် ဖြေရှင်းပြီး `flutter doctor` ကို ပြန် Run လုပ်ပါ။

Windows Desktop အတွက် Visual Studio မတပ်ဆင်ရသေးလို့ Warning ပြနေတယ်ဆိုရင် Android Application ရေးသားခြင်းအတွက် မဖြစ်မနေ ပြင်ဆင်ရမယ်လို့ မဆိုလိုပါဘူး။ ဒီသင်တန်းရဲ့အဓိက Target က Android ဖြစ်တဲ့အတွက် Android Toolchain ကို အဓိကထားစစ်ဆေးပါ။

1.10. Android Emulator ပြင်ဆင်ခြင်း

Android Phone မရှိသေးရင် Android Emulator ကို အသုံးပြုပြီး Flutter App ကို စမ်းသပ်နိုင်ပါတယ်။

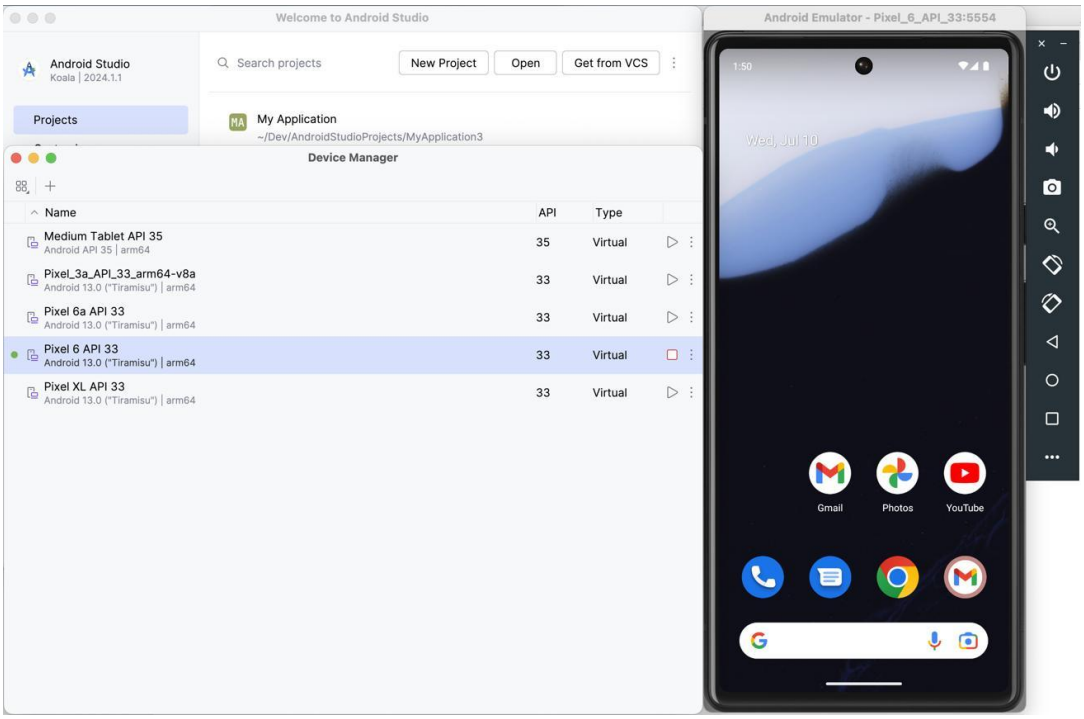


Figure 1.9. Android Emulation Installation

1. Android Studio ကို ဖွင့်ပါ။
2. `Tools` → `Device Manager` ကို ဝင်ပါ။
3. `Create Virtual Device` ကို နှိပ်ပါ။
4. Pixel Device တစ်ခုကို ရွေးချယ်ပါ။

5. လိုအပ်တဲ့ Android System Image ကို Download လုပ်ပါ။
6. Emulator ကို Start လုပ်ပါ။

Emulator စတင်အသုံးပြုနိုင်ပြီဆိုရင် PowerShell မှာ -

```
flutter devices
```

လို့ ရိုက်ထည့်ပါ။ Emulator ကို စာရင်းထဲမှာ တွေ့ရပါမယ်။

Emulator မတက်လာရင် BIOS/UEFI Virtualization နဲ့ Windows ရဲ့လိုအပ်တဲ့ Virtualization Configuration တွေကို စစ်ဆေးရန်နိုင်ပါတယ်။ Emulator အစား USB Debugging ဖွင့်ထားတဲ့ Android Phone ကိုလည်း အသုံးပြုနိုင်ပါတယ်။

1.11. လေ့ကျင့်ခန်း

Installation Checklist

- Git ကို တပ်ဆင်ပြီး git --version ဖြင့် စစ်ဆေးပါ။
- Flutter SDK ကို C:\src\flutter ထဲမှာ Extract လုပ်ပါ။
- PATH ထဲမှာ C:\src\flutter\bin ကို ထည့်ပါ။
- flutter --version နဲ့ dart --version ကို Run လုပ်ပါ။
- Android Studio မှာ Android SDK နဲ့ SDK Tools တွေကို ပြင်ဆင်ပါ။
- VS Code မှာ Flutter နဲ့ Dart Extensions တပ်ဆင်ပါ။
- flutter doctor -v ကို Run လုပ်ပြီး ရလဒ်ကို စစ်ဆေးပါ။
- Android Emulator သို့မဟုတ် Physical Phone ကို flutter devices နဲ့ စစ်ဆေးပါ။

1.12. Summary

ဒီသင်ခန်းစာမှာ Flutter SDK, Dart SDK, Git, Android Studio, Android SDK နဲ့ VS Code တို့ကို ပြင်ဆင်ခဲ့ပြီး Environment Variables နဲ့ `flutter doctor` အသုံးပြုပုံကို လေ့လာခဲ့ပါတယ်။

နောက်သင်ခန်းစာမှာ Lesson 2: Create Your First Flutter Project ကို ဆက်လက်လေ့လာပြီး `flutter create` Command အသုံးပြုခြင်း၊ Project Folder Structure နားလည်ခြင်း၊ `main.dart` ရဲ့ အလုပ်လုပ်ပုံနဲ့ ပထမဆုံး Flutter Application ကို Run ခြင်းတို့ကို လက်တွေ့ရေးသားသွားမှာ ဖြစ်ပါတယ်။

CHAPTER 2

Creating Your First Flutter Application



ဒီ Chapter မှာ Flutter ကို အသုံးပြုပြီး ပထမဆုံး Mobile Application တစ်ခုကို ကိုယ်တိုင်တည်ဆောက်မှာ ဖြစ်ပါတယ်။ Flutter Project တည်ဆောက်ခြင်း၊ Project Structure ကို နားလည်ခြင်း၊ `main.dart`၊ `MaterialApp`၊ `Scaffold` တို့ကို အသုံးပြုခြင်းနှင့် Android Emulator ပေါ်မှာ Application ကို Run ခြင်းတို့ကို အဆင့်ဆင့် လေ့လာနိုင်ပါတယ်။

2.1. Chapter ၏ ရည်ရွယ်ချက်များ

သင်ခန်းစာပြီးဆုံးပါက အောက်ပါအချက်များကို လုပ်ဆောင်နိုင်မှာ ဖြစ်ပါတယ်။

- Flutter Project အသစ်တစ်ခု ဖန်တီးနိုင်ခြင်း
- Flutter Project ရဲ့ Folder Structure ကို နားလည်ခြင်း
- `main()` နှင့် `runApp()` ကို အသုံးပြုခြင်း
- `MaterialApp` နှင့် `Scaffold` ကို အသုံးပြုခြင်း
- Text၊ AppBar၊ Button နှင့် Icon များ ထည့်သွင်းခြင်း
- Android Emulator ပေါ်တွင် Application ကို Run ခြင်း

2.2. Flutter Project အသစ် ဖန်တီးခြင်း

Step 1: Terminal ဖွင့်ပါ

VS Code ကို ဖွင့်ပြီး Terminal ကို ဖွင့်ပါ။

Menu မှ `Terminal → New Terminal` ကို ရွေးချယ်နိုင်ပါတယ်။

Flutter တပ်ဆင်ထားမှုကို စစ်ဆေးရန် အောက်ပါ Command ကို ရိုက်ထည့်ပါ။

```
flutter --version
```

Flutter SDK အလုပ်လုပ်ခြင်း ရှိ၊ မရှိ စစ်ဆေးရန် —

```
flutter doctor
```

ပြဿနာမရှိပါက Project စတင်တည်ဆောက်နိုင်ပါပြီ။

Step 2: Project ဖန်တီးပါ

Terminal မှာ အောက်ပါ Command ကို ရိုက်ထည့်ပါ။

```
flutter create my_first_app
```

Project Folder ထဲသို့ ဝင်ပါ။

```
cd my_first_app
```

VS Code မှာ Project ကို ဖွင့်ရန် —

```
code .
```

အသုံးပြုမည့် Commands

Command တစ်ခုချင်းစီကို Terminal မှာ အစဉ်လိုက် Run ပါ။

```
flutter create my_first_app
cd my_first_app
code .
```

2.3. Flutter Project Structure

Flutter Project ကို ဖန်တီးပြီးနောက် အောက်ပါ Folder Structure ကို တွေ့ရမှာ ဖြစ်ပါတယ်။

```
my_first_app/
```

```
android/
```

Android Application အတွက် Native Configuration များ

```
ios/
```

iOS Application အတွက် Configuration များ

```
lib/
```

```
main.dart
```

Application ရဲ့အဓိက Dart Source Code ရေးသားရာနေရာ

```
test/
```

Unit Test နှင့် Widget Test များ

```
web/
```

Web Platform အတွက် ဖိုင်များ (ထည့်သွင်းထားပါက)

```
pubspec.yaml
```

Package Dependencies၊ Assets နှင့် Project Configuration များ

အရေးကြီးချက်: ဒီသင်ခန်းစာမှာ အဓိက ပြင်ဆင်ရမယ့်ဖိုင်က `lib/main.dart` ဖြစ်ပါတယ်။

2.4. main.dart ကို နားလည်ခြင်း

`lib/main.dart` ဖိုင်ကို ဖွင့်ပါ။ Flutter က အစပိုင်းမှာ ထည့်ပေးထားတဲ့ Sample Code ကို ဖျက်ပြီး အောက်ပါ Code ကို ရေးသားပါ။

Complete Source Code

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'My First Flutter App',
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('My First App'),
          backgroundColor: Colors.blue,
          foregroundColor: Colors.white,
        ),
        body: const Center(
          child: Text(
            'Hello Flutter!',
            style: TextStyle(
              fontSize: 28,
              fontWeight: FontWeight.bold,
              color: Colors.blue,
            ),
          ),
        ),
      ),
    );
  }
}
```

ဒီ Code ကို Save လုပ်ပါ။ (`Ctrl + S`)

Code တစ်ခုချင်းစီ ရှင်းလင်းချက်

1. Import Flutter Material Library

```
import 'package:flutter/material.dart';
```

`MaterialApp`၊ `Scaffold`၊ `Text`၊ `AppBar`၊ `Center` နှင့် `Colors` ကဲ့သို့သော Material Design Widgets များကို အသုံးပြုနိုင်စေပါတယ်။

2. main() Function

```
void main() {
  runApp(const MyApp());
}
```

Dart Application တစ်ခု စတင်အလုပ်လုပ်တဲ့အခါ `main()` Function ကနေ စတင်ပါတယ်။ `runApp()` က `MyApp` Widget ကို Flutter Application ရဲ့ Root Widget အဖြစ် စတင်ပြသပေးပါတယ်။

3. MyApp Class

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

`MyApp` ဟာ `StatelessWidget` ကို အခြေခံထားတဲ့ Widget ဖြစ်ပါတယ်။ ဒီ Widget ကိုယ်တိုင်မှာ ပြောင်းလဲနေတဲ့ State မရှိတဲ့အတွက် `StatelessWidget` ကို အသုံးပြုထားတာ ဖြစ်ပါတယ်။

4. build() Method

```
@override
Widget build(BuildContext context) {
```

`build()` Method က Screen ပေါ်မှာ ဘယ်လို UI ကို ပြသရမလဲဆိုတာ သတ်မှတ်ပေးပါတယ်။

2.5. MaterialApp ဆိုတာဘာလဲ?

`MaterialApp` ဟာ Flutter Application တစ်ခုရဲ့ အဓိက Configuration နဲ့ Navigation စနစ်ကို စီမံပေးတဲ့ Widget ဖြစ်ပါတယ်။

အသုံးများတဲ့ Properties များကတော့ —

Property	ရှင်းလင်းချက်
`title`	Application ရဲ့အမည်
`home`	Application စဖွင့်ချိန်မှာ ပြသမယ့် Widget
`theme`	Application တစ်ခုလုံးရဲ့ Theme
`routes`	Screen များအကြား Navigation အတွက် Routes
`debugShowCheckedModeBanner`	Debug Banner ကို ပြ/မပြ သတ်မှတ်ခြင်း

ဥပမာ —

```
MaterialApp(
  title: 'Shopping App',
  debugShowCheckedModeBanner: false,
  home: const MyHomePage(),
);
```

နောက်ပိုင်း Shopping Application တည်ဆောက်တဲ့အခါ Login Page၊ Product List Page နဲ့ Cart Page တို့ကို စီမံရာမှာ `MaterialApp` ကို အဓိက အသုံးပြုရမှာ ဖြစ်ပါတယ်။

2.6. Scaffold ဆိုတာဘာလဲ?

`Scaffold` ဟာ Material Design ပုံစံ Screen တစ်ခုရဲ့အခြေခံ Layout ကို တည်ဆောက်ပေးတဲ့ Widget ဖြစ်ပါတယ်။

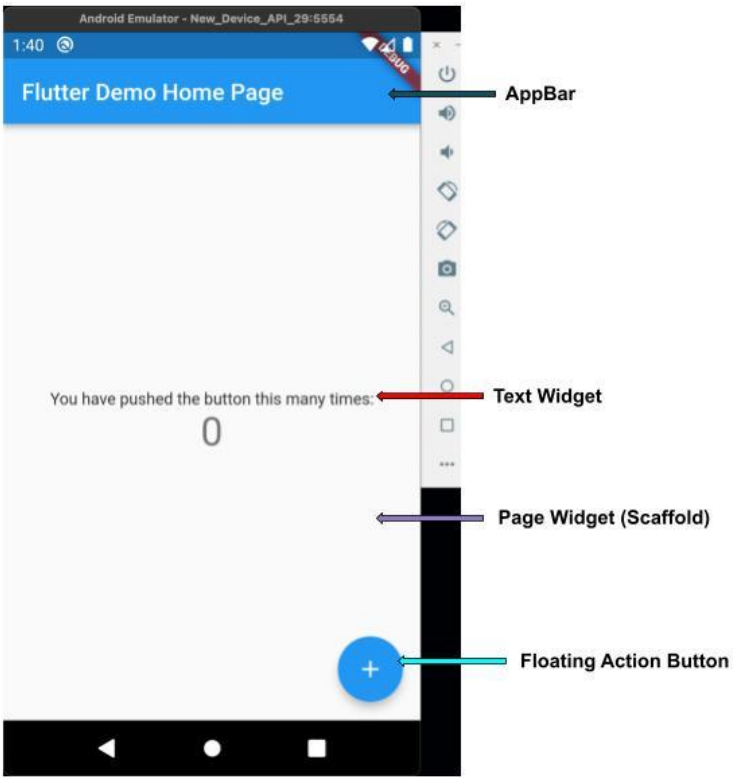


Figure 2.1. Default Layout components

`Scaffold` မှာ အသုံးများတဲ့ Properties များကတော့ —

Property	အသုံးပြုပုံ
`appBar`	Screen အပေါ် ပိုင်း Toolbar
`body`	အဓိက Content
`floatingActionButton`	Screen ပေါ် ရှိ Floating Button
`drawer`	ဘေးဘက် Navigation Menu
`bottomNavigationBar`	Screen အောက်ခြေ Navigation Bar

ဥပမာ —

```
Scaffold(
  appBar: AppBar(
    title: const Text('Home Page'),
  ),
  body: const Center(
    child: Text('Welcome'),
  ),
);
```

ဒီဥပမာမှာ `AppBar` က Screen ရဲ့အပေါ် ပိုင်းမှာ ပေါ်လာပြီး `body` ထဲက `Welcome` စာသားက အလယ်မှာ ပေါ်လာမှာ ဖြစ်ပါတယ်။

2.7. App ကို Android Emulator ပေါ်တွင် Run ခြင်း

Step 1: Emulator စတင်ပါ

Android Studio ကို ဖွင့်ပြီး —

1. `Tools → Device Manager` ကို ရွေးပါ။
2. ရှိပြီးသား Virtual Device တစ်ခုကို ရွေးပါ၊ သို့မဟုတ် Virtual Device အသစ်ဖန်တီးပါ။
3. Emulator ရဲ့ Start ခလုတ်ကို နှိပ်ပါ။
4. Android Home Screen ပေါ်လာတဲ့အထိ စောင့်ပါ။

Step 2: Emulator ကို စစ်ဆေးပါ

VS Code Terminal မှာ —

```
flutter devices
```

Emulator ကို စာရင်းထဲမှာ တွေ့ရပါမယ်။

Step 3: Application Run ပါ

Project Folder (`my_first_app`) အတွင်းမှာ အောက်ပါ Command ကို Run ပါ။

```
flutter run
```

ပထမဆုံး Run တဲ့အခါ Build လုပ်ရတာကြောင့် အချိန်အနည်းငယ် ကြာနိုင်ပါတယ်။
အောင်မြင်သွားပါက Emulator ပေါ်မှာ `My First App` ဆိုတဲ့ AppBar နဲ့ `Hello Flutter!` ဆိုတဲ့ စာသားကို မြင်ရမှာ ဖြစ်ပါတယ်။

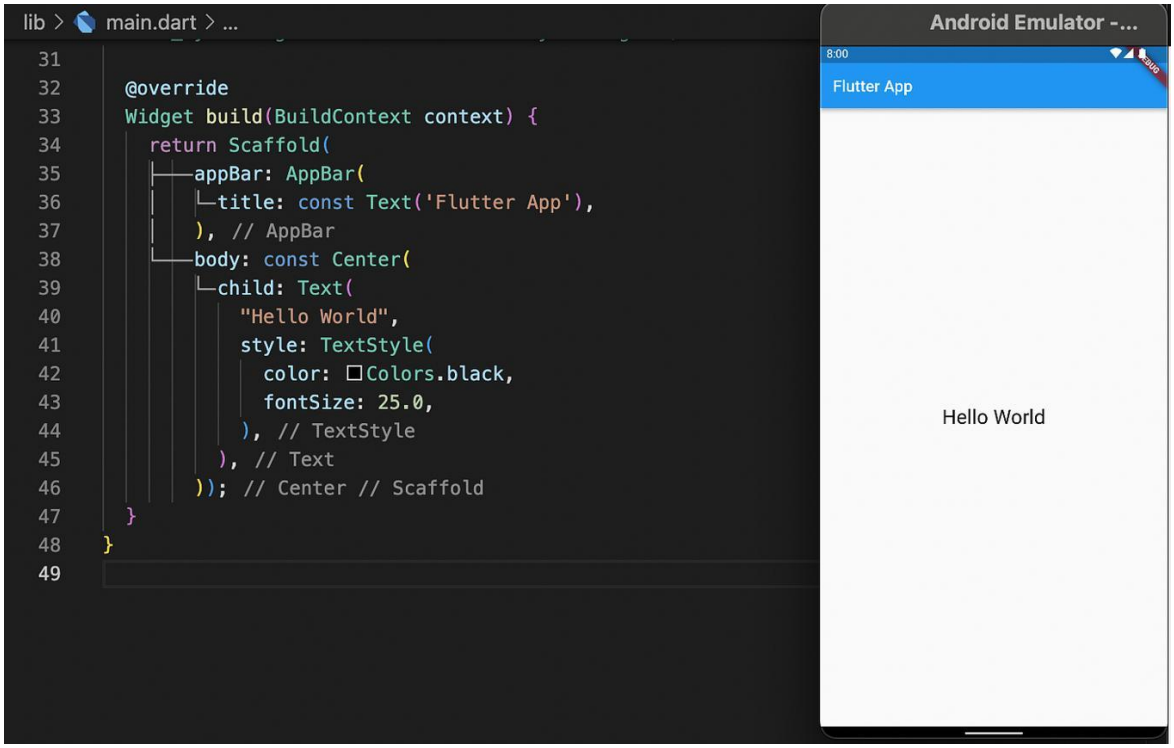


Figure 2.2. First App Run Output Display

Hot Reload အသုံးပြုခြင်း

Application Run နေစဉ် `main.dart` ထဲက Text ကို ပြောင်းလဲပြီး Save လုပ်ကြည့်ပါ။

```
child: Text(
  'Welcome to Flutter!',
  style: TextStyle(fontSize: 28),
),
```

Terminal မှာ Hot Reload လုပ်နိုင်သလို VS Code ရဲ့ Flutter Hot Reload ခလုတ်ကိုလည်း အသုံးပြုနိုင်ပါတယ်။ အများအားဖြင့် App ကို အစအဆုံး ပြန်ဖွင့်စရာမလိုဘဲ UI ပြောင်းလဲမှုကို ချက်ချင်း မြင်ရနိုင်ပါတယ်။

2.8. လက်တွေ့လေ့ကျင့်ခန်း

Lesson 2 Checklist

- my_first_app Project ကို ဖန်တီးပြီးပြီ
- main.dart ထဲက Source Code ကို ရေးပြီးပြီ
- MaterialApp နှင့် Scaffold ကို နားလည်ပြီ
- Emulator ပေါ်မှာ Application ကို Run ပြီးပြီ
- Hello Flutter! ကို Welcome to Flutter! သို့ ပြောင်းပြီးပြီ

Challenge Task

မိမိကိုယ်ပိုင် Profile Application တစ်ခု တည်ဆောက်ကြည့်ပါ။

- AppBar Title ကို `My Profile` ဟု ပြောင်းပါ။
- Body ထဲမှာ မိမိအမည်ကို ဖော်ပြပါ။
- မိမိသင်ယူနေတဲ့ Course အမည်ကို ထည့်ပါ။
- `Icons.person` Icon ကို အသုံးပြုပါ။
- Text နဲ့ Icon တို့ရဲ့ အရောင်နှင့် အရွယ်အစားကို ပြောင်းလဲကြည့်ပါ။

နမူနာ Widget —

```
body: const Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      Icon(
        Icons.person,
        size: 80,
        color: Colors.blue,
      ),
      SizedBox(height: 16),
      Text(
        'My Profile',
```

```
        style: TextStyle(fontSize: 28),
      ),
      Text('Flutter Student'),
    ],
  ),
),
```

2.9. Summary

ဒီသင်ခန်းစာမှာ Flutter Project အသစ်ဖန်တီးခြင်း၊ Folder Structure ကို နားလည်ခြင်း၊ `main.dart` မှာ Source Code ရေးသားခြင်း၊ `MaterialApp` နဲ့ `Scaffold` ကို အသုံးပြုခြင်း၊ Emulator ပေါ်မှာ Application ကို Run ခြင်းတို့ကို လေ့လာခဲ့ပါတယ်။

နောက်သင်ခန်းစာမှာ Lesson 3: Dart Programming Fundamentals for Flutter ကို ဆက်လက်လေ့လာပြီး Variables၊ Data Types၊ Functions၊ Lists၊ Maps နဲ့ Classes များကို Flutter Application နမူနာများဖြင့် လက်တွေ့ရေးသားသွားမှာ ဖြစ်ပါတယ်။

CHAPTER 3
Dart Programming Fundamentals



ဒီ Chapter မှာ Flutter Application တွေ ရေးသားတဲ့အခါ မရှိမဖြစ်လိုအပ်တဲ့ Dart Programming အခြေခံများကို လေ့လာသွားမှာဖြစ်ပါတယ်။ Variables, Data Types, Operators, Conditions, Loops, Functions, Lists, Maps, Classes နဲ့ Null Safety တို့ကို လက်တွေ့ Code Examples တွေနဲ့ ရှင်းပြပေးထားပါတယ်။

3.1 Chapter ၏ ရည်ရွယ်ချက်များ

သင်ခန်းစာပြီးဆုံးတဲ့အခါ အောက်ပါတို့ကို လုပ်ဆောင်နိုင်မှာဖြစ်ပါတယ်။

- Dart Variables နဲ့ Data Types တွေ အသုံးပြုနိုင်ခြင်း
- Conditions နဲ့ Loops တွေ ရေးသားနိုင်ခြင်း
- Functions တွေ ဖန်တီးပြီး ပြန်လည်အသုံးပြုနိုင်ခြင်း
- Lists နဲ့ Maps အသုံးပြုပြီး Product Data တွေ သိမ်းဆည်းနိုင်ခြင်း
- Class နဲ့ Object တွေ ဖန်တီးနိုင်ခြင်း
- Null Safety ကို အသုံးပြုပြီး Error ဖြစ်နိုင်ခြေ လျော့ချနိုင်ခြင်း

3.2. Dart Programming Environment

Flutter SDK တပ်ဆင်ပြီးသားဆိုရင် Dart SDK ကို သီးခြားတပ်ဆင်စရာ မလိုအပ်ပါဘူး။ Terminal မှာ အောက်ပါ Command တွေကို အသုံးပြုပြီး စစ်ဆေးနိုင်ပါတယ်။

```
flutter --version
dart --version
```

Dart Code ကို စမ်းသပ်ဖို့ `ecommerce_app` Project ထဲမှာ `lib` Folder အောက်က `main.dart` ဖိုင်ကို အသုံးပြုနိုင်ပါတယ်။

3.3. Variables and Data Types

Variable ဆိုတာ Data တွေကို ယာယီသိမ်းဆည်းထားဖို့ အသုံးပြုတဲ့ နေရာဖြစ်ပါတယ်။ Dart မှာ `String`, `int`, `double`, `bool`, `var`, `final` နဲ့ `const` တို့ကို အသုံးပြုနိုင်ပါတယ်။

```
void main() {
  String productName = "iPhone 17";
  int quantity = 2;
  double price = 999.99;
  bool isAvailable = true;

  var category = "Mobile Phone";
  final taxRate = 0.05;
  const appName = "Shopping App";

  print(productName);
  print(quantity);
  print(price);
  print(isAvailable);
  print(category);
  print(taxRate);
  print(appName);
}
```

ရှင်းလင်းချက်

- `String` — စာသားတွေ သိမ်းဆည်းပါတယ်။
- `int` — ကိန်းပြည့်တွေ သိမ်းဆည်းပါတယ်။
- `double` — ဒသမကိန်းတွေ သိမ်းဆည်းပါတယ်။

- `bool` — `true` သို့မဟုတ် `false` တန်ဖိုးတွေ သိမ်းဆည်းပါတယ်။
- `var` — ပထမဆုံးသတ်မှတ်တဲ့ တန်ဖိုးအပေါ် မူတည်ပြီး Type ကို သတ်မှတ်ပါတယ်။
- `final` — တန်ဖိုးကို တစ်ကြိမ်သာ သတ်မှတ်နိုင်ပါတယ်။
- `const` — Compile-time Constant အဖြစ် သတ်မှတ်ပါတယ်။

Flutter မှာ Product Name, Price, Quantity နဲ့ Login Status တို့ကို ကိုယ်စားပြုရာမှာ ဒီ Data Types တွေကို အသုံးပြုနိုင်ပါတယ်။

3.4. Operators

Operators တွေကို တွက်ချက်မှု၊ နှိုင်းယှဉ်မှုနဲ့ Logic ဆိုင်ရာ လုပ်ဆောင်ချက်တွေအတွက် အသုံးပြုပါတယ်။

```
void main() {
  double price = 100;
  int quantity = 3;

  double total = price * quantity;
  double discount = 20;
  double finalPrice = total - discount;

  print("Total: $total");
  print("Final Price: $finalPrice");

  print(price > 50);
  print(quantity == 3);

  bool hasStock = true;
  bool isLoggedIn = false;

  print(hasStock && isLoggedIn);
  print(hasStock || isLoggedIn);
  print(!isLoggedIn);
}
```

အဓိက Operators များမှာ -

| Operator | အသုံးပြုပုံ | |---|---| | +, -, *, / | သခ်ၤာတွက်ချက်မှု | | % | အကြွင်းတွက်ချက်မှု | | ==, != | တူညီမှု၊ မတူညီမှု | | >, <, >=, <= | နှိုင်းယှဉ်မှု | | && | အခြေအနေနှစ်ခုလုံး မှန်ရမည် | | || | အခြေအနေတစ်ခုခု မှန်ရမည် | | ! | Boolean တန်ဖိုးကို ပြောင်းပြန်လုပ်ခြင်း |

3.5. Conditions — if, else, switch

Condition တွေကို အသုံးပြုပြီး သတ်မှတ်ထားတဲ့ အခြေအနေအလိုက် မတူညီတဲ့ Code တွေကို လုပ်ဆောင်နိုင်ပါတယ်။

```
void main() {
  double price = 150;
  bool isLoggedIn = true;

  if (price >= 100) {
    print("Expensive Product");
  } else {
    print("Affordable Product");
  }

  if (isLoggedIn) {
    print("Welcome to Shopping App");
  } else {
    print("Please Login");
  }

  int orderStatus = 2;

  switch (orderStatus) {
    case 1:
      print("Pending");
      break;
    case 2:
      print("Processing");
      break;
    case 3:
      print("Delivered");
      break;
    default:
      print("Unknown Status");
  }
}
```

Shopping Application မှာ Login ဝင်ထားခြင်း ရှိ၊ မရှိ စစ်ဆေးခြင်း၊ Product Stock စစ်ဆေးခြင်းနဲ့ Order Status ပြသခြင်းတို့အတွက် Conditions တွေကို အသုံးပြုနိုင်ပါတယ်။

3.6. Loops — for, while, for-in

Loops တွေက Code တစ်ခုကို အကြိမ်ကြိမ် လုပ်ဆောင်စေဖို့ အသုံးပြုပါတယ်။

```
void main() {
  // for loop
  for (int i = 1; i <= 5; i++) {
    print("Product $i");
  }

  // while loop
```

```
  int count = 1;

  while (count <= 3) {
    print("Count: $count");
    count++;
  }

  // for-in loop
  List<String> products = [
    "Phone",
    "Laptop",
    "Tablet"
  ];

  for (String product in products) {
    print(product);
  }
}
```

Product List ထဲက ပစ္စည်းတစ်ခုချင်းစီကို ဖော်ပြတဲ့အခါ `for-in` Loop က ရိုးရှင်းပြီး ဖတ်ရလွယ်ကူပါတယ်။

3.7. Functions

Function ဆိုတာ သီးခြားလုပ်ဆောင်ချက်တစ်ခုကို စုစည်းရေးသားထားတဲ့ Code အပိုင်းဖြစ်ပါတယ်။ Function တွေကို လိုအပ်တဲ့နေရာတိုင်းမှာ ပြန်လည်အသုံးပြုနိုင်ပါတယ်။

```
double calculateTotal(double price, int quantity) {
  return price * quantity;
}

void showWelcome(String name) {
  print("Welcome, $name!");
}

void main() {
  showWelcome("Mg Mg");

  double total = calculateTotal(50, 3);

  print("Total Price: $total");
}
```

`calculateTotal()` က Price နဲ့ Quantity ကို လက်ခံပြီး Total Price ကို ပြန်ပေးပါတယ်။

`showWelcome()` ကတော့ User Name ကို လက်ခံပြီး Welcome Message ကို ပြသပါတယ်။

Function တွေကို အသုံးပြုခြင်းအားဖြင့် Shopping Cart တွက်ချက်မှု၊ Discount တွက်ချက်မှုနဲ့ Product Search လုပ်ဆောင်ချက်တွေကို သီးခြားစီ ရေးသားနိုင်ပါတယ်။

3.8. Lists

List ဆိုတာ Data အများအပြားကို အစဉ်လိုက် သိမ်းဆည်းနိုင်တဲ့ Collection တစ်မျိုးဖြစ်ပါတယ်။

```
void main() {
  List<String> products = [
    "iPhone",
    "Samsung",
    "Xiaomi"
  ];

  // Read
  print(products[0]);

  // Add
  products.add("Oppo");

  // Update
  products[1] = "Samsung Galaxy";

  // Delete
  products.remove("Xiaomi");

  // Display all products
  for (String product in products) {
    print(product);
  }

  print("Total Products: ${products.length}");
}
```

Dart မှာ List Index က `0` ကနေ စတင်ပါတယ်။ ဒါကြောင့် `products[0]` က ပထမဆုံး Product ကို ရည်ညွှန်းပါတယ်။

Shopping Application ရဲ့ Product List, Cart Items နဲ့ Order Items တွေကို သိမ်းဆည်းရာမှာ Lists တွေကို အသုံးပြုနိုင်ပါတယ်။

3.9. Maps

Map ဆိုတာ Key နဲ့ Value ကို တွဲဖက်သိမ်းဆည်းထားတဲ့ Collection ဖြစ်ပါတယ်။ Product တစ်ခုရဲ့ `id`, `name`, `price` စတဲ့ အချက်အလက်တွေကို သိမ်းဆည်းဖို့ အသုံးဝင်ပါတယ်။

```
void main() {
  Map<String, dynamic> product = {
    "id": 1,
    "name": "iPhone",
    "price": 999.99,
    "inStock": true,
  };

  // Read
  print(product["name"]);
  print(product["price"]);

  // Add
  product["category"] = "Mobile";

  // Update
  product["price"] = 899.99;

  // Display
  print(product);

  // Delete
  product.remove("category");
}
```

`Map<String, dynamic>` မှာ Key က `String` ဖြစ်ပြီး Value က မတူညီတဲ့ Data Types တွေ ဖြစ်နိုင်ပါတယ်။ `dynamic` ကို လွယ်ကူစွာ အသုံးပြုနိုင်ပေမဲ့ Type Safety အတွက် လိုအပ်တဲ့နေရာမှာ သတ်မှတ်ထားတဲ့ Data Type ကို အသုံးပြုတာ ပိုကောင်းပါတယ်။

3.10. Classes and Objects

Class ဆိုတာ Object တွေ ဖန်တီးဖို့ အသုံးပြုတဲ့ ပုံစံတစ်ခုဖြစ်ပါတယ်။ Product Class ထဲမှာ Product ရဲ့ ID, Name နဲ့ Price တို့ကို သတ်မှတ်နိုင်ပါတယ်။

```
class Product {
  int id;
  String name;
  double price;

  Product({
    required this.id,
    required this.name,
    required this.price,
  });

  void display() {
    print("ID: $id");
    print("Name: $name");
    print("Price: $price");
  }
```

```
}

void main() {
  Product product1 = Product(
    id: 1,
    name: "iPhone",
    price: 999.99,
  );

  Product product2 = Product(
    id: 2,
    name: "Laptop",
    price: 1299.99,
  );

  product1.display();
  product2.display();
}
```

အဓိက မှတ်သားရန်

- `class Product` — Product Class ကို ဖန်တီးခြင်း
- `Product(...)` — Constructor မှတစ်ဆင့် Object ဖန်တီးခြင်း
- `required` — Object ဖန်တီးတဲ့အခါ မဖြစ်မနေ ထည့်ပေးရမယ့် Parameter
- `this.id` — လက်ရှိ Object ရဲ့ `id` Property ကို ရည်ညွှန်းခြင်း
- `display()` — Product အချက်အလက်တွေကို ပြသတဲ့ Method

Flutter Application မှာ Product, User, CartItem နဲ့ Order စတဲ့ Data Model တွေကို Class တွေနဲ့ သတ်မှတ်ရေးသားနိုင်ပါတယ်။

3.11. Null Safety

Null Safety ဆိုတာ Variable တစ်ခုမှာ Value မရှိတဲ့ `null` တန်ဖိုးကို ခွင့်ပြုမပြု ကြိုတင်သတ်မှတ်ပေးတဲ့ Dart Feature ဖြစ်ပါတယ်။

```
void main() {
  String name = "Mg Mg";

  // Nullable variable
  String? phoneNumber;

  phoneNumber = "09123456789";

  print(name);
  print(phoneNumber);
```

```
  // Null-aware operator
  String? address;

  print(address ?? "Address not provided");

  // Null check
  if (address != null) {
    print(address.length);
  }

  // Null assertion
  String? email = "user@example.com";

  print(email!.length);
}
```

Null Safety Operators

Syntax	အဓိပ္ပာယ်
`String name`	`null` မဖြစ်ရတဲ့ String
`String? name`	`null` ဖြစ်နိုင်တဲ့ String
`value ?? defaultValue`	Value က null ဖြစ်ရင် Default Value အသုံးပြုခြင်း
`value?.length`	Null မဟုတ်မှ Property ကို အသုံးပြုခြင်း
`value!`	Value က Null မဟုတ်ဘူးလို့ အတည်ပြုခြင်း

`!` Operator ကို သတိထားပြီး အသုံးပြုရပါတယ်။ Value က တကယ် `null` ဖြစ်နေခဲ့ရင် Runtime Error ဖြစ်နိုင်ပါတယ်။ ပုံမှန်အားဖြင့် `??`, `?.` နဲ့ `if (value != null)` တို့ကို ဦးစားပေးအသုံးပြုသင့်ပါတယ်။

3.12. Mini Project — Product List Application

ယခုအပိုင်းမှာ သင်ယူခဲ့တဲ့ Variables, Lists, Classes, Loops နဲ့ Functions တွေကို ပေါင်းစပ်ပြီး Console ပေါ်မှာ Product List တစ်ခု ဖော်ပြကြည့်ပါမယ်။

`lib/main.dart` ထဲမှာ အောက်ပါ Code ကို ထည့်သွင်းပါ။

```
class Product {
```

```
  final int id;
  final String name;
  final double price;

  Product({
    required this.id,
    required this.name,
    required this.price,
  });
}

double calculateTotal(List<Product> products) {
  double total = 0;

  for (final product in products) {
    total += product.price;
  }

  return total;
}

void main() {
  final List<Product> products = [
    Product(id: 1, name: "Phone", price: 500),
    Product(id: 2, name: "Laptop", price: 1200),
    Product(id: 3, name: "Headphones", price: 80),
  ];

  print("=== Shopping Product List ===");

  for (final product in products) {
    print(
      "${product.id}. ${product.name} - \$${product.price}",
    );
  }

  final total = calculateTotal(products);

  print("----------------------------");
  print("Total Products: ${products.length}");
  print("Total Price: \$${total}");
}
```

Run လုပ်ရန်

Terminal ကို ဖွင့်ပြီး အောက်ပါ Command ကို အသုံးပြုပါ။

```
dart run lib/main.dart
```

မျှော်မှန်းထားတဲ့ Output က အောက်ပါအတိုင်း ဖြစ်ပါတယ်။

```
=== Shopping Product List ===
1. Phone - $500.0
```

```
2. Laptop - $1200.0
3. Headphones - $80.0
---------------------------
Total Products: 3
Total Price: $1780.0
```

ဒီ Mini Project မှာ Product Class ကို အသုံးပြုပြီး Product Objects တွေ ဖန်တီးထားပါတယ်။ List ထဲမှာ သိမ်းဆည်းထားတဲ့ Product တွေကို Loop နဲ့ ဖော်ပြပြီး Function တစ်ခုကနေ Total Price ကို တွက်ချက်ထားတာ ဖြစ်ပါတယ်။

3.13. လေ့ကျင့်ခန်းများ

Practice Exercises

1. Student Class တစ်ခု ဖန်တီးပြီး id, name, age Properties များ ထည့်ပါ။
2. Product List ထဲက Price $100 ထက်နည်းသော Product များကိုသာ ဖော်ပြပါ။
3. Product List ထဲသို့ Product အသစ်တစ်ခု Add လုပ်ပါ။
4. Product တစ်ခုချင်းစီ၏ Price ကို Quantity နှင့် မြှောက်ပြီး Total တွက်ပါ။
5. Nullable String Variable တစ်ခု ဖန်တီးပြီး Null ဖြစ်လျှင် Default Value ပြပါ။

3.14. Summary

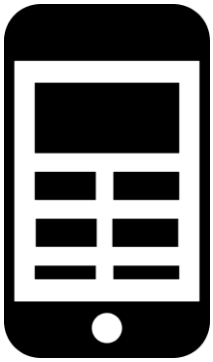
ဒီသင်ခန်းစာမှာ Dart Programming ရဲ့အခြေခံအစိတ်အပိုင်းတွေကို လေ့လာခဲ့ပါတယ်။ အထူးသဖြင့် Classes, Lists နဲ့ Functions တွေဟာ Flutter Shopping Application ရေးသားရာမှာ အရေးကြီးပါတယ်။ Null Safety ကို နားလည်ထားခြင်းကလည်း Application ရဲ့ Stability ကို ပိုမိုကောင်းမွန်စေပါတယ်။

နောက်သင်ခန်းစာ — Lesson 4: Flutter Widgets and Responsive UI

နောက်သင်ခန်းစာမှာ `StatelessWidget`, `StatefulWidget`, `Container`, `Row`, `Column`, `Image`, `Text`, `Button` နဲ့ `ListView` တို့ကို အသုံးပြုပြီး Shopping Application ရဲ့ Product UI ကို လက်တွေ့တည်ဆောက်သွားမှာဖြစ်ပါတယ်။

CHAPTER 4

Flutter Widgets and Application Layouts



ဒီ Chapter မှာ Flutter Application တစ်ခုရဲ့ UI ကို တည်ဆောက်ရာမှာ အရေးကြီးတဲ့ `StatelessWidget`, `StatefulWidget`, `Text`, `Container`, `Row`, `Column`, `Stack`, `Padding`, `Expanded` နဲ့ `Widget Tree` တို့ကို လေ့လာသွားမှာ ဖြစ်ပါတယ်။

သင်ခန်းစာအဆုံးမှာ Shopping Application တစ်ခုအတွက် Product Card UI ကို ကိုယ်တိုင်ရေးသားနိုင်မှာ ဖြစ်ပါတယ်။

4.1. Flutter Widget ဆိုတာဘာလဲ?

Flutter မှာ Application ရဲ့ User Interface (UI) ကို တည်ဆောက်ဖို့ အသုံးပြုတဲ့ အခြေခံအစိတ်အပိုင်းကို Widget လို့ ခေါ်ပါတယ်။

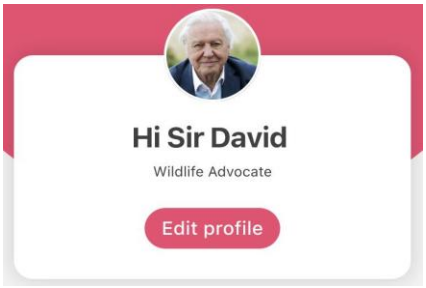
ဥပမာ -

- `Text` — စာသားဖော်ပြရန်
- `Container` — အရွယ်အစား၊ နောက်ခံအရောင်နဲ့ Decoration သတ်မှတ်ရန်
- `Row` — Widget များကို အလျားလိုက် စီရန်

- `Column` — Widget များကို ဒေါင်လိုက် စီရန်
- `Padding` — Widget ပတ်လည် နေရာလွတ်ထားရန်
- `Expanded` — ရရှိနိုင်တဲ့ နေရာကို အချိုးကျ ဖြန့်ဝေရန်
- `Stack` — Widget များကို အပေါ်အောက် ထပ်တင်ရန်

Flutter UI တစ်ခုမှာ Widget တစ်ခုအတွင်း အခြား Widget များကို ထည့်သွင်းပြီး တည်ဆောက်လေ့ရှိပါတယ်။

4.2. StatelessWidget နှင့် StatefulWidget



StatelessWidget

UI တည်ဆောက်ရာမှာ ကိုယ်ပိုင်ပြောင်းလဲနိုင်တဲ့ State မလိုအပ်တဲ့ Widget ဖြစ်ပါတယ်။ စာသား၊ Logo၊ ပုံနဲ့ အမြဲတမ်းတူညီနေတဲ့ Layout များအတွက် အသုံးပြုနိုင်ပါတယ်။

ဥပမာ - App Title, Product Name, Heading



```
class _CounterWidgetState extends State<CounterWidget> {
  int _count = 0; // State variable to hold the current count

  void _incrementCount() {
    setState(() {
      _count++;
    });
  }

  void _decrementCount() {
    setState(() {
      _count--;
    });
  }

  @override
  Widget build(BuildContext context) {
    // Widget build logic goes here
  }
}
```

StatefulWidget

User က Button နှိပ်ခြင်း၊ Counter တိုးခြင်း စတဲ့ လုပ်ဆောင်ချက်များကြောင့် UI ပြောင်းလဲဖို့ လိုအပ်တဲ့အခါ အသုံးပြုပါတယ်။

ဥပမာ - Counter, Like Button, Shopping Cart Quantity

Example 1 — StatelessWidget

`lib/main.dart` ဖိုင်ကို အောက်ပါအတိုင်း ရေးသားပါ။

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Flutter Lesson 4'),
        ),
        body: const Center(
          child: Text(
            'Hello Flutter!',
            style: TextStyle(fontSize: 24),
          ),
        ),
      ),
    );
  }
}
```

ရှင်းလင်းချက်

- `MyApp` က `StatelessWidget` ကို အမွေဆက်ခံထားပါတယ်။
- `build()` က UI Widget များကို ပြန်ပေးပါတယ်။
- `MaterialApp` က Material Design Application တစ်ခုအတွက် အခြေခံဖြစ်ပါတယ်။
- `Scaffold` က AppBar နဲ့ Body အတွက် Layout ပေးပါတယ်။
- `Center` က အတွင်းရှိ Widget ကို အလယ်မှာ နေရာချထားပါတယ်။

Example 2 — StatefulWidget

Counter တစ်ခုကို Button နှိပ်တိုင်း တိုးသွားအောင် တည်ဆောက်ကြည့်ပါ။

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(
    home: CounterPage(),
```

```
    debugShowCheckedModeBanner: false,
  ));
}

class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int count = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Counter App')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              'Count: $count',
              style: const TextStyle(fontSize: 28),
            ),
            const SizedBox(height: 16),
            ElevatedButton(
              onPressed: () {
                setState(() {
                  count++;
                });
              },
              child: const Text('Increase'),
            ),
          ],
        ),
      ),
    );
  }
}
```

မှတ်သားရန်

`setState()` ကို ခေါ်လိုက်တဲ့အခါ Flutter က လိုအပ်တဲ့ UI ကို ပြန်လည်တည်ဆောက်ပေးပါတယ်။ ဒါကြောင့် Button နှိပ်လိုက်တာနဲ့ `count` တန်ဖိုးအသစ်ကို မြင်ရမှာ ဖြစ်ပါတယ်။

4.3. အခြေခံ Widgets များ

4.3.1 Text Widget

စာသားဖော်ပြဖို့ အသုံးပြုပါတယ်။

```
const Text(
  'Welcome to Flutter',
  textAlign: TextAlign.center,
  style: TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
    color: Colors.blue,
  ),
)
```

အသုံးများတဲ့ Properties တွေကတော့ `fontSize`, `fontWeight`, `color`, `textAlign` နဲ့ `maxLines` တို့ ဖြစ်ပါတယ်။

4.3.2 Container Widget

Widget တစ်ခုအတွက် အကျယ်၊ အမြင့်၊ နောက်ခံအရောင်၊ Border နဲ့ Margin စတာတွေ သတ်မှတ်နိုင်ပါတယ်။

```
Container(
  width: 200,
  height: 100,
  alignment: Alignment.center,
  padding: const EdgeInsets.all(12),
  margin: const EdgeInsets.all(10),
  decoration: BoxDecoration(
    color: Colors.blue,
    borderRadius: BorderRadius.circular(12),
  ),
  child: const Text(
    'Product Box',
    style: TextStyle(color: Colors.white),
  ),
)
```

- `width`, `height` — အရွယ်အစား
- `padding` — အတွင်းပိုင်း နေရာလွတ်
- `margin` — အပြင်ပိုင်း နေရာလွတ်
- `decoration` — နောက်ခံ၊ Border နဲ့ ထောင့်ဝိုင်းပုံစံ

4.3.3 Padding Widget

အတွင်းရှိ Widget ပတ်လည်မှာ နေရာလွတ်ထားနိုင်ပါတယ်။

```
Padding(
  padding: const EdgeInsets.all(16),
  child: const Text('Hello World'),
)
```

တစ်ဖက်စီ သတ်မှတ်လိုပါက `EdgeInsets.only(left: 10, top: 20)` ကို အသုံးပြုနိုင်ပါတယ်။

4.3.4 Row Widget

Widget များကို ဘယ်မှညာသို့ အလျားလိုက် စီပေးပါတယ်။

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: const [
    Icon(Icons.home),
    Icon(Icons.search),
    Icon(Icons.person),
  ],
)
```

`mainAxisAlignment` က Row အတွက် အလျားလိုက် နေရာချထားမှုကို ထိန်းချုပ်ပါတယ်။

4.3.5 Column Widget

Widget များကို အပေါ်မှအောက်သို့ ဒေါင်လိုက် စီပေးပါတယ်။

```
Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: const [
    Text('Product Name'),
    Text('Price: \$25'),
    Text('In Stock'),
  ],
)
```

Column အတွက် `mainAxisAlignment` က ဒေါင်လိုက်နေရာချထားမှုကို ထိန်းချုပ်ပြီး `crossAxisAlignment` က အလျားလိုက် နေရာချထားမှုကို ထိန်းချုပ်ပါတယ်။

4.3.6 Expanded Widget

`Row` သို့မဟုတ် `Column` ထဲမှာ ကျန်ရှိနေတဲ့ နေရာကို Widget များအကြား ဖြန့်ဝေဖို့ အသုံးပြုပါတယ်။

```
Row(
  children: [
    Expanded(
      flex: 2,
      child: Container(
        height: 80,
        color: Colors.blue,
      ),
    ),
    Expanded(
      flex: 1,
      child: Container(
        height: 80,
        color: Colors.orange,
      ),
    ),
  ],
)
```

ဒီဥပမာမှာ ပထမ Container က `flex: 2` ဖြစ်ပြီး ဒုတိယ Container က `flex: 1` ဖြစ်တဲ့အတွက် ရရှိနိုင်တဲ့ အကျယ်ကို ၂:၁ အချိုးဖြင့် ဖြန့်ဝေပေးပါတယ်။

4.3.7 Stack Widget

Widget တစ်ခုအပေါ်မှာ အခြား Widget တစ်ခုကို ထပ်တင်ချင်တဲ့အခါ အသုံးပြုပါတယ်။

```
Stack(
  children: [
    Container(
      width: 200,
      height: 150,
      color: Colors.blue,
    ),
    const Positioned(
      top: 10,
      right: 10,
      child: Icon(
        Icons.favorite,
        color: Colors.red,
        size: 30,
      ),
    ),
    const Positioned(
      bottom: 10,
      left: 10,
      child: Text(
        'Featured',
```

```
          style: TextStyle(color: Colors.white),
        ),
      ),
    ],
)
```

`Positioned` က Stack အတွင်းမှာ Widget ရဲ့တည်နေရာကို သတ်မှတ်ပေးပါတယ်။ Product Image ပေါ်မှာ Favorite Icon သို့မဟုတ် Discount Label တင်ဖို့ အသုံးဝင်ပါတယ်။

4.4. Widget Tree ဆိုတာဘာလဲ?

Widget Tree ဆိုတာ UI ထဲမှာ Widget တစ်ခုနဲ့တစ်ခု ဘယ်လို အဆင့်ဆင့် ချိတ်ဆက်ထားသလဲဆိုတာကို ဖော်ပြတဲ့ ဖွဲ့စည်းပုံ ဖြစ်ပါတယ်။

ဥပမာ -

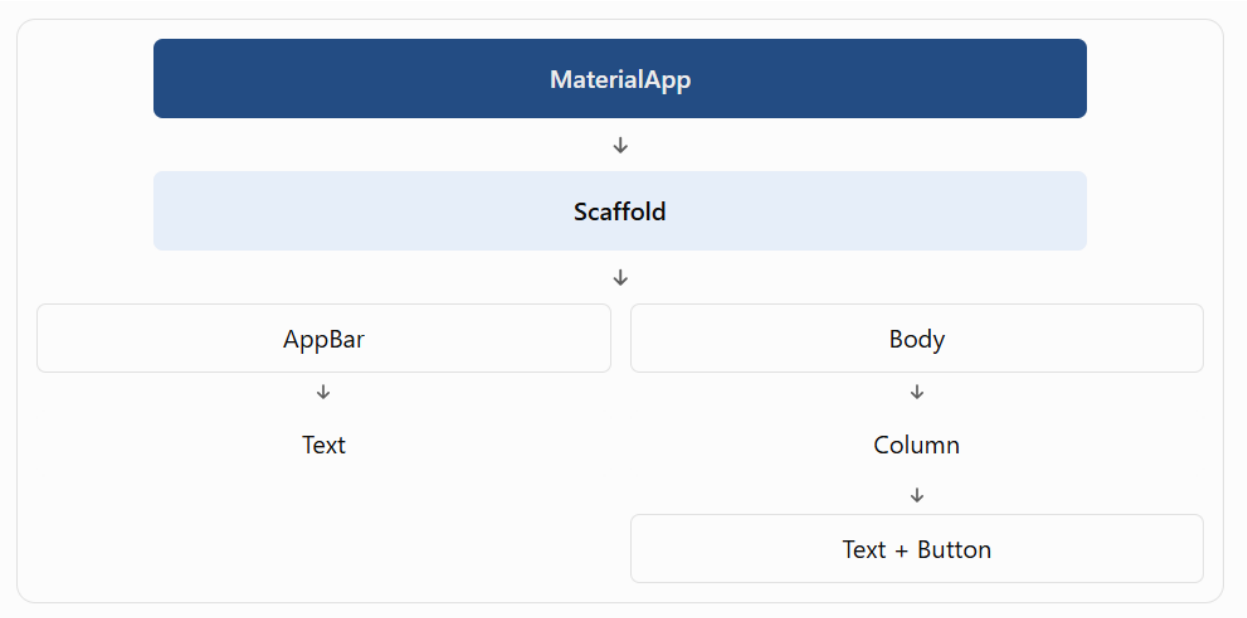


Figure 4.1. Wiget Tree

Flutter Code ထဲမှာ `child` ဆိုတာ ပုံမှန်အားဖြင့် Widget တစ်ခုကို လက်ခံပြီး `children` ဆိုတာ Widget များစွာကို List အဖြစ် လက်ခံပါတယ်။

```
Center(
  child: Column(
    children: const [
      Text('Name'),
      Text('Price'),
    ],
  ),
)
```

ဒီမှာ `Center` က Parent Widget ဖြစ်ပြီး `Column` က သူ့ရဲ့ Child Widget ဖြစ်ပါတယ်။ `Column` မှာတော့ `Text` Widget နှစ်ခုကို ထပ်မံထည့်သွင်းထားပါတယ်။

4.5. လက်တွေ့ Project — Shopping Product Card

ဒီအပိုင်းမှာ လေ့လာခဲ့တဲ့ Widgets တွေကို ပေါင်းစပ်ပြီး Shopping Application အတွက် Product Card တစ်ခု တည်ဆောက်ပါမယ်။

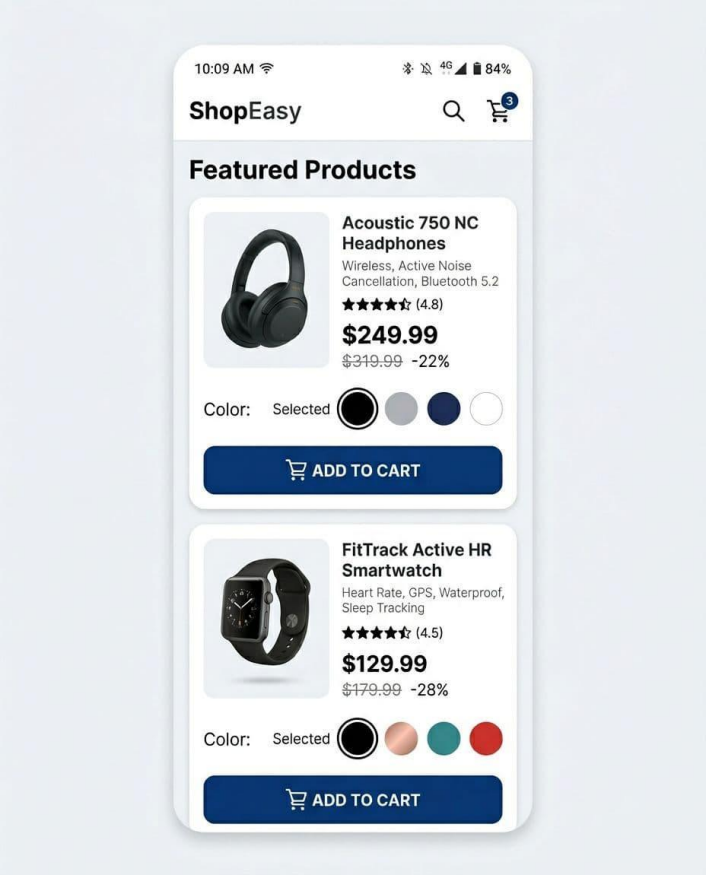


Figure 4.2. Shopping Product Card

အောက်ပါ Code ကို `lib/main.dart` ထဲမှာ ထည့်သွင်းပြီး Run လုပ်ကြည့်ပါ။

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Shopping App',
      home: const ProductPage(),
    );
  }
}
```

```
class ProductPage extends StatefulWidget {
  const ProductPage({super.key});

  @override
  State<ProductPage> createState() => _ProductPageState();
}

class _ProductPageState extends State<ProductPage> {
  int quantity = 1;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('My Shopping App'),
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Stack(
              children: [
                Container(
                  height: 220,
                  width: double.infinity,
                  decoration: BoxDecoration(
                    color: Colors.blue.shade50,
                    borderRadius: BorderRadius.circular(16),
                  ),
                  child: const Icon(
                    Icons.smartphone,
                    size: 120,
                    color: Colors.blue,
                  ),
                ),
                const Positioned(
                  top: 12,
                  right: 12,
                  child: Icon(
                    Icons.favorite_border,
                    color: Colors.red,
                    size: 30,
                  ),
                ),
              ],
            ),
            const SizedBox(height: 20),
            const Text(
              'Smartphone Pro',
              style: TextStyle(
                fontSize: 24,
                fontWeight: FontWeight.bold,
              ),
            ),
```

```
const SizedBox(height: 8),
const Text(
  'Price: \$599.00',
  style: TextStyle(
    fontSize: 20,
    color: Colors.green,
  ),
),
const SizedBox(height: 16),
Row(
  children: [
    const Text(
      'Quantity:',
      style: TextStyle(fontSize: 16),
    ),
    const SizedBox(width: 12),
    IconButton(
      onPressed: () {
        if (quantity > 1) {
          setState(() {
            quantity--;
          });
        }
      },
      icon: const Icon(Icons.remove_circle_outline),
    ),
    Text(
      '$quantity',
      style: const TextStyle(fontSize: 18),
    ),
    IconButton(
      onPressed: () {
        setState(() {
          quantity++;
        });
      },
      icon: const Icon(Icons.add_circle_outline),
    ),
  ],
),
const Spacer(),
Row(
  children: [
    Expanded(
      child: ElevatedButton.icon(
        onPressed: () {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(
              content: Text(
                '$quantity item(s) added to cart',
              ),
            ),
          );
        },
        icon: const Icon(Icons.shopping_cart),
        label: const Text('Add to Cart'),
      ),
```

```
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

Code ထဲမှာ အသုံးပြုထားတဲ့ Concepts များ

Widget	အသုံးပြုပုံ
`StatefulWidget`	Quantity ကို ပြောင်းလဲရန်
`Scaffold`	App ရဲ့အဓိက Layout
`Padding`	Screen အနားမှ နေရာလွတ်ထားရန်
`Column`	Product အချက်အလက်များကို ဒေါင်လိုက်စီရန်
`Stack`	Product ပုံပေါ် Favorite Icon တင်ရန်
`Row`	Quantity Controls နဲ့ Button များကို အလျားလိုက်စီရန်
`Expanded`	Add to Cart Button ကို အကျယ်ဖြန့်ရန်
`setState()`	Quantity ပြောင်းလဲမှုကို UI မှာ ဖော်ပြရန်

မှတ်ချက်။ ဒီ Project က UI နဲ့ Quantity ပြောင်းလဲမှုကို လေ့ကျင့်ဖို့ ဖြစ်ပါတယ်။ Cart Data ကို Database ထဲ သိမ်းဆည်းခြင်း၊ Login နဲ့ Payment လုပ်ဆောင်ချက်တွေကို နောက်သင်ခန်းစာတွေမှာ ဆက်လက်တည်ဆောက်နိုင်ပါတယ်။

4.6. လေ့ကျင့်ခန်းများ

Practice Checklist

- Exercise 1 — Text Styling
  Text Widget တစ်ခုကို Font Size 30၊ အပြာရောင်နဲ့ Bold ဖြစ်အောင် ရေးပါ။
- Exercise 2 — Row and Column
  Student Name၊ Course နဲ့ Age ကို Column ထဲမှာ ဖော်ပြပြီး Icon ၃ ခုကို Row ထဲမှာ စီပါ။
- Exercise 3 — Container Design

- Width 250၊ Height 150၊ Background Color Orange နဲ့ Border Radius 15 ရှိတဲ့ Container တည်ဆောက်ပါ။
- Exercise 4 — StatefulWidget
  Counter App တစ်ခု ဖန်တီးပြီး Increase နဲ့ Decrease Button များ ထည့်ပါ။ Counter က 0 အောက် မကျစေရပါ။
- Exercise 5 — Product Card
  Product Name၊ Price၊ Quantity Controls နဲ့ Add to Cart Button ပါတဲ့ Product Card တစ်ခု ဖန်တီးပါ။

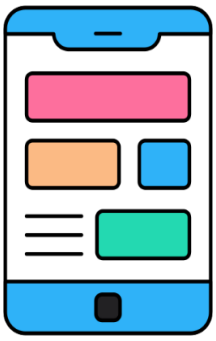
4.7. Summary

ဒီ Chapter မှာ Flutter UI တည်ဆောက်ရာမှာ အသုံးများတဲ့ Widgets တွေနဲ့ Widget Tree ဖွဲ့စည်းပုံကို လေ့လာခဲ့ပါတယ်။ `StatelessWidget` နဲ့ `StatefulWidget` ရဲ့ကွာခြားချက်၊ `Row` နဲ့ `Column` အသုံးပြုပုံ၊ `Stack` နဲ့ `Positioned` ဖြင့် Widget များကို ထပ်တင်ပုံ၊ `Expanded` ဖြင့် နေရာဖြန့်ဝေပုံတို့ကို နားလည်ထားဖို့ အရေးကြီးပါတယ်။

နောက်သင်ခန်းစာ Lesson 5: Flutter Responsive UI Design and Styling မှာ Screen Size အမျိုးမျိုးအတွက် UI ပြင်ဆင်ခြင်း၊ `MediaQuery`, `LayoutBuilder`, Theme, Colors, Fonts နဲ့ Reusable Widgets များကို ဆက်လက်လေ့လာနိုင်ပါတယ်။

CHAPTER 5

UI Design, Themes and Responsive Layouts



ဒီ Chapter မှာ Flutter Application တစ်ခုရဲ့ UI ကို လှပပြီး အသုံးပြုရလွယ်ကူအောင် တည်ဆောက်နည်း၊ Color၊ Font၊ Image၊ Icon၊ Material Design၊ Theme နှင့် `MediaQuery` ကို အသုံးပြုပြီး မတူညီတဲ့ Screen Size များအတွက် Responsive UI တည်ဆောက်နည်းတွေကို လေ့လာသွားမှာ ဖြစ်ပါတယ်။

5.1. Chapter ၏ ရွယ်ချက်များ

ဒီသင်ခန်းစာပြီးဆုံးတဲ့အခါ အောက်ပါအကြောင်းအရာတွေကို လက်တွေ့အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

- Flutter မှာ Color နှင့် Font များ သတ်မှတ်နိုင်ခြင်း။
- Image နှင့် Icon များ ထည့်သွင်းအသုံးပြုနိုင်ခြင်း။
- Material Design UI တည်ဆောက်နိုင်ခြင်း။
- Application တစ်ခုလုံးအတွက် Theme သတ်မှတ်နိုင်ခြင်း။
- `MediaQuery` ဖြင့် Screen Width နှင့် Height ရယူနိုင်ခြင်း။
- Mobile၊ Tablet နှင့် Desktop Screen Size များအတွက် Responsive UI တည်ဆောက်နိုင်ခြင်း။

5.2. Flutter Project တည်ဆောက်ခြင်း

Lesson 2 မှာ တည်ဆောက်ခဲ့တဲ့ Project ကို ဆက်လက်အသုံးပြုနိုင်ပါတယ်။ Project အသစ်နဲ့ စမ်းသပ်ချင်ရင် Terminal မှာ အောက်ပါ Command ကို ရိုက်ထည့်ပါ။

```
flutter create shopping_ui
cd shopping_ui
code .
```

အဓိက ပြင်ဆင်ရမယ့် File ကတော့ `lib/main.dart` ဖြစ်ပါတယ်။

5.3. Colors အသုံးပြုခြင်း

Flutter မှာ `Colors` class ကို အသုံးပြုပြီး Text၊ Background၊ Button နဲ့ အခြား UI Components များအတွက် အရောင်တွေ သတ်မှတ်နိုင်ပါတယ်။

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        backgroundColor: Colors.grey.shade100,
        appBar: AppBar(
          title: const Text('Color Example'),
          backgroundColor: Colors.blue,
          foregroundColor: Colors.white,
        ),
        body: Center(
          child: Container(
            padding: const EdgeInsets.all(24),
            decoration: BoxDecoration(
              color: Colors.blue.shade100,
              borderRadius: BorderRadius.circular(16),
            ),
            child: const Text(
              'Welcome to Flutter',
              style: TextStyle(
```

```
                color: Colors.blue,
                fontSize: 24,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

အရေးကြီးတဲ့ Properties များ

Property	ရှင်းလင်းချက်
`color`	Widget ရဲ့အရောင်
`backgroundColor`	နောက်ခံအရောင်
`Colors.blue`	သတ်မှတ်ထားတဲ့ အပြာရောင်
`Colors.grey.shade100`	မီးခိုးဖျော့ရောင်
`BorderRadius.circular(16)`	ထောင့်ဝိုင်းအရွယ်အစား

မိမိစိတ်ကြိုက် Hex Color အသုံးပြုချင်ရင် `Color(0xFF6750A4)` ကဲ့သို့ သတ်မှတ်နိုင်ပါတယ်။ `FF` က Opacity အပြည့်ကို ကိုယ်စားပြုပါတယ်။

5.4. Fonts နှင့် Text Styling

`TextStyle` ကို အသုံးပြုပြီး စာလုံးအရွယ်အစား၊ အရောင်၊ အလေးချိန်နှင့် အခြား Text ပုံစံတွေကို ပြင်ဆင်နိုင်ပါတယ်။

```
const Text(
  'Featured Products',
  style: TextStyle(
    fontSize: 26,
    fontWeight: FontWeight.bold,
    color: Color(0xFF263238),
    letterSpacing: 0.5,
  ),
```

```
)
```

အသုံးများတဲ့ Text Properties များကတော့ -

- `fontSize` — စာလုံးအရွယ်အစား
- `fontWeight` — စာလုံးအထူအပါး
- `color` — စာလုံးအရောင်
- `letterSpacing` — စာလုံးတစ်လုံးနဲ့တစ်လုံးကြား အကွာအဝေး
- `textAlign` — စာသားနေရာချထားမှု
- `maxLines` — ဖော်ပြမယ့် စာကြောင်းအများဆုံးအရေအတွက်

ဥပမာ -

```
const Text(
  'This is a shopping application.',
  textAlign: TextAlign.center,
  maxLines: 2,
  overflow: TextOverflow.ellipsis,
  style: TextStyle(
    fontSize: 16,
    color: Colors.black87,
  ),
)
```

Custom Font အသုံးပြုလိုပါက Font File ကို `assets/fonts/` ထဲထည့်ပြီး `pubspec.yaml` မှာ သတ်မှတ်နိုင်ပါတယ်။ မြန်မာစာများအတွက်လည်း မြန်မာ Unicode ကို မှန်ကန်စွာ ဖော်ပြနိုင်တဲ့ Font ကို ရွေးချယ်ပါ။

5.5. Images အသုံးပြုခြင်း

Flutter မှာ Image ကို Local Assets မှဖြစ်စေ၊ Network URL မှဖြစ်စေ ဖော်ပြနိုင်ပါတယ်။

5.5.1 Local Image

Project ထဲမှာ အောက်ပါ Folder Structure ကို တည်ဆောက်ပါ။

```
shopping_ui/
├── assets/
│   └── images/
```

```
│       └── product.jpg
├── lib/
│   └── main.dart
└── pubspec.yaml
```

`pubspec.yaml` မှာ Assets ကို မှတ်ပုံတင်ပါ။

```
flutter:
  uses-material-design: true

  assets:
    - assets/images/
```

ထို့နောက် Terminal မှာ Run ပါ။

```
flutter pub get
```

Image ကို ဖော်ပြရန် -

```
Image.asset(
  'assets/images/product.jpg',
  width: 200,
  height: 200,
  fit: BoxFit.cover,
)
```

5.5.2 Network Image

```
Image.network(
  'https://picsum.photos/400/300',
  width: 250,
  height: 180,
  fit: BoxFit.cover,
  errorBuilder: (context, error, stackTrace) {
    return const Icon(
      Icons.broken_image,
      size: 60,
    );
  },
)
```

`BoxFit.cover` က သတ်မှတ်ထားတဲ့ နေရာအတွင်း ပုံကို ပြည့်အောင် ဖော်ပြပေးပါတယ်။ Shopping Application တွေမှာ Product Images အတွက် မကြာခဏ အသုံးပြုပါတယ်။

5.6. Icons အသုံးပြုခြင်း

Flutter ရဲ့ Built-in Material Icons တွေကို `Icon` Widget ဖြင့် အသုံးပြုနိုင်ပါတယ်။

```
const Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Icon(
      Icons.shopping_cart,
      color: Colors.blue,
      size: 32,
    ),
    SizedBox(width: 12),
    Icon(
      Icons.favorite,
      color: Colors.red,
      size: 32,
    ),
    SizedBox(width: 12),
    Icon(
      Icons.person,
      color: Colors.green,
      size: 32,
    ),
  ],
)
```

အသုံးများတဲ့ Icons များ -

Icon	အသုံးပြုမှု
`Icons.home`	Home Page
`Icons.shopping_cart`	Shopping Cart
`Icons.search`	Search
`Icons.favorite`	Wishlist
`Icons.person`	User Profile
`Icons.menu`	Navigation Menu
`Icons.notifications`	Notifications

Icon ကို နှိပ်လို့ရတဲ့ Button အဖြစ် အသုံးပြုချင်ရင် `IconButton` ကို အသုံးပြုပါ။

```
IconButton(
  onPressed: () {
    debugPrint('Cart button clicked');
  },
  icon: const Icon(Icons.shopping_cart),
  tooltip: 'Shopping Cart',
)
```

5.7. Material Design

Material Design က Google ရဲ့ Design System ဖြစ်ပြီး Flutter မှာ `MaterialApp`, `Scaffold`, `AppBar`, `Card`, `ElevatedButton` နဲ့ အခြား Widgets များကို အသုံးပြုပြီး တည်ဆောက်နိုင်ပါတယ်။

ဥပမာ -

```
Scaffold(
  appBar: AppBar(
    title: const Text('My Shop'),
  ),
  body: Padding(
    padding: const EdgeInsets.all(16),
    child: Card(
      elevation: 3,
      child: ListTile(
        leading: const Icon(Icons.shopping_bag),
        title: const Text('Product Name'),
        subtitle: const Text('Price: \$25.00'),
        trailing: ElevatedButton(
          onPressed: () {},
          child: const Text('Buy'),
        ),
      ),
    ),
  ),
)
```

ဒီဥပမာမှာ `Scaffold` က Page Layout ကို ပံ့ပိုးပေးပြီး `Card` က Product အချက်အလက်တွေကို သီးခြား Card ပုံစံနဲ့ ဖော်ပြပေးပါတယ်။

5.8. Application Theme သတ်မှတ်ခြင်း

Application တစ်ခုမှာ Page တိုင်းအတွက် အရောင်နဲ့ Button ပုံစံတွေကို သီးခြားသတ်မှတ်နေရင် Code များလာနိုင်ပါတယ်။ `ThemeData` ကို အသုံးပြုရင် Application တစ်ခုလုံးရဲ့ Design ကို တစ်နေရာတည်းကနေ စီမံနိုင်ပါတယ်။

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ),
        scaffoldBackgroundColor: Colors.grey.shade50,
        appBarTheme: const AppBarTheme(
          centerTitle: true,
        ),
        cardTheme: CardThemeData(
          elevation: 2,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(16),
          ),
        ),
      ),
      home: const HomePage(),
    );
  }
}

class HomePage extends StatelessWidget {
  const HomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('My Shopping App'),
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () {},
          child: const Text('Explore Products'),
        ),
      ),
    );
  }
}
```

မှတ်ချက်။ ဒီဥပမာမှာ အသုံးပြုထားတဲ့ `CardThemeData` ဟာ လက်ရှိ Flutter SDK တွေမှာ သင့်တော်တဲ့ Theme Type ဖြစ်ပါတယ်။ SDK Version အဟောင်းတွေမှာ `CardTheme` ကို အသုံးပြုထားနိုင်တာကြောင့် Compile Error ရှိရင် မိမိ Flutter Version နဲ့ ကိုက်ညီအောင် ပြင်ဆင်ပါ။

5.9. MediaQuery ဖြင့် Screen Size ရယူခြင်း

Responsive UI တည်ဆောက်ဖို့ Screen ရဲ့ Width နဲ့ Height ကို သိဖို့လိုပါတယ်။ `MediaQuery` က လက်ရှိ App Window ရဲ့ Screen Information တွေကို ရယူပေးပါတယ်။

```
@override
Widget build(BuildContext context) {
  final screenWidth = MediaQuery.sizeOf(context).width;
  final screenHeight = MediaQuery.sizeOf(context).height;

  return Scaffold(
    body: Center(
      child: Text(
        'Width: ${screenWidth.toStringAsFixed(0)} px\n'
        'Height: ${screenHeight.toStringAsFixed(0)} px',
        textAlign: TextAlign.center,
      ),
    ),
  );
}
```

အဓိကအသုံးပြုရမယ့် Properties များ -

- `MediaQuery.sizeOf(context).width` — App Window ရဲ့ အကျယ်
- `MediaQuery.sizeOf(context).height` — App Window ရဲ့ အမြင့်
- `MediaQuery.paddingOf(context)` — System UI ကြောင့် အသုံးမပြုနိုင်တဲ့ Insets
- `MediaQuery.textScalerOf(context)` — User သတ်မှတ်ထားတဲ့ Text Scaling

သတိပြုရန်။ Screen Width ကို ရယူနိုင်တာနဲ့ UI က အလိုအလျောက် Responsive ဖြစ်သွားတာ မဟုတ်ပါဘူး။ ရရှိလာတဲ့ Width အလိုက် Widget Size၊ Column၊ Grid စတာတွေကို ပြောင်းလဲစီမံပေးရမှာ ဖြစ်ပါတယ်။

5.10. Responsive Layout တည်ဆောက်ခြင်း

Mobile၊ Tablet နဲ့ Desktop တွေမှာ Screen Width မတူညီတဲ့အတွက် Product တွေကို ဖော်ပြပုံ ပြောင်းလဲပေးနိုင်ပါတယ်။

ဥပမာ -

- Width 600 px အောက် — တစ်တန်းမှာ Product 2 ခု

- Width 600 px မှ 900 px — တစ်တန်းမှာ Product 3 ခု
- Width 900 px နှင့်အထက် — တစ်တန်းမှာ Product 4 ခု

`GridView.builder` နဲ့ `MediaQuery` ကို ပေါင်းစပ်အသုံးပြုနိုင်ပါတယ်။

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ),
      ),
      home: const ProductPage(),
    );
  }
}

class ProductPage extends StatelessWidget {
  const ProductPage({super.key});

  final List<Map<String, dynamic>> products = const [
    {
      'name': 'Smartphone',
      'price': 299,
      'icon': Icons.phone_android,
    },
    {
      'name': 'Laptop',
      'price': 799,
      'icon': Icons.laptop,
    },
    {
      'name': 'Headphones',
      'price': 59,
      'icon': Icons.headphones,
    },
    {
      'name': 'Smart Watch',
      'price': 99,
      'icon': Icons.watch,
    },
```

```
    {
      'name': 'Camera',
      'price': 450,
      'icon': Icons.camera_alt,
    },
    {
      'name': 'Speaker',
      'price': 89,
      'icon': Icons.speaker,
    },
  ];

  @override
  Widget build(BuildContext context) {
    final width = MediaQuery.sizeOf(context).width;

    final columns = width < 600
        ? 2
        : width < 900
            ? 3
            : 4;

    return Scaffold(
      appBar: AppBar(
        title: const Text('My Shop'),
        actions: [
          IconButton(
            onPressed: () {},
            icon: const Icon(Icons.shopping_cart),
          ),
        ],
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text(
              'Featured Products',
              style: TextStyle(
                fontSize: 24,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 16),
            Expanded(
              child: GridView.builder(
                itemCount: products.length,
                gridDelegate:
                    SliverGridDelegateWithFixedCrossAxisCount(
                  crossAxisCount: columns,
                  crossAxisSpacing: 12,
                  mainAxisSpacing: 12,
                  childAspectRatio: 0.8,
                ),
                itemBuilder: (context, index) {
                  final product = products[index];
```

```
        return Card(
          clipBehavior: Clip.antiAlias,
          child: Padding(
            padding: const EdgeInsets.all(12),
            child: Column(
              mainAxisAlignment:
                  MainAxisAlignment.center,
              children: [
                Icon(
                  product['icon'] as IconData,
                  size: 52,
                  color: Theme.of(context)
                      .colorScheme.primary,
                ),
                const SizedBox(height: 12),
                Text(
                  product['name'] as String,
                  textAlign: TextAlign.center,
                  maxLines: 2,
                  overflow: TextOverflow.ellipsis,
                  style: const TextStyle(
                    fontWeight: FontWeight.bold,
                  ),
                ),
                const SizedBox(height: 8),
                Text(
                  '\$${product['price']}',
                  style: const TextStyle(
                    fontSize: 18,
                    color: Colors.green,
                  ),
                ),
                const SizedBox(height: 8),
                SizedBox(
                  width: double.infinity,
                  child: ElevatedButton(
                    onPressed: () {
                      ScaffoldMessenger.of(context)
                          .showSnackBar(
                        SnackBar(
                          content: Text(
                            '${product['name']} added',
                          ),
                        ),
                      );
                    },
                    child: const Text('Add'),
                  ),
                ),
              ],
            ),
          ),
        );
      },
    ),
  ),
```

```
          ],
        ),
      ),
    );
  }
}
```

Responsive UI ရဲ့ အဓိက Logic

```
final width = MediaQuery.sizeOf(context).width;

final columns = width < 600
    ? 2
    : width < 900
        ? 3
        : 4;
```

ဒီ Code မှာ Screen Width အလိုက် Grid Column အရေအတွက်ကို သတ်မှတ်ထားပါတယ်။ ဒါကြောင့် Screen ကျဉ်းတဲ့အခါ Product Card အနည်းငယ်စီသာ ပြသပြီး Screen ကျယ်လာတဲ့အခါ တစ်တန်းတည်းမှာ Product ပိုများများ ပြသနိုင်ပါတယ်။

ပိုမိုရှုပ်ထွေးတဲ့ UI တွေမှာ `LayoutBuilder` ကို အသုံးပြုခြင်းကလည်း ကောင်းမွန်ပါတယ်။ အထူးသဖြင့် Widget တစ်ခုရဲ့ Parent က ပေးထားတဲ့ Width အလိုက် Layout ပြောင်းလဲချင်တဲ့အခါ အသုံးဝင်ပါတယ်။

5.11. Light Theme နှင့် Dark Theme

Flutter မှာ Light Theme နဲ့ Dark Theme နှစ်မျိုးလုံးကို သတ်မှတ်နိုင်ပါတယ်။

```
MaterialApp(
  theme: ThemeData(
    brightness: Brightness.light,
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.blue,
      brightness: Brightness.light,
    ),
  ),
  darkTheme: ThemeData(
    brightness: Brightness.dark,
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.blue,
      brightness: Brightness.dark,
    ),
  ),
  themeMode: ThemeMode.system,
```

```
  home: const ProductPage(),
)
```

`ThemeMode.system` ကို သတ်မှတ်ထားရင် အသုံးပြုသူရဲ့ Device Theme Setting အတိုင်း Light သို့မဟုတ် Dark Mode ကို ရွေးချယ်ပေးပါတယ်။ `ThemeMode.light` နဲ့ `ThemeMode.dark` ကို အသုံးပြုပြီး Mode တစ်မျိုးတည်းကိုလည်း သတ်မှတ်နိုင်ပါတယ်။

5.12. လက်တွေ့လေ့ကျင့်ခန်းများ

အောက်ပါ Exercises တွေကို ကိုယ်တိုင် ပြင်ဆင်စမ်းသပ်ကြည့်ပါ။

Exercise Checklist

1. AppBar ရဲ့ Background Color ကို Deep Purple ပြောင်းပါ။
2. Product Card တစ်ခုစီမှာ Product Image ထည့်ပါ။
3. Product Name နဲ့ Price အတွက် TextStyle သီးခြားသတ်မှတ်ပါ။
4. Screen Width 500 px အောက်မှာ Product တစ်တန်းလျှင် 1 ခုသာ ပြသပါ။
5. Light Theme နဲ့ Dark Theme ကို ပြောင်းလဲအသုံးပြုနိုင်အောင် ပြုလုပ်ပါ။
6. Add Button ကို နှိပ်တဲ့အခါ Product Name ပါဝင်တဲ့ SnackBar ပြပါ။

5.13. Summary

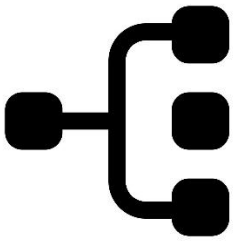
ဒီသင်ခန်းစာမှာ Flutter UI Design အတွက် Colors၊ Fonts၊ Images၊ Icons၊ Material Design၊ `ThemeData`၊ `MediaQuery` နဲ့ Responsive Grid Layout များကို လေ့လာခဲ့ပါတယ်။

Shopping Application တစ်ခု တည်ဆောက်တဲ့အခါ UI ဟာ လှပရုံသာမက Screen Size မတူညီတဲ့ Device များမှာလည်း မှန်ကန်စွာ အလုပ်လုပ်ဖို့ လိုအပ်ပါတယ်။ ဒါကြောင့် `MediaQuery` နဲ့ `LayoutBuilder` တို့ကို နားလည်ထားခြင်းက နောက်ပိုင်း Product List၊ Product Details၊ Shopping Cart နဲ့ Checkout Page တွေ တည်ဆောက်ရာမှာ အထောက်အကူပြုမှာ ဖြစ်ပါတယ်။

နောက်သင်ခန်းစာ — Chapter 6: Navigation and Multiple Screens in Flutter

ဒီသင်ခန်းစာမှာ `Navigator`, `MaterialPageRoute`, `push()`, `pop()`, Screen များအကြား Data ပေးပို့ခြင်းနဲ့ Shopping Application အတွက် Home Page၊ Product Details Page တို့ကို ချိတ်ဆက်တည်ဆောက်သွားမှာ ဖြစ်ပါတယ်။

CHAPTER 6
Navigation and Routing



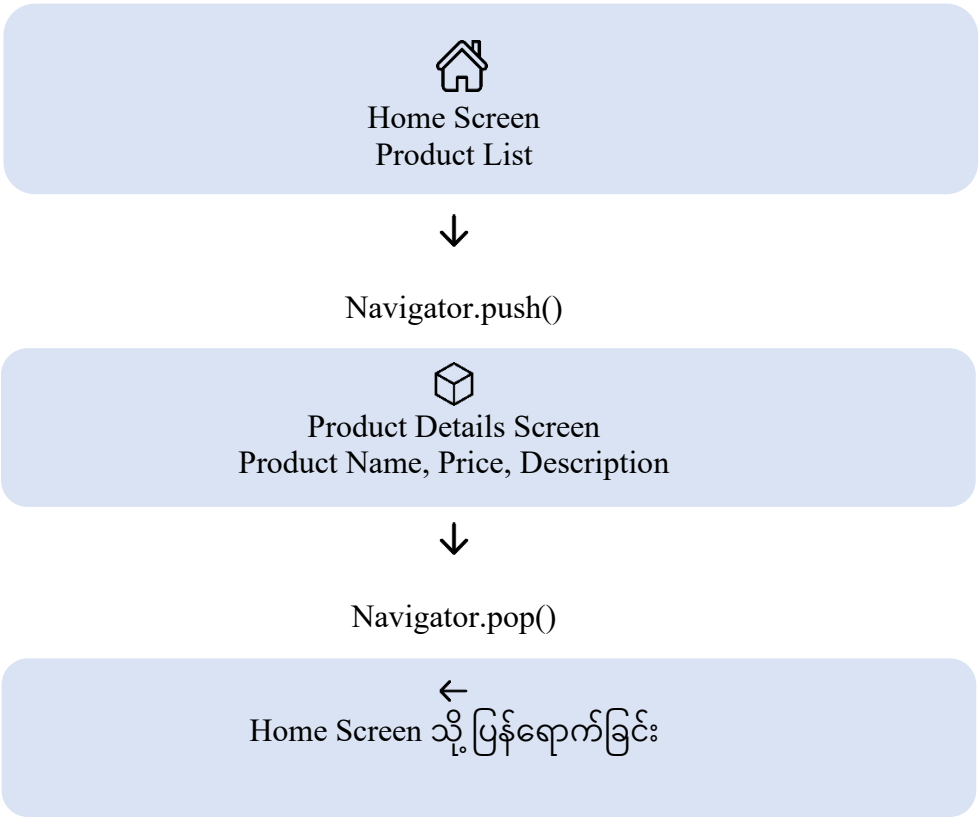
Flutter Application တစ်ခုတွင် Home Screen မှ Product Details Screen၊ Login Screen၊ Cart Screen နှင့် အခြား Screen များသို့ ပြောင်းလဲသွားလာနိုင်ရန် Navigation and Routing ကို အသုံးပြုရပါတယ်။ ဒီသင်ခန်းစာမှာ `Navigator`, `Routes`, `Passing Arguments` နဲ့ `Back Navigation` တို့ကို လက်တွေ့ Code Example များဖြင့် လေ့လာသွားမှာ ဖြစ်ပါတယ်။

6.1. Chapter ၏ရည်ရွယ်ချက်များ

ဒီသင်ခန်းစာပြီးဆုံးတဲ့အခါ အောက်ပါအကြောင်းအရာတွေကို လုပ်ဆောင်နိုင်မှာ ဖြစ်ပါတယ်။

- `Navigator.push()` ဖြင့် Screen အသစ်သို့ သွားခြင်း
- `Navigator.pop()` ဖြင့် ယခင် Screen သို့ ပြန်သွားခြင်း
- Named Routes အသုံးပြုခြင်း
- Screen တစ်ခုမှ တစ်ခုသို့ Arguments ပေးပို့ခြင်း
- Screen တစ်ခုမှ Result ပြန်လည်ရယူခြင်း
- Shopping Application အတွက် Product Details Navigation တည်ဆောက်ခြင်း

6.2. Flutter Navigation အလုပ်လုပ်ပုံ



Flutter ရဲ့ Navigator ဟာ Screen တွေကို Stack ပုံစံနဲ့ စီမံခန့်ခွဲပေးပါတယ်။ Screen အသစ်ကို `push()` လုပ်တဲ့အခါ Stack ပေါ်ကို ထပ်ထည့်ပြီး `pop()` လုပ်တဲ့အခါ လက်ရှိ Screen ကို ဖယ်ရှားကာ အရင် Screen သို့ ပြန်သွားစေပါတယ်။

6.3. Navigator.push() နှင့် Navigator.pop()

ပထမဆုံး Flutter Project တစ်ခု ဖန်တီးပါ။

```
flutter create navigation_app
cd navigation_app
code .
```

`lib/main.dart` ဖိုင်ကို အောက်ပါ Code ဖြင့် အစားထိုးပါ။

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Navigation Lesson',
      home: const HomeScreen(),
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Home Screen'),
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) =>
                    const DetailsScreen(),
              ),
            );
          },
          child: const Text('Open Details'),
        ),
      ),
    );
  }
}

class DetailsScreen extends StatelessWidget {
  const DetailsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Details Screen'),
```

```
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.pop(context);
          },
          child: const Text('Go Back'),
        ),
      ),
    );
  }
}
```

Code ရှင်းလင်းချက်

- `Navigator.push()` က Details Screen ကို ဖွင့်ပေးပါတယ်။
- `MaterialPageRoute()` က Screen ပြောင်းလဲမှုအတွက် Route တစ်ခု ဖန်တီးပေးပါတယ်။
- `builder` က ပြသမယ့် Widget ကို သတ်မှတ်ပေးပါတယ်။
- `Navigator.pop(context)` က လက်ရှိ Screen ကို ပိတ်ပြီး Home Screen သို့ ပြန်သွားစေပါတယ်။

Run လုပ်ရန် -

```
flutter run
```

6.4. Named Routes အသုံးပြုခြင်း

Application ကြီးလာတဲ့အခါ Screen တစ်ခုချင်းစီကို `MaterialPageRoute` ဖြင့် တိုက်ရိုက်သတ်မှတ်မယ့်အစား Route Name များ သတ်မှတ်ပြီး အသုံးပြုနိုင်ပါတယ်။

`MaterialApp` ထဲမှာ `routes` ကို သတ်မှတ်ပေးပါ။

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      initialRoute: '/',
      routes: {
        '/': (context) => const HomeScreen(),
        '/details': (context) => const DetailsScreen(),
        '/login': (context) => const LoginScreen(),
      },
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Home')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton(
              onPressed: () {
                Navigator.pushNamed(context, '/details');
              },
              child: const Text('Product Details'),
            ),
            ElevatedButton(
              onPressed: () {
                Navigator.pushNamed(context, '/login');
              },
              child: const Text('Login'),
            ),
          ],
        ),
      ),
    );
  }
}

class DetailsScreen extends StatelessWidget {
  const DetailsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Product Details')),
      body: const Center(
        child: Text('Product Details Screen'),
      ),
    );
  }
}
```

```
class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: const Center(
        child: Text('Login Screen'),
      ),
    );
  }
}
```

အဓိက Methods များ

Method	အသုံးပြုပုံ
`Navigator.push()`	Widget Route အသစ်သို့ သွားခြင်း
`Navigator.pushNamed()`	Route Name ဖြင့် သွားခြင်း
`Navigator.pop()`	လက်ရှိ Route ကို ပိတ်ခြင်း
`Navigator.pushReplacementNamed()`	လက်ရှိ Route ကို အစားထိုးခြင်း
`Navigator.pushNamedAndRemoveUntil()`	သတ်မှတ်ထားသော Route များကို ဖယ်ရှားပြီး သွားခြင်း

မှတ်ချက်။ Named Routes သည် ရိုးရှင်းသော Application များအတွက် အသုံးဝင်ပါတယ်။ ရှုပ်ထွေးတဲ့ Navigation Flow၊ Deep Links နဲ့ URL-based routing များအတွက် `go_router` ကဲ့သို့သော Package ကိုလည်း စဉ်းစားနိုင်ပါတယ်။

6.5. Passing Arguments — Screen တစ်ခုမှ တစ်ခုသို့ Data ပေးပို့ခြင်း

Shopping Application မှာ Product List ထဲက Product တစ်ခုကို နှိပ်လိုက်တဲ့အခါ Product ID၊ Name နဲ့ Price ကို Details Screen သို့ ပေးပို့ဖို့ လိုအပ်ပါတယ်။

Home Screen မှ Arguments ပေးပို့ခြင်း

```
Navigator.pushNamed(
```

```
  context,
  '/details',
  arguments: {
    'id': 1,
    'name': 'Wireless Headphones',
    'price': 45.99,
  },
);
```

Details Screen မှ Arguments လက်ခံခြင်း

`DetailsScreen` ရဲ့ `build()` Method ထဲမှာ -

```
final product =
    ModalRoute.of(context)!.settings.arguments
        as Map<String, dynamic>;

final id = product['id'];
final name = product['name'];
final price = product['price'];
```

အောက်ပါအတိုင်း UI မှာ ပြသနိုင်ပါတယ်။

```
body: Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      Text('Product ID: $id'),
      Text('Product Name: $name'),
      Text('Price: \$${price.toStringAsFixed(2)}'),
    ],
  ),
),
```

ဒီနည်းလမ်းနဲ့ Product တစ်ခုချင်းစီရဲ့ အချက်အလက်တွေကို Details Screen မှာ ပြသနိုင်ပါတယ်။ လက်တွေ့ Project ကြီးတွေမှာတော့ `Product` Model Class တစ်ခု ဖန်တီးပြီး Type-safe ဖြစ်အောင် Data ပေးပို့တာက ပိုမိုကောင်းမွန်ပါတယ်။

6.6. Back Navigation နှင့် Result ပြန်ယူခြင်း

Screen တစ်ခုကနေ တစ်ခုသို့ သွားပြီး ပြန်လာတဲ့အခါ Result ကိုလည်း ရယူနိုင်ပါတယ်။

```
final result = await Navigator.push<String>(
```

```
  context,
  MaterialPageRoute(
    builder: (context) => const SelectColorScreen(),
  ),
);

if (!context.mounted) return;

if (result != null) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Selected: $result')),
  );
}
```

နောက် Screen မှာ Result ပြန်ပို့ရန် -

```
Navigator.pop(context, 'Blue');
```

ဒီ Example မှာ `SelectColorScreen` က `'Blue'` ဆိုတဲ့ String ကို ပြန်ပို့ပေးပြီး ယခင် Screen က `result` Variable မှတစ်ဆင့် လက်ခံရရှိမှာ ဖြစ်ပါတယ်။

Back Navigation ပြုလုပ်ရာမှာ AppBar ရဲ့ Back Button၊ Android ရဲ့ System Back Button နဲ့ `Navigator.pop()` တို့ကို အသုံးပြုနိုင်ပါတယ်။ သို့သော် Route Stack မှာ ပြန်သွားစရာ Screen မရှိတဲ့အခါ `pop()` မလုပ်သင့်ပါဘူး။

6.7. လက်တွေ့လေ့ကျင့်ခန်းများ

လေ့ကျင့်ခန်း Checklist

1. Exercise 1 — Home & Details
2. Home Screen နဲ့ Details Screen တည်ဆောက်ပြီး Button နှိပ်တဲ့အခါ Screen ပြောင်းလဲအောင် လုပ်ပါ။
3. Exercise 2 — Named Routes
4. Home, Login, Register နဲ့ Product Screen များကို Route Name ဖြင့် ချိတ်ဆက်ပါ။
5. Exercise 3 — Product Arguments
6. Product ID, Name, Price နဲ့ Description တို့ကို Details Screen သို့ ပေးပို့ပြီး ပြသပါ။
7. Exercise 4 — Back Result
8. Color Selection Screen တည်ဆောက်ပြီး ရွေးချယ်ထားသော Color Name ကို Home Screen သို့ ပြန်ပို့ပါ။

6.8. Summary

ဒီသင်ခန်းစာမှာ Flutter Navigation ရဲ့ အခြေခံသဘောတရားတွေ၊ `Navigator.push()`, `Navigator.pop()`, Named Routes၊ Arguments ပေးပို့ခြင်းနဲ့ Result ပြန်ယူခြင်းတို့ကို လေ့လာခဲ့ပါတယ်။ ဒီအကြောင်းအရာတွေကို နားလည်ထားရင် Shopping Application ထဲမှာ Home၊ Product Details၊ Login၊ Register နဲ့ Cart Screen များကို ချိတ်ဆက်တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။

နောက်သင်ခန်းစာ — Chapter 7: Forms and User Input

`TextField`, `TextFormField`, Form Validation၊ User Input Handling နဲ့ Login/Register Form များကို လက်တွေ့တည်ဆောက်သွားမှာ ဖြစ်ပါတယ်။

CHAPTER 7

Forms and User Input Validation



ဒီ Chapter မှာ Flutter Shopping Application အတွက် အသုံးပြုသူထံမှ အချက်အလက်များ လက်ခံရယူခြင်း၊ Form တည်ဆောက်ခြင်း၊ Input Validation ပြုလုပ်ခြင်းနှင့် Dropdown၊ Checkbox များ အသုံးပြုပုံကို လက်တွေ့လေ့လာသွားမှာ ဖြစ်ပါတယ်။

သင်ခန်းစာအဆုံးမှာ Product Form တစ်ခု တည်ဆောက်ပြီး Product Name၊ Price၊ Category၊ Description နှင့် Stock Availability တို့ကို ထည့်သွင်းစစ်ဆေးနိုင်မှာ ဖြစ်ပါတယ်။

7.1. Chapter ၏ရည်ရွယ်ချက်များ

- `TextField` နှင့် `TextFormField` အသုံးပြုခြင်း
- `TextEditingController` ဖြင့် Input Value များ ရယူခြင်း
- `Form` နှင့် `GlobalKey<FormState>` အသုံးပြုခြင်း
- `validator` ဖြင့် မဖြစ်မနေဖြည့်ရမည့် Field များ စစ်ဆေးခြင်း
- `DropdownButtonFormField` ဖြင့် Category ရွေးချယ်ခြင်း
- `Checkbox` ဖြင့် Stock Availability သတ်မှတ်ခြင်း
- Form Submit နှင့် Reset လုပ်ခြင်း

7.2. Flutter Form အလုပ်လုပ်ပုံ

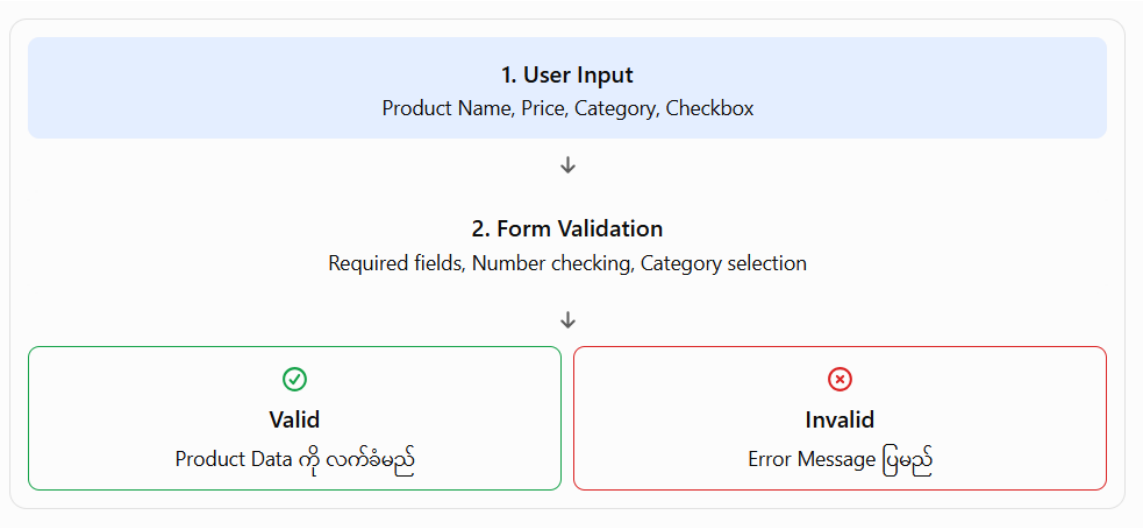


Figure 7.1. Flutter Form အလုပ်လုပ်ပုံ

7.3. TextField နှင့် TextEditingController

`TextField` သည် အသုံးပြုသူက စာသားများ ရိုက်ထည့်နိုင်ရန် အသုံးပြုပါတယ်။
`TextEditingController` ကတော့ ထည့်သွင်းထားသော စာသားများကို ဖတ်ခြင်း၊ ပြင်ဆင်ခြင်းနှင့် ရှင်းလင်းခြင်းတို့ ပြုလုပ်နိုင်စေပါတယ်။

ဥပမာ — Product Name ကို ရယူခြင်း။

```
import 'package:flutter/material.dart';

class ProductInputPage extends StatefulWidget {
  const ProductInputPage({super.key});

  @override
  State<ProductInputPage> createState() =>
      _ProductInputPageState();
}

class _ProductInputPageState
    extends State<ProductInputPage> {
  final TextEditingController nameController =
      TextEditingController();
```

```
  @override
  void dispose() {
    nameController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Product Input')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: nameController,
              decoration: const InputDecoration(
                labelText: 'Product Name',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 16),
            ElevatedButton(
              onPressed: () {
                final name = nameController.text.trim();

                ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(
                    content: Text('Product: $name'),
                  ),
                );
              },
              child: const Text('Get Product Name'),
            ),
          ],
        ),
      ),
    );
  }
}
```

မှတ်သားရန်

- `nameController.text` ဖြင့် ထည့်ထားသော Value ကို ဖတ်နိုင်ပါတယ်။
- `trim()` ဖြင့် ရှေ့နှင့်နောက်ရှိ အပို Space များကို ဖယ်ရှားနိုင်ပါတယ်။
- `nameController.clear()` ဖြင့် Input ကို ရှင်းနိုင်ပါတယ်။
- `dispose()` ဖြင့် Controller ကို အသုံးမလိုတော့သည့်အခါ ရှင်းလင်းပေးရပါတယ်။

7.4. Form, GlobalKey နှင့် Validators

`Form` သည် Input Field အများအပြားကို တစ်စုတစ်စည်းတည်း စီမံခန့်ခွဲပေးပါတယ်။
`GlobalKey<FormState>` ဖြင့် Form State ကို ရယူပြီး `validate()` ခေါ်သုံးနိုင်ပါတယ်။

`TextFormField` မှာ `validator` သတ်မှတ်ထားရင် Form Validate လုပ်သည့်အခါ Error Message ပြသနိုင်ပါတယ်။

```
final formKey = GlobalKey<FormState>();

Form(
  key: formKey,
  child: TextFormField(
    decoration: const InputDecoration(
      labelText: 'Product Name',
    ),
    validator: (value) {
      if (value == null || value.trim().isEmpty) {
        return 'Product Name ဖြည့်ပေးပါ';
      }

      if (value.trim().length < 3) {
        return 'အနည်းဆုံး စာလုံး ၃ လုံး ဖြည့်ပေးပါ';
      }

      return null;
    },
  ),
)
```

Validation အောင်မြင်လျှင် `validator` က `null` ပြန်ပေးပြီး မအောင်မြင်လျှင် Error Message ကို String အဖြစ် ပြန်ပေးရပါတယ်။
Form ကို စစ်ဆေးရန် —

```
if (formKey.currentState!.validate()) {
  // Form data is valid
}
```

7.5. DropdownButtonFormField နှင့် Checkbox

`DropdownButtonFormField` သည် Category ကဲ့သို့ ရွေးချယ်စရာ အနည်းငယ်ရှိသည့် Data များအတွက် အသုံးပြုပါတယ်။

```
String? selectedCategory;
```

```
DropdownButtonFormField<String>(
  value: selectedCategory,
  decoration: const InputDecoration(
    labelText: 'Category',
    border: OutlineInputBorder(),
  ),
  items: const [
    DropdownMenuItem(
      value: 'Electronics',
      child: Text('Electronics'),
    ),
    DropdownMenuItem(
      value: 'Clothing',
      child: Text('Clothing'),
    ),
    DropdownMenuItem(
      value: 'Shoes',
      child: Text('Shoes'),
    ),
  ],
  onChanged: (value) {
    setState(() {
      selectedCategory = value;
    });
  },
  validator: (value) {
    if (value == null) {
      return 'Category ရွေးချယ်ပေးပါ';
    }
    return null;
  },
)
```

`Checkbox` က Boolean Value (`true` / `false`) ကို ကိုယ်စားပြုပါတယ်။ ဥပမာ Product တစ်ခု Stock ထဲမှာ ရှိ၊ မရှိ သတ်မှတ်နိုင်ပါတယ်။

```
bool inStock = true;

CheckboxListTile(
  title: const Text('Product is in stock'),
  value: inStock,
  onChanged: (value) {
    setState(() {
      inStock = value ?? false;
    });
  },
)
```

7.6. လက်တွေ့ Project — Product Form

အောက်ပါ Example မှာ အထက်ပါ အကြောင်းအရာအားလုံးကို ပေါင်းစပ်ထားပါတယ်။ Product Name၊ Price၊ Category၊ Description နှင့် Stock Status တို့ကို ဖြည့်သွင်းပြီး Validation အောင်မြင်မှ Submit လုပ်နိုင်မှာ ဖြစ်ပါတယ်။

Project တည်ဆောက်ခြင်း

Terminal မှာ အောက်ပါ Command များကို Run ပါ။

```
flutter create product_form_app
cd product_form_app
```

ပြီးရင် `lib/main.dart` ဖိုင်ထဲက Code ကို အောက်ပါအတိုင်း အစားထိုးပါ။

```
import 'package:flutter/material.dart';

void main() {
  runApp(const ProductFormApp());
}

class ProductFormApp extends StatelessWidget {
  const ProductFormApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Product Form',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.indigo,
        ),
        useMaterial3: true,
      ),
      home: const ProductFormPage(),
    );
  }
}

class ProductFormPage extends StatefulWidget {
  const ProductFormPage({super.key});
```

```
  @override
  State<ProductFormPage> createState() =>
      _ProductFormPageState();
}

class _ProductFormPageState
    extends State<ProductFormPage> {
  final _formKey = GlobalKey<FormState>();

  final _nameController = TextEditingController();
  final _priceController = TextEditingController();
  final _descriptionController =
      TextEditingController();

  String? _selectedCategory;
  bool _inStock = true;

  final List<String> _categories = [
    'Electronics',
    'Clothing',
    'Shoes',
    'Books',
    'Accessories',
  ];

  void _submitForm() {
    if (!_formKey.currentState!.validate()) {
      return;
    }

    final name = _nameController.text.trim();
    final price = double.parse(
      _priceController.text.trim(),
    );
    final description =
        _descriptionController.text.trim();
    final category = _selectedCategory!;

    showDialog(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Product Submitted'),
        content: SingleChildScrollView(
          child: Text(
            'Name: $name\n'
            'Price: \$${price.toStringAsFixed(2)}\n'
            'Category: $category\n'
            'Description: $description\n'
            'In Stock: ${_inStock ? "Yes" : "No"}',
          ),
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context),
            child: const Text('Close'),
          ),
```

```
        ],
      ),
    );
  }

  void _resetForm() {
    _formKey.currentState?.reset();

    _nameController.clear();
    _priceController.clear();
    _descriptionController.clear();

    setState(() {
      _selectedCategory = null;
      _inStock = true;
    });
  }

  @override
  void dispose() {
    _nameController.dispose();
    _priceController.dispose();
    _descriptionController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Add New Product'),
        centerTitle: true,
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(20),
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment:
                  CrossAxisAlignment.stretch,
              children: [
                const Icon(
                  Icons.inventory_2,
                  size: 64,
                  color: Colors.indigo,
                ),
                const SizedBox(height: 20),

                TextFormField(
                  controller: _nameController,
                  decoration: const InputDecoration(
                    labelText: 'Product Name',
                    hintText: 'Enter product name',
                    prefixIcon: Icon(Icons.shopping_bag),
                    border: OutlineInputBorder(),
                  ),
```

```
  validator: (value) {
    if (value == null ||
        value.trim().isEmpty) {
      return 'Product Name ဖြည့်ပါ';
    }

    if (value.trim().length < 3) {
      return 'အနည်းဆုံး စာလုံး ၃ လုံး လိုအပ်ပါတယ်';
    }

    return null;
  },
),

const SizedBox(height: 16),

TextFormField(
  controller: _priceController,
  keyboardType:
      const TextInputType.numberWithOptions(
    decimal: true,
  ),
  decoration: const InputDecoration(
    labelText: 'Price (USD)',
    hintText: 'Enter product price',
    prefixIcon: Icon(Icons.attach_money),
    border: OutlineInputBorder(),
  ),
  validator: (value) {
    if (value == null ||
        value.trim().isEmpty) {
      return 'Price ဖြည့်ပါ';
    }

    final price =
        double.tryParse(value.trim());

    if (price == null) {
      return 'မှန်ကန်သော နံပါတ် ဖြည့်ပါ';
    }

    if (!price.isFinite || price <= 0) {
      return 'Price သည် သုညထက် ကြီးရပါမယ်';
    }

    return null;
  },
),

const SizedBox(height: 16),

DropdownButtonFormField<String>(
  value: _selectedCategory,
```

```
    isExpanded: true,
    decoration: const InputDecoration(
      labelText: 'Category',
      prefixIcon: Icon(Icons.category),
      border: OutlineInputBorder(),
    ),
    items: _categories.map((category) {
      return DropdownMenuItem<String>(
        value: category,
        child: Text(category),
      );
    }).toList(),
    onChanged: (value) {
      setState(() {
        _selectedCategory = value;
      });
    },
    validator: (value) {
      if (value == null) {
        return 'Category ရွေးချယ်ပါ';
      }
      return null;
    },
  ),

  const SizedBox(height: 16),

  TextFormField(
    controller: _descriptionController,
    maxLines: 3,
    decoration: const InputDecoration(
      labelText: 'Description',
      hintText: 'Enter product description',
      alignLabelWithHint: true,
      border: OutlineInputBorder(),
    ),
    validator: (value) {
      if (value == null ||
          value.trim().isEmpty) {
        return 'Description ဖြည့်ပါ';
      }

      return null;
    },
  ),

  const SizedBox(height: 8),

  CheckboxListTile(
    contentPadding: EdgeInsets.zero,
    title: const Text('Product is in stock'),
    value: _inStock,
    controlAffinity:
        ListTileControlAffinity.leading,
    onChanged: (value) {
```

```
                    setState(() {
                      _inStock = value ?? false;
                    });
                  },
                ),

                const SizedBox(height: 20),

                ElevatedButton.icon(
                  onPressed: _submitForm,
                  icon: const Icon(Icons.save),
                  label: const Text('Submit Product'),
                  style: ElevatedButton.styleFrom(
                    padding: const EdgeInsets.all(16),
                  ),
                ),

                const SizedBox(height: 10),

                OutlinedButton(
                  onPressed: _resetForm,
                  child: const Text('Reset Form'),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

Application ကို Run ခြင်း

```
flutter run
```

Emulator သို့မဟုတ် ချိတ်ဆက်ထားသော Android Device ပေါ်မှာ Application ကို စမ်းသပ်နိုင်ပါတယ်။

Product Form Preview

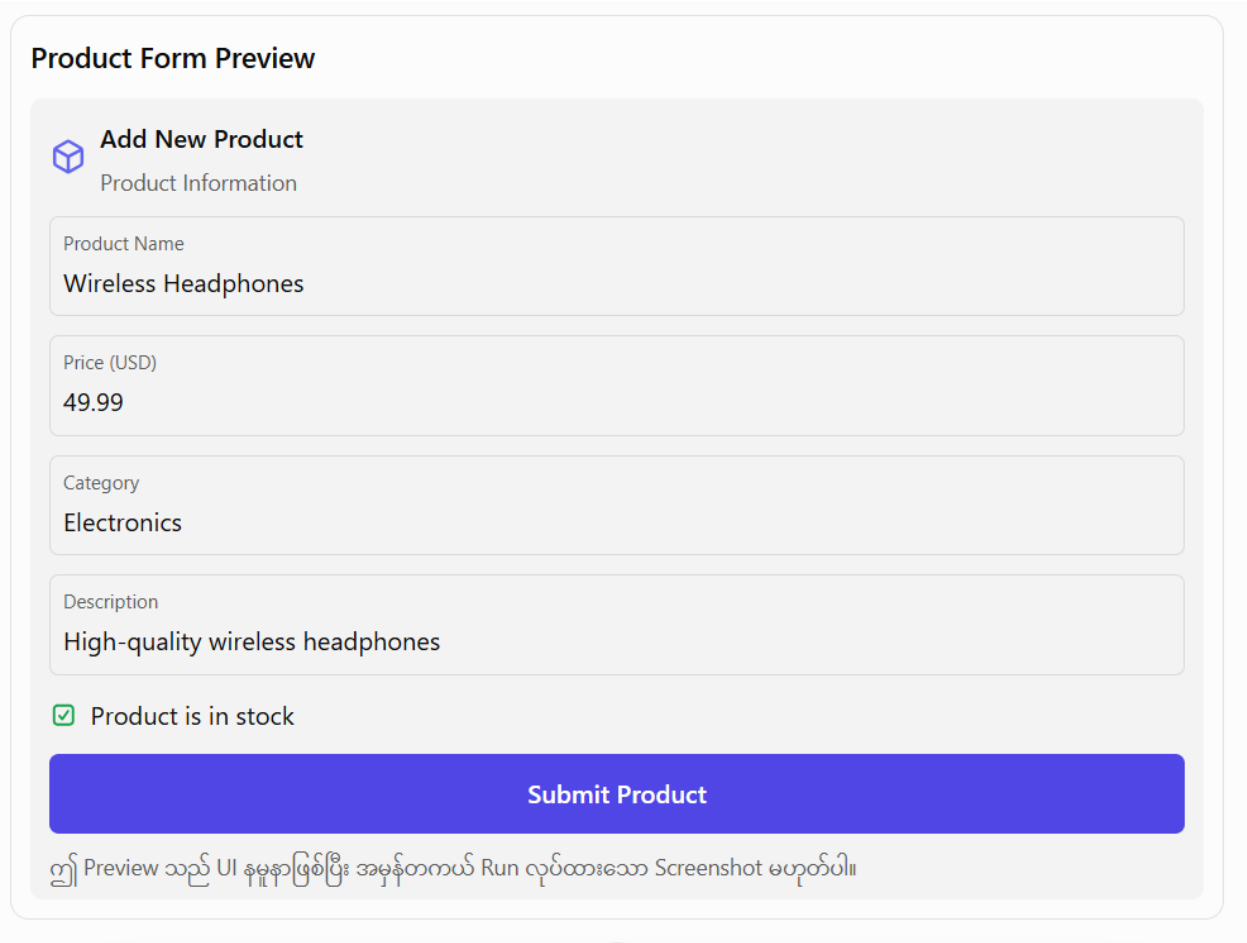


Figure 7.2. Product Preview From နမူနာ

7.7. Form Validation စမ်းသပ်ရန်

အောက်ပါ စမ်းသပ်မှုများကို ကိုယ်တိုင်လုပ်ကြည့်ပါ။

စမ်းသပ်မှု	မျှော်မှန်းရလဒ်
Product Name အလွတ်ထားခြင်း	Error Message ပြမည်
Name ကို TV ဟု ထည့်ခြင်း	အနည်းဆုံး ၃ လုံးလိုအပ်ကြောင်း ပြမည်

Price ကို `abc` ထည့်ခြင်း	Number Error ပြမည်
Price ကို `-10` ထည့်ခြင်း	Price Error ပြမည်
Category မရွေးခြင်း	Category Error ပြမည်
Field အားလုံး မှန်ကန်စွာဖြည့်ခြင်း	Product Submitted Dialog ပြမည်
Reset ကို နှိပ်ခြင်း	Form Value များ ရှင်းလင်းမည်

8. အိမ်စာနှင့် လက်တွေ့လေ့ကျင့်ခန်း

လေ့ကျင့်ခန်း Checklist

1. Email Field ထည့်ပါ
   Email ပုံစံ မှန်ကန်မှုကို စစ်ဆေးပါ။
2. Stock Quantity ထည့်ပါ
   Quantity သည် 0 သို့မဟုတ် အပေါင်းကိန်း ဖြစ်ရပါမယ်။
3. Discount Field ထည့်ပါ
   Discount ကို 0% မှ 100% အတွင်းသာ လက်ခံပါ။
4. Image URL ထည့်ပါ
   Product Image URL ကို လက်ခံရယူပါ။
5. Product List ပြပါ
   Submit ပြီးနောက် Product များကို ListView ဖြင့် ပြပါ။

9. အရေးကြီးသော မှတ်ချက်များ

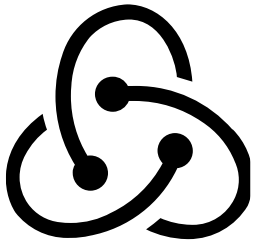
- `TextField` သည် ပုံမှန် Input အတွက် ဖြစ်ပြီး `TextFormField` သည် Form Validation နှင့် ပိုမိုအဆင်ပြေပါတယ်။
- `GlobalKey<FormState>` ကို အသုံးပြုပြီး `validate()` ဖြင့် Field အားလုံးကို စစ်ဆေးနိုင်ပါတယ်။

- `double.tryParse()` ကို အသုံးပြုခြင်းက Number မှားထည့်ထားသည့်အခါ Exception မဖြစ်စေဘဲ စစ်ဆေးနိုင်ပါတယ်။
- `TextEditingController` များကို `dispose()` လုပ်ပေးရပါတယ်။
- ဤသင်ခန်းစာ၏ Submit လုပ်ဆောင်ချက်သည် Dialog ထဲတွင် Data ပြသခြင်းသာ ဖြစ်ပြီး Database ထဲသို့ သိမ်းဆည်းခြင်း မပါဝင်သေးပါ။

နောက်သင်ခန်းစာမှာ Lesson 8: State Management with setState and Provider ကို လေ့လာပြီး Product Form မှရရှိသော Data များကို Application အတွင်း စနစ်တကျ စီမံခန့်ခွဲသွားနိုင်ပါတယ်။

CHAPTER 8

State Management with setState and Callbacks



ဒီ Chapter မှာ Flutter Application တစ်ခုအတွင်း Data ပြောင်းလဲသွားတဲ့အခါ UI ကို အလိုအလျောက် Update လုပ်နိုင်ဖို့ `StatefulWidget`, `setState()` နဲ့ Callback Functions တွေကို အသုံးပြုနည်း လေ့လာသွားမှာဖြစ်ပါတယ်။ ထို့အပြင် Parent Widget နဲ့ Child Widget အကြား Data ပေးပို့ခြင်း၊ Event ပြန်ပို့ခြင်းနှင့် Shopping Cart အရေအတွက်ကို Update လုပ်ခြင်းတို့ကို လက်တွေ့ရေးသားသွားမှာဖြစ်ပါတယ်။

8.1. State Management ဆိုတာဘာလဲ?

State ဆိုတာ Application တစ်ခုရဲ့လက်ရှိအခြေအနေကို ဖော်ပြတဲ့ Data ဖြစ်ပါတယ်။ ဥပမာ Shopping Application မှာ Product အရေအတွက်၊ Cart ထဲထည့်ထားတဲ့ ပစ္စည်းအရေအတွက်၊ Favorite ဖြစ်/မဖြစ်နဲ့ Login ဝင်ထား/မဝင်ထား စတဲ့ Data တွေဟာ State တွေဖြစ်ပါတယ်။

အသုံးပြုသူက Add to Cart ခလုတ်ကို နှိပ်လိုက်တဲ့အခါ Cart အရေအတွက် ပြောင်းလဲသွားနိုင်ပါတယ်။ ဒီလို Data ပြောင်းလဲသွားတဲ့အခါ Screen ပေါ်က Widget တွေကိုလည်း လိုက်လျောညီထွေ ပြောင်းလဲပြသဖို့ လိုအပ်ပါတယ်။

Flutter မှာ ရိုးရှင်းတဲ့ Local State တွေကို စီမံခန့်ခွဲဖို့ `StatefulWidget` နဲ့ `setState()` ကို အသုံးပြုနိုင်ပါတယ်။

8.2. StatelessWidget နှင့် StatefulWidget ကွာခြားချက်

StatelessWidget

Widget ကိုယ်တိုင် ထိန်းသိမ်းထားတဲ့ Mutable State မရှိပါဘူး။ ပေးထားတဲ့ Data နဲ့ Configuration အပေါ်မူတည်ပြီး UI ကို တည်ဆောက်ပေးပါတယ်။

```
class Welcome extends StatelessWidget {
  const Welcome({super.key});

  @override
  Widget build(BuildContext context) {
    return const Text('Welcome');
  }
}
```

StatefulWidget

အချိန်နှင့်အမျှ ပြောင်းလဲနိုင်တဲ့ State ကို သီးခြား State Object ထဲမှာ ထိန်းသိမ်းပြီး `setState()` ဖြင့် UI ကို ပြန်တည်ဆောက်နိုင်ပါတယ်။

```
class Counter extends StatefulWidget {
  const Counter({super.key});

  @override
  State<Counter> createState() =>
      _CounterState();
}

class _CounterState extends State<Counter> {
  int count = 0;

  @override
  Widget build(BuildContext context) {
    return Text('$count');
  }
}
```

8.3. setState() ကို အသုံးပြုခြင်း

`setState()` က State ပြောင်းလဲသွားကြောင်း Flutter ကို အသိပေးတဲ့ Method ဖြစ်ပါတယ်။ ပုံမှန်အားဖြင့် `setState()` ထဲမှာ State ကို ပြောင်းလဲပေးပြီးနောက် Flutter က သက်ဆိုင်ရာ Widget ရဲ့ `build()` Method ကို ပြန်ခေါ်ပေးပါတယ်။

Syntax:

```
setState(() {
  // Update state variables here
});
```

ဥပမာ Counter Application တစ်ခု တည်ဆောက်ကြည့်ရအောင်။

lib/main.dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: CounterPage(),
    );
  }
}

class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int count = 0;

  void increase() {
    setState(() {
      count++;
    });
```

```
}

void decrease() {
  setState(() {
    if (count > 0) {
      count--;
    }
  });
}

void reset() {
  setState(() {
    count = 0;
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('State Management'),
      backgroundColor: Colors.blue,
      foregroundColor: Colors.white,
    ),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          const Text(
            'Counter Value',
            style: TextStyle(fontSize: 22),
          ),
          const SizedBox(height: 16),
          Text(
            '$count',
            style: const TextStyle(
              fontSize: 48,
              fontWeight: FontWeight.bold,
            ),
          ),
          const SizedBox(height: 24),
          Wrap(
            spacing: 10,
            children: [
              ElevatedButton(
                onPressed: decrease,
                child: const Text('Decrease'),
              ),
              ElevatedButton(
                onPressed: increase,
                child: const Text('Increase'),
              ),
              ElevatedButton(
                onPressed: reset,
                child: const Text('Reset'),
              ),
            ],
```

```
            ),
          ],
        ),
      ),
    );
  }
}
```

အလုပ်လုပ်ပုံ

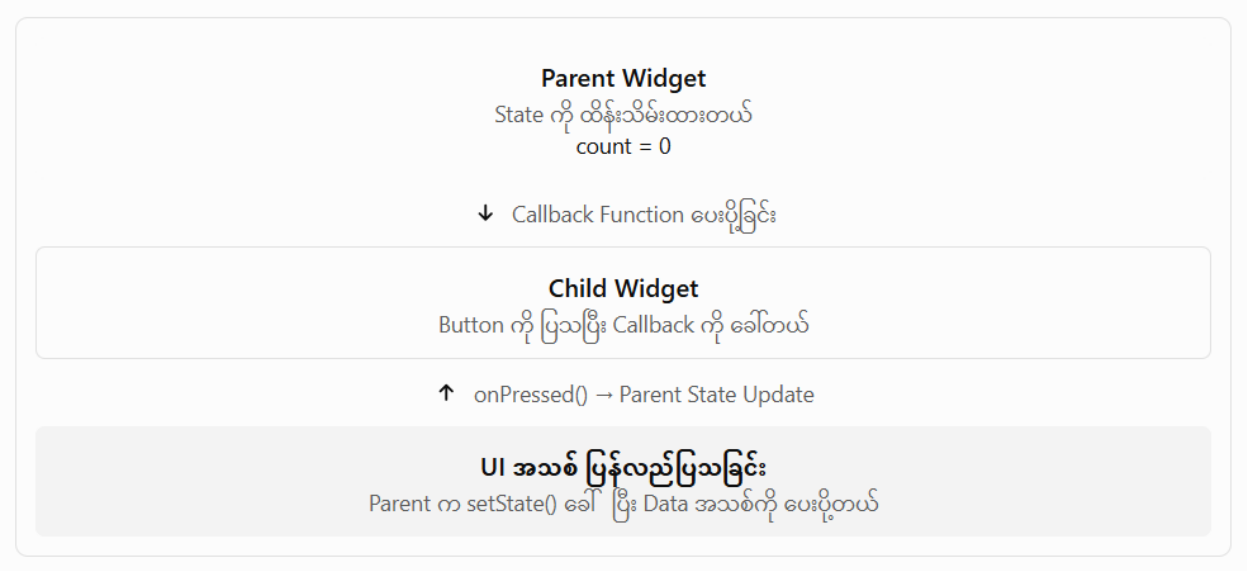
1. `count` Variable ရဲ့မူလတန်ဖိုးက `0` ဖြစ်ပါတယ်။
2. Increase ကို နှိပ်တဲ့အခါ `increase()` Function ကို ခေါ်ပါတယ်။
3. `setState()` အတွင်း `count++` ဖြင့် တန်ဖိုးကို တစ်တိုးပါတယ်။
4. Flutter က `build()` ကို ပြန်ခေါ် ပြီး Counter Value အသစ်ကို ပြသပေးပါတယ်။
5. Decrease နဲ့ Reset ခလုတ်တွေကလည်း State ကို ပြောင်းလဲပေးပါတယ်။

မှတ်ချက် — `setState()` မသုံးဘဲ Variable တစ်ခုကို ပြောင်းလဲရုံနဲ့ Flutter က အဲဒီပြောင်းလဲမှုကြောင့် UI ကို ပြန်တည်ဆောက်ပေးမယ်လို့ မယူဆသင့်ပါဘူး။

8.4. Callback Functions ဆိုတာဘာလဲ?

Callback ဆိုတာ Function တစ်ခုကို အခြား Widget သို့မဟုတ် Function တစ်ခုဆီ Parameter အဖြစ် ပေးပို့ပြီး လိုအပ်တဲ့အချိန်မှာ ပြန်ခေါ်အသုံးပြုတဲ့ နည်းလမ်းဖြစ်ပါတယ်။

ဥပမာ Parent Widget က Child Widget တစ်ခုကို ခလုတ်ပြသခိုင်းပြီး Child က ခလုတ်နှိပ်တဲ့အခါ Parent ကို အသိပေးနိုင်ပါတယ်။



8.5. Parent-Child Widget Communication လက်တွေ့ဥပမာ

ဒီဥပမာမှာ Parent Widget က Counter State ကို ထိန်းသိမ်းပြီး Child Widget က Increase ခလုတ်ကို ပြသပေးမှာဖြစ်ပါတယ်။

lib/main.dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: ParentPage(),
    );
  }
}

// Parent Widget
class ParentPage extends StatefulWidget {
  const ParentPage({super.key});

  @override
```

```
  State<ParentPage> createState() => _ParentPageState();
}

class _ParentPageState extends State<ParentPage> {
  int count = 0;

  void increaseCount() {
    setState(() {
      count++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Parent-Child Communication'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              'Count: $count',
              style: const TextStyle(fontSize: 32),
            ),
            const SizedBox(height: 20),

            // Pass data and callback to Child
            CounterButton(
              count: count,
              onIncrease: increaseCount,
            ),
          ],
        ),
      ),
    );
  }
}

// Child Widget
class CounterButton extends StatelessWidget {
  final int count;
  final VoidCallback onIncrease;

  const CounterButton({
    super.key,
    required this.count,
    required this.onIncrease,
  });

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Child Count: $count'),
        const SizedBox(height: 10),
```

```
        ElevatedButton(
          onPressed: onIncrease,
          child: const Text('Increase from Child'),
        ),
      ],
    );
  }
}
```

Code ကို အသေးစိတ်နားလည်ခြင်း

၁။ Parent က State ကို ထိန်းသိမ်းထားခြင်း

```
int count = 0;
```

`count` ကို Parent Widget ရဲ့ State ထဲမှာ သတ်မှတ်ထားပါတယ်။ ဒါကြောင့် Counter တန်ဖိုးကို ဘယ်နေရာမှာ ပြောင်းလဲမလဲဆိုတာ Parent က ထိန်းချုပ်နိုင်ပါတယ်။

၂။ Data နဲ့ Callback ကို Child ထံပေးပို့ခြင်း

```
CounterButton(
  count: count,
  onIncrease: increaseCount,
)
```

`count` က Child ထံ ပေးပို့တဲ့ Data ဖြစ်ပြီး `onIncrease` က Parent ရဲ့ Function ဖြစ်ပါတယ်။

၃။ Callback ကို လက်ခံခြင်း

```
final VoidCallback onIncrease;
```

`VoidCallback` ဆိုတာ Parameter မလိုအပ်ဘဲ Return Value မပေးတဲ့ Function အမျိုးအစား ဖြစ်ပါတယ်။

၄။ Child က Callback ကို ခေါ်ခြင်း

```
onPressed: onIncrease
```

အသုံးပြုသူက Button ကို နှိပ်တဲ့အခါ Parent ရဲ့ `increaseCount()` Function ကို ခေါ်ပေးပါတယ်။ Parent က `setState()` ခေါ် ပြီး `count` ကို တိုးလိုက်တာကြောင့် Parent နဲ့ Child နှစ်ခုစလုံးမှာ Count အသစ်ကို မြင်ရမှာဖြစ်ပါတယ်။

သတိပြုရန် — `onPressed: onIncrease` လို့ Function ကို ပေးပို့တာဖြစ်ပြီး `onPressed: onIncrease()` လို့ ရေးလိုက်ရင် Function ကို ချက်ချင်းခေါ် မိနိုင်ပါတယ်။

8.6. Shopping Application မှာ Cart Quantity Update လုပ်ခြင်း

အခု သင်ခန်းစာကို Shopping Application နဲ့ ချိတ်ဆက်ကြည့်ရအောင်။ Product တစ်ခုချင်းစီမှာ Add to Cart ခလုတ်တစ်ခု ထည့်ပြီး Cart ထဲရှိ ပစ္စည်းအရေအတွက်ကို Parent Widget က စီမံခန့်ခွဲမှာဖြစ်ပါတယ်။

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(
    debugShowCheckedModeBanner: false,
    home: ShopPage(),
  ));
}

class ShopPage extends StatefulWidget {
  const ShopPage({super.key});

  @override
  State<ShopPage> createState() => _ShopPageState();
}

class _ShopPageState extends State<ShopPage> {
  int cartCount = 0;

  void addToCart() {
    setState(() {
      cartCount++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('My Shopping App'),
        actions: [
          Padding(
```

```
            padding: const EdgeInsets.all(16),
            child: Center(
              child: Text(
                'Cart: $cartCount',
                style: const TextStyle(fontSize: 18),
              ),
            ),
          ),
        ],
      ),
      body: Center(
        child: Card(
          margin: const EdgeInsets.all(20),
          child: Padding(
            padding: const EdgeInsets.all(20),
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                const Icon(
                  Icons.shopping_bag,
                  size: 80,
                  color: Colors.blue,
                ),
                const SizedBox(height: 12),
                const Text(
                  'Flutter T-Shirt',
                  style: TextStyle(
                    fontSize: 22,
                    fontWeight: FontWeight.bold,
                  ),
                ),
                const SizedBox(height: 8),
                const Text('Price: 25000 MMK'),
                const SizedBox(height: 16),
                ElevatedButton.icon(
                  onPressed: addToCart,
                  icon: const Icon(Icons.add_shopping_cart),
                  label: const Text('Add to Cart'),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

ဒီဥပမာမှာ Add to Cart ခလုတ်ကို နှိပ်တိုင်း `cartCount` တစ်တိုးပြီး AppBar ထဲက Cart အရေအတွက်လည်း Update ဖြစ်သွားမှာဖြစ်ပါတယ်။ ဒီနည်းလမ်းဟာ Shopping Application ရဲ့ ပိုမိုရှုပ်ထွေးတဲ့ Cart System တည်ဆောက်ရာမှာ အခြေခံအဖြစ် အသုံးဝင်ပါတယ်။

သို့သော် ဒီဥပမာမှာ Cart ထဲရှိ Product ID၊ Quantity နဲ့ Price စတဲ့ အသေးစိတ် Data တွေကို မသိမ်းထားသေးပါဘူး။ လက်တွေ့ Project မှာ Product List သို့မဟုတ် Cart Item List ကို ထိန်းသိမ်းပြီး Quantity နဲ့ Total Price ကို တွက်ချက်ရမှာဖြစ်ပါတယ်။

8.7. setState() အသုံးပြုရာမှာ သတိပြုရန်

- `setState()` ကို `State` Class အတွင်းကနေ ခေါ်သုံးပါ။
- UI နဲ့ မသက်ဆိုင်တဲ့ အလုပ်တွေကို `setState()` အတွင်း မလိုအပ်ဘဲ မထည့်ပါနဲ့။
- `build()` Method အတွင်း `setState()` ကို အမြဲခေါ်နေခြင်းကို ရှောင်ကြဉ်ပါ။ မလိုအပ်တဲ့ Rebuild များ ဖြစ်စေနိုင်ပါတယ်။
- Network API ကနေ Data ယူခြင်းလို Asynchronous အလုပ်တွေကို `setState()` အတွင်း တိုက်ရိုက်လုပ်မယ့်အစား အရင်လုပ်ဆောင်ပြီး ရလာတဲ့ Data ကို `setState()` ဖြင့် Update လုပ်ပါ။
- Widget တစ်ခုကို ဖယ်ရှားပြီးနောက် Asynchronous အလုပ်တစ်ခု ပြီးဆုံးလာနိုင်တဲ့ အခြေအနေမှာ `setState()` မခေါ် မီ `mounted` ကို စစ်ဆေးဖို့ လိုနိုင်ပါတယ်။
- Parent က State ကို ထိန်းသိမ်းပြီး Child က Callback မှတစ်ဆင့် ပြန်လည်အသိပေးတဲ့ပုံစံကို အသုံးပြုရင် Data Flow ကို နားလည်ရလွယ်ကူပါတယ်။

8.8. လက်တွေ့လေ့ကျင့်ခန်းများ

လေ့ကျင့်ခန်း Checklist

1. Counter App
   Increase, Decrease နဲ့ Reset ခလုတ်တွေ ပါဝင်တဲ့ App တစ်ခု တည်ဆောက်ပါ။ Count ကို သုည‌အောက် မရောက်စေပါနဲ့။
2. Favorite Product
   Heart Icon ကို နှိပ်တဲ့အခါ အနီရောင်နဲ့ မီးခိုးရောင် အလှည့်ကျ ပြောင်းလဲပါစေ။
3. Parent-Child Communication

Parent က Username ကို Child ထံ ပေးပို့ပြီး Child က Button နှိပ်တဲ့အခါ Parent မှာ Message ပြသပါ။

4. Shopping Cart
   Product သုံးမျိုး ထည့်ပါ။ Add to Cart နှိပ်တိုင်း Cart Quantity တိုးပြီး Total Price ကို Update လုပ်ပါ။

8.9.Summary

ဒီသင်ခန်းစာမှာ `StatefulWidget` နဲ့ `setState()` ကို အသုံးပြုပြီး UI ကို Data ပြောင်းလဲမှုအလိုက် Update လုပ်နည်း၊ Callback Function တွေကတစ်ဆင့် Parent နဲ့ Child Widget အကြား ဆက်သွယ်နည်း၊ Shopping Cart Quantity ကို ပြောင်းလဲနည်းတို့ကို လေ့လာခဲ့ပါတယ်။

နောက်သင်ခန်းစာမှာ Lesson 9: Lists, GridView and Dynamic Product Display ကို ဆက်လက်လေ့လာပြီး Product Array တွေကို `ListView.builder`, `GridView.builder` နဲ့ ပြသခြင်း၊ Product Card တွေ တည်ဆောက်ခြင်းနဲ့ Dynamic Data ကို UI မှာ ဖော်ပြခြင်းတို့ကို လက်တွေ့ရေးသားနိုင်မှာဖြစ်ပါတယ်။

CHAPTER 9

Lists, GridViews and CRUD Operations



ဒီသင်ခန်းစာမှာ Flutter ဖြင့် Shopping Application တစ်ခု တည်ဆောက်ရာတွင် မရှိမဖြစ်လိုအပ်တဲ့ `ListView.builder`, `GridView.builder`, Model Classes နဲ့ CRUD Operations များကို လေ့လာသွားမှာဖြစ်ပါတယ်။ Product များကို List ပုံစံ၊ Grid ပုံစံဖြင့် ဖော်ပြခြင်း၊ Product အသစ်ထည့်ခြင်း၊ ပြင်ဆင်ခြင်းနှင့် ဖျက်ခြင်းတို့ကို လက်တွေ့ရေးသားသွားပါမယ်။

9.1. သင်ခန်းစာရည်ရွယ်ချက်များ

သင်ခန်းစာပြီးဆုံးပါက အောက်ပါတို့ကို လုပ်ဆောင်နိုင်မှာဖြစ်ပါတယ်။

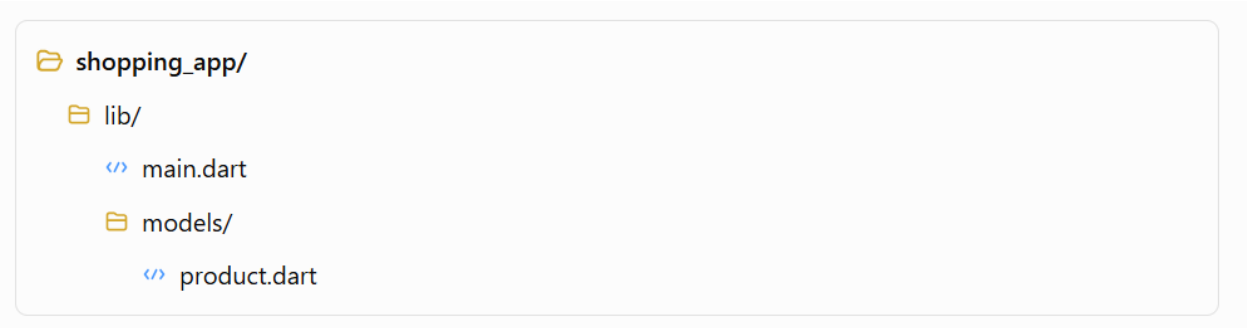
- `ListView.builder` ဖြင့် Product List တည်ဆောက်ခြင်း။
- `GridView.builder` ဖြင့် Product Grid တည်ဆောက်ခြင်း။
- Dart Model Class ဖြင့် Product Data ကို စနစ်တကျ သိမ်းဆည်းခြင်း။
- `List` နှင့် `setState()` ကို အသုံးပြုပြီး Data ပြောင်းလဲမှုကို UI တွင် ဖော်ပြခြင်း။
- Create, Read, Update, Delete (CRUD) Operations တည်ဆောက်ခြင်း။
- Add Product နှင့် Edit Product Form များ ဖန်တီးခြင်း။

9.2. Project တည်ဆောက်ခြင်း

ယခင်သင်ခန်းစာများမှ Flutter Project ကို ဆက်လက်အသုံးပြုနိုင်ပါတယ်။ Project အသစ်တည်ဆောက်လိုပါက Terminal မှာ အောက်ပါ Command ကို ရိုက်ထည့်ပါ။

```
flutter create shopping_app
cd shopping_app
flutter run
```

ဒီသင်ခန်းစာအတွက် ပြင်ဆင်ရမယ့် ဖိုင်ဖွဲ့စည်းပုံက အောက်ပါအတိုင်းဖြစ်ပါတယ်။



ဒီဥပမာမှာ Database သို့မဟုတ် အပို Package များ မလိုအပ်သေးပါဘူး။ Data တွေကို Memory ထဲမှာပဲ သိမ်းထားမှာဖြစ်တဲ့အတွက် App ပိတ်လိုက်ရင် ထည့်သွင်းထားတဲ့ Data များ ပျောက်သွားနိုင်ပါတယ်။

9.3. Model Class တည်ဆောက်ခြင်း

ဖိုင်အသစ် `lib/models/product.dart` ကို ဖန်တီးပါ။

Model Class ဆိုတာ Product တစ်ခုမှာ ပါဝင်မယ့် `id`, `name`, `price` နဲ့ `description` စတဲ့ Data တွေကို စနစ်တကျ သတ်မှတ်ပေးတဲ့ Class ဖြစ်ပါတယ်။

```
class Product {
  final int id;
  String name;
  double price;
  String description;

  Product({
```

```
    required this.id,
    required this.name,
    required this.price,
    required this.description,
  });
}
```

Code ရှင်းလင်းချက်

- `id` — Product တစ်ခုချင်းစီအတွက် သီးသန့် ID ဖြစ်ပါတယ်။
- `name` — Product အမည်ကို သိမ်းဆည်းပါတယ်။
- `price` — Product ဈေးနှုန်းကို `double` ဖြင့် သိမ်းဆည်းပါတယ်။
- `description` — Product အကြောင်းအရာကို သိမ်းဆည်းပါတယ်။
- `required` — Object တည်ဆောက်ချိန်တွင် သတ်မှတ်ထားတဲ့ Data များ မဖြစ်မနေ ပေးရမှာဖြစ်ပါတယ်။

`id` ကို `final` သတ်မှတ်ထားတဲ့အတွက် Product ဖန်တီးပြီးနောက် ID ကို ပြောင်းလဲလို့မရပါဘူး။ `name`, `price` နဲ့ `description` ကိုတော့ Update လုပ်နိုင်ပါတယ်။

9.4. ListView.builder အသုံးပြုခြင်း

`ListView.builder` ဟာ Data အများအပြားကို Scroll လုပ်ပြီး ကြည့်ရှုနိုင်တဲ့ List UI တည်ဆောက်ရာမှာ အသုံးဝင်ပါတယ်။ လိုအပ်တဲ့ Item များကိုသာ တည်ဆောက်ပေးတာကြောင့် Data အများအပြားရှိတဲ့အခါ အထူးသင့်လျော်ပါတယ်။

ဥပမာ Product တစ်ခုချင်းစီကို အမည်၊ ဈေးနှုန်းနဲ့ Icon တို့ဖြင့် ဖော်ပြနိုင်ပါတယ်။

```
ListView.builder(
  itemCount: products.length,
  itemBuilder: (context, index) {
    final product = products[index];

    return ListTile(
      leading: const Icon(Icons.shopping_bag),
      title: Text(product.name),
      subtitle: Text('\$${product.price.toStringAsFixed(2)}'),
      trailing: const Icon(Icons.arrow_forward_ios),
    );
  },
```

)

အဓိက Properties များ

Property	အသုံးပြုပုံ
`itemCount`	ဖော်ပြမယ့် Item အရေအတွက်
`itemBuilder`	Item တစ်ခုချင်းစီအတွက် Widget တည်ဆောက်ခြင်း
`index`	လက်ရှိ Item ရဲ့ List Position
`products[index]`	သက်ဆိုင်ရာ Product Data ကို ရယူခြင်း

9.5. GridView.builder အသုံးပြုခြင်း

Shopping Application တွေမှာ Product များကို နှစ်ကော်လံ၊ သုံးကော်လံ စသဖြင့် Grid ပုံစံဖော်ပြတာကို တွေ့ရလေ့ရှိပါတယ်။

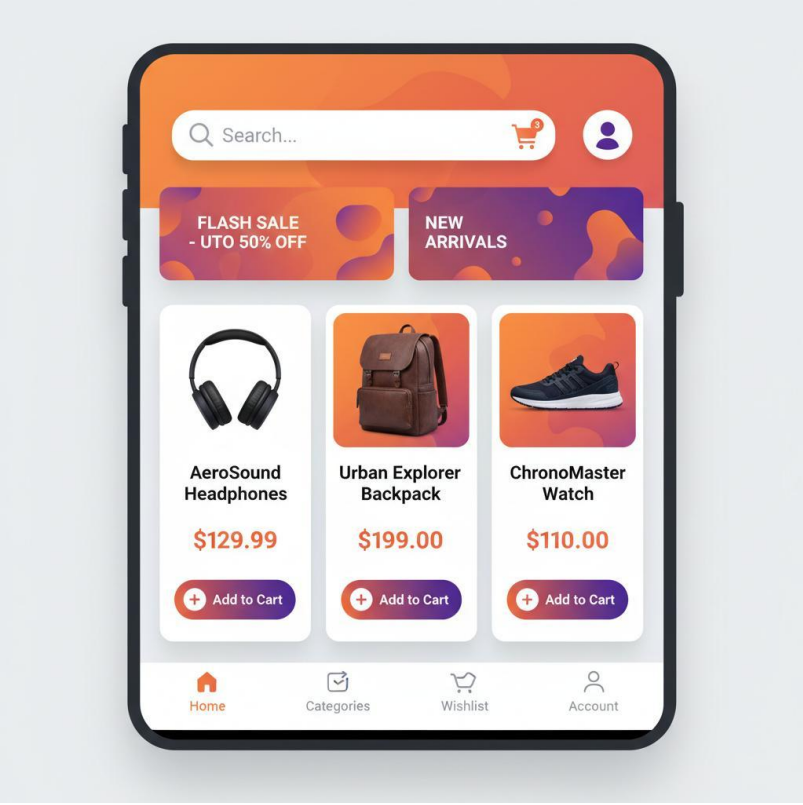


Figure 9.1. Product list on tablet screen(Large Screen)

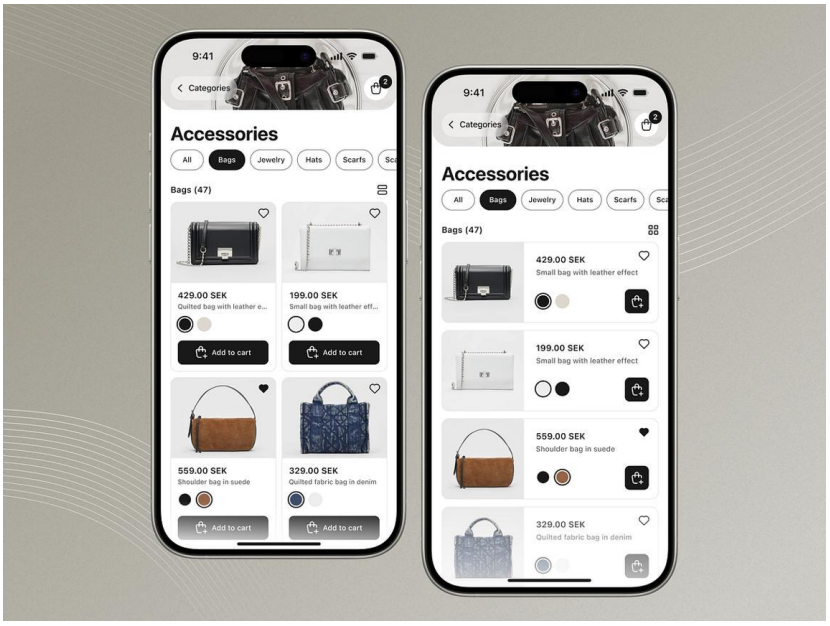


Figure 9.2. Product list displayed on medium mobile screen size

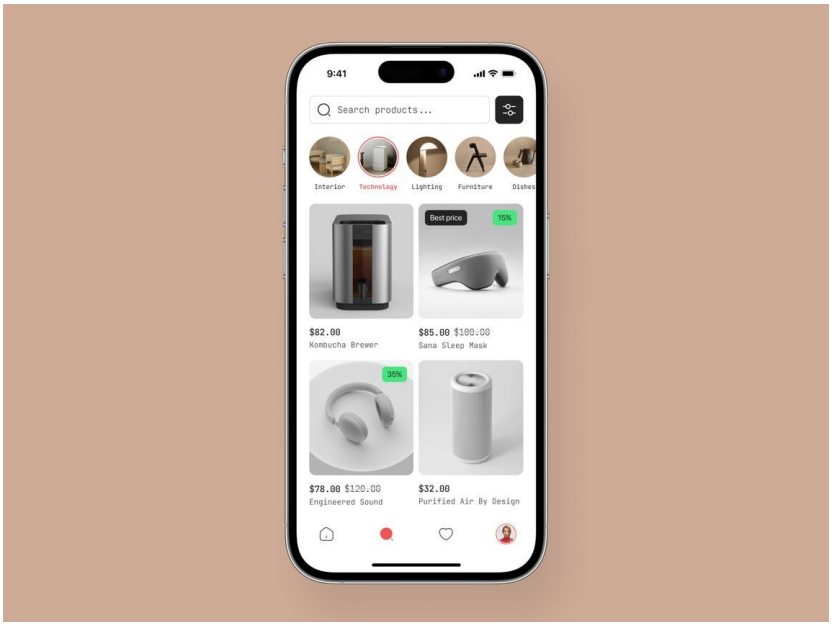


Figure 9.3. Product list displayed on small mobile screen size

`GridView.builder` ကို အသုံးပြုရာမှာ `crossAxisCount` ဖြင့် တစ်တန်းတွင် ဖော်ပြမယ့် Column အရေအတွက်ကို သတ်မှတ်နိုင်ပါတယ်။

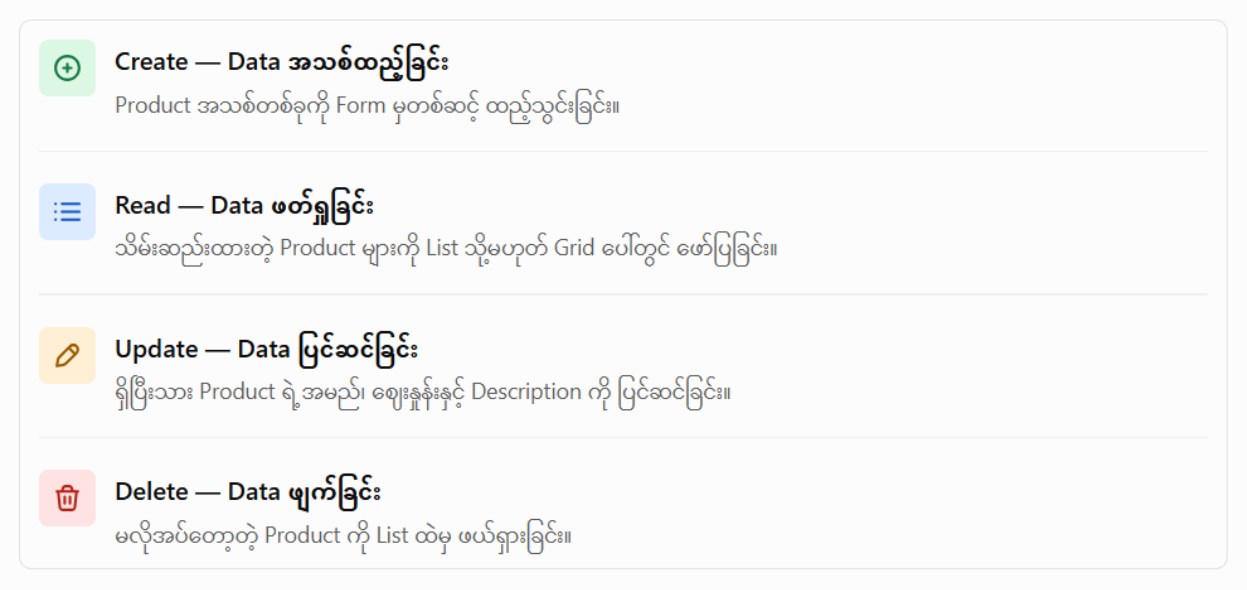
```
GridView.builder(
  padding: const EdgeInsets.all(12),
  itemCount: products.length,
  gridDelegate:
      const SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
    crossAxisSpacing: 12,
    mainAxisSpacing: 12,
    childAspectRatio: 0.85,
  ),
  itemBuilder: (context, index) {
    final product = products[index];

    return Card(
      child: Padding(
        padding: const EdgeInsets.all(12),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(
              Icons.shopping_bag,
              size: 48,
            ),
            const SizedBox(height: 12),
            Text(
              product.name,
              textAlign: TextAlign.center,
            ),
            Text(
              '\$${product.price.toStringAsFixed(2)}',
            ),
          ],
        ),
      ),
    );
  },
)
```

- `crossAxisCount: 2` — တစ်တန်းမှာ Product နှစ်ခု ဖော်ပြပါတယ်။
- `crossAxisSpacing` — ဘေးတိုက် Item များအကြား အကွာအဝေး ဖြစ်ပါတယ်။
- `mainAxisSpacing` — တန်းတစ်တန်းနှင့် နောက်တန်းအကြား အကွာအဝေး ဖြစ်ပါတယ်။
- `childAspectRatio` — Grid Item ရဲ့ အနံနှင့် အမြင့်အချိုး ဖြစ်ပါတယ်။

9.6. CRUD Operations ဆိုတာဘာလဲ

CRUD ဆိုတာ Application တစ်ခုမှာ Data များကို စီမံခန့်ခွဲဖို့ အဓိကလုပ်ဆောင်ချက် လေးမျိုးကို ဆိုလိုပါတယ်။



9.7. Complete Source Code — Product CRUD Application

ယခုအပိုင်းမှာ Product များကို Create, Read, Update, Delete လုပ်နိုင်တဲ့ Flutter Application အပြည့်အစုံ တည်ဆောက်ပါမယ်။

`lib/main.dart` ဖိုင်ထဲရှိ Code အားလုံးကို အောက်ပါအတိုင်း အစားထိုးရေးသားပါ။

```
import 'package:flutter/material.dart';
import 'models/product.dart';

void main() {
  runApp(const ShoppingApp());
}

class ShoppingApp extends StatelessWidget {
  const ShoppingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Shopping CRUD',
      theme: ThemeData(
        colorSchemeSeed: Colors.teal,
        useMaterial3: true,
      ),
      home: const ProductScreen(),
    );
  }
}
```

```
class ProductScreen extends StatefulWidget {
  const ProductScreen({super.key});

  @override
  State<ProductScreen> createState() => _ProductScreenState();
}

class _ProductScreenState extends State<ProductScreen> {
  final List<Product> products = [
    Product(
      id: 1,
      name: 'Laptop',
      price: 850,
      description: 'Laptop for programming',
    ),
    Product(
      id: 2,
      name: 'Smartphone',
      price: 350,
      description: 'Android smartphone',
    ),
    Product(
      id: 3,
      name: 'Headphones',
      price: 45,
      description: 'Wireless headphones',
    ),
  ];

  int nextId = 4;
  bool showGrid = true;

  Future<void> openProductForm({Product? product}) async {
    final nameController = TextEditingController(
      text: product?.name ?? '',
    );

    final priceController = TextEditingController(
      text: product == null ? '' : product.price.toString(),
    );

    final descriptionController = TextEditingController(
      text: product?.description ?? '',
    );

    final formKey = GlobalKey<FormState>();

    final result = await showDialog<bool>(
      context: context,
      builder: (dialogContext) {
        return AlertDialog(
          title: Text(
            product == null ? 'Add Product' : 'Edit Product',
          ),
          content: Form(
            key: formKey,
```

```
      child: SingleChildScrollView(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            TextFormField(
              controller: nameController,
              decoration: const InputDecoration(
                labelText: 'Product Name',
              ),
              validator: (value) {
                if (value == null || value.trim().isEmpty) {
                  return 'Enter product name';
                }
                return null;
              },
            ),
            TextFormField(
              controller: priceController,
              keyboardType:
                  const TextInputType.numberWithOptions(
                decimal: true,
              ),
              decoration: const InputDecoration(
                labelText: 'Price',
              ),
              validator: (value) {
                final price = double.tryParse(
                  value?.trim() ?? '',
                );

                if (price == null || price <= 0) {
                  return 'Enter a price greater than zero';
                }
                return null;
              },
            ),
            TextFormField(
              controller: descriptionController,
              decoration: const InputDecoration(
                labelText: 'Description',
              ),
              maxLines: 2,
            ),
          ],
        ),
      ),
    ),
    actions: [
      TextButton(
        onPressed: () {
          Navigator.pop(dialogContext, false);
        },
        child: const Text('Cancel'),
      ),
      FilledButton(
        onPressed: () {
          if (formKey.currentState!.validate()) {
```

```
                Navigator.pop(dialogContext, true);
              }
            },
            child: const Text('Save'),
          ),
        ],
      );
    },
  );

  if (result == true && mounted) {
    final name = nameController.text.trim();
    final price = double.parse(priceController.text.trim());
    final description = descriptionController.text.trim();

    setState(() {
      if (product == null) {
        products.add(
          Product(
            id: nextId++,
            name: name,
            price: price,
            description: description,
          ),
        );
      } else {
        product.name = name;
        product.price = price;
        product.description = description;
      }
    });
  }

  nameController.dispose();
  priceController.dispose();
  descriptionController.dispose();
}

void deleteProduct(Product product) {
  setState(() {
    products.removeWhere((item) => item.id == product.id);
  });

  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text('${product.name} deleted'),
    ),
  );
}

Widget buildProductCard(Product product) {
  return Card(
    margin: const EdgeInsets.all(6),
    clipBehavior: Clip.antiAlias,
    child: Padding(
      padding: const EdgeInsets.all(12),
      child: Column(
```

```
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Expanded(
              child: Center(
                child: Icon(
                  Icons.shopping_bag,
                  size: 48,
                  color: Colors.teal,
                ),
              ),
            ),
            Text(
              product.name,
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: const TextStyle(
                fontSize: 16,
                fontWeight: FontWeight.bold,
              ),
            ),
            Text(
              '\$${product.price.toStringAsFixed(2)}',
              style: const TextStyle(
                color: Colors.teal,
                fontWeight: FontWeight.bold,
              ),
            ),
            Text(
              product.description,
              maxLines: 2,
              overflow: TextOverflow.ellipsis,
            ),
            Row(
              mainAxisAlignment: MainAxisAlignment.end,
              children: [
                IconButton(
                  tooltip: 'Edit',
                  onPressed: () => openProductForm(
                    product: product,
                  ),
                  icon: const Icon(Icons.edit),
                ),
                IconButton(
                  tooltip: 'Delete',
                  onPressed: () => deleteProduct(product),
                  icon: const Icon(
                    Icons.delete,
                    color: Colors.red,
                  ),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
```

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Shopping Products'),
      actions: [
        IconButton(
          tooltip: showGrid ? 'List View' : 'Grid View',
          onPressed: () {
            setState(() {
              showGrid = !showGrid;
            });
          },
          icon: Icon(
            showGrid ? Icons.view_list : Icons.grid_view,
          ),
        ),
      ],
    ),
    body: products.isEmpty
        ? const Center(
            child: Text('No products available'),
          )
        : showGrid
            ? GridView.builder(
                padding: const EdgeInsets.all(8),
                itemCount: products.length,
                gridDelegate:
                    const SliverGridDelegateWithFixedCrossAxisCount(
                  crossAxisCount: 2,
                  crossAxisSpacing: 4,
                  mainAxisSpacing: 4,
                  childAspectRatio: 0.68,
                ),
                itemBuilder: (context, index) {
                  return buildProductCard(products[index]);
                },
              )
            : ListView.builder(
                padding: const EdgeInsets.all(8),
                itemCount: products.length,
                itemBuilder: (context, index) {
                  final product = products[index];

                  return Card(
                    child: ListTile(
                      leading: const CircleAvatar(
                        child: Icon(Icons.shopping_bag),
                      ),
                      title: Text(product.name),
                      subtitle: Text(
                        '\$${product.price.toStringAsFixed(2)}'
                        '\n${product.description}',
                      ),
                      isThreeLine: true,
                      trailing: PopupMenuButton<String>(
```

```
                        onSelected: (value) {
                          if (value == 'edit') {
                            openProductForm(product: product);
                          } else if (value == 'delete') {
                            deleteProduct(product);
                          }
                        },
                        itemBuilder: (context) => const [
                          PopupMenuItem(
                            value: 'edit',
                            child: Text('Edit'),
                          ),
                          PopupMenuItem(
                            value: 'delete',
                            child: Text('Delete'),
                          ),
                        ],
                      ),
                    ),
                  );
                },
              ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => openProductForm(),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

9.8. Source Code အလုပ်လုပ်ပုံ ရှင်းလင်းချက်

Create — Product အသစ်ထည့်ခြင်း

`FloatingActionButton` ကို နှိပ်လိုက်တဲ့အခါ `openProductForm()` ကို ခေါ်ပါတယ်။ Form ထဲမှာ Product အမည်နဲ့ ဈေးနှုန်း ထည့်သွင်းပြီး Save နှိပ်ပါက `products.add()` ဖြင့် Data အသစ်ထည့်ပေးပါတယ်။

Read — Product များဖော်ပြခြင်း

`products` List ထဲမှာ သိမ်းထားတဲ့ Data များကို `GridView.builder` သို့မဟုတ် `ListView.builder` ဖြင့် ဖော်ပြပါတယ်။ AppBar ရှိ View Icon ကို နှိပ်ပြီး Grid နဲ့ List ကို ပြောင်းလဲကြည့်ရှုနိုင်ပါတယ်။

Update — Product ပြင်ဆင်ခြင်း

Edit Icon ကို နှိပ်လိုက်ရင် လက်ရှိ Product Data များကို Form ထဲမှာ ဖြည့်ပေးပါတယ်။ Save လုပ်တဲ့အခါ ရှိပြီးသား Product ရဲ့ `name`, `price`, `description` တို့ကို ပြောင်းလဲပြီး `setState()` ဖြင့် UI ကို ပြန်လည် Update လုပ်ပါတယ်။

Delete — Product ဖျက်ခြင်း

Delete Icon ကို နှိပ်တဲ့အခါ `deleteProduct()` ကို ခေါ်ပါတယ်။ `removeWhere()` က သတ်မှတ်ထားတဲ့ ID နဲ့ ကိုက်ညီတဲ့ Product ကို List ထဲက ဖယ်ရှားပေးပါတယ်။

setState() ရဲ့အရေးပါပုံ

`setState()` ကို ခေါ်လိုက်တဲ့အခါ Flutter က Widget ကို ပြန်လည် Build လုပ်ပေးပါတယ်။ ဒါကြောင့် Product အသစ်ထည့်ခြင်း၊ ပြင်ဆင်ခြင်း၊ ဖျက်ခြင်းတို့ကို Screen ပေါ်မှာ ချက်ချင်းမြင်တွေ့နိုင်ပါတယ်။

9.9. Application ကို Run ပြီး စမ်းသပ်ခြင်း

Terminal မှာ အောက်ပါ Command ကို အသုံးပြုပါ။

```
flutter analyze
flutter run
```

စမ်းသပ်ရာမှာ အောက်ပါ Checklist ကို အသုံးပြုနိုင်ပါတယ်။

Testing Checklist

1. App စတင်ဖွင့်ချိန်တွင် Sample Products ၃ ခု ပေါ်လာခြင်း
2. Grid View မှ List View သို့ ပြောင်းလဲနိုင်ခြင်း
3. Product အသစ်တစ်ခု ထည့်သွင်းနိုင်ခြင်း
4. Product အမည်နှင့် ဈေးနှုန်းကို ပြင်ဆင်နိုင်ခြင်း
5. Product ကို Delete လုပ်နိုင်ခြင်း
6. Product အမည်မထည့်ပါက Validation Error ပြခြင်း
7. ဈေးနှုန်းသည် သုညထက်ကြီးသော Number ဖြစ်ရန် စစ်ဆေးခြင်း
8. Product အားလုံးဖျက်ပြီးနောက် Empty State ပြသခြင်း

သတိပြုရန် — ဒီ Project ဟာ လက်ရှိအချိန်မှာ In-memory CRUD Example ဖြစ်ပါတယ်။ App ကို ပိတ်ပြီး ပြန်ဖွင့်တဲ့အခါ Data များကို ပြန်လည်ရရှိလိုပါက SQLite, Hive သို့မဟုတ် API/Backend Database တစ်ခုခုနဲ့ ချိတ်ဆက်ဖို့ လိုအပ်ပါတယ်။

9.10. လက်တွေ့လေ့ကျင့်ခန်းများ

1. Product Model ထဲမှာ `category` Field အသစ်ထည့်ပါ။
2. Product များကို ဈေးနှုန်းအနည်းမှ အများအလိုက် Sort လုပ်ပါ။
3. Product Name ဖြင့် Search လုပ်နိုင်တဲ့ Search Bar တည်ဆောက်ပါ။
4. Delete မလုပ်မီ Confirmation Dialog ပြပါ။
5. Grid View မှာ Product Image များကို ဖော်ပြပါ။
6. Product အရေအတွက်ကို AppBar ပေါ်မှာ ပြသပါ။

နောက်သင်ခန်းစာ — Lesson 10: SQLite Database Integration

နောက်သင်ခန်းစာမှာ Product Data များကို SQLite Database ထဲမှာ အမြဲတမ်းသိမ်းဆည်းခြင်း၊ Database Table တည်ဆောက်ခြင်း၊ Insert, Select, Update, Delete SQL Queries များရေးသားခြင်းနှင့် App ပိတ်ပြီး ပြန်ဖွင့်သည့်အခါ Data မပျောက်အောင် စီမံခြင်းတို့ကို လေ့လာနိုင်ပါတယ်။

CHAPTER 10

Advanced State Management with Provider / Riverpod

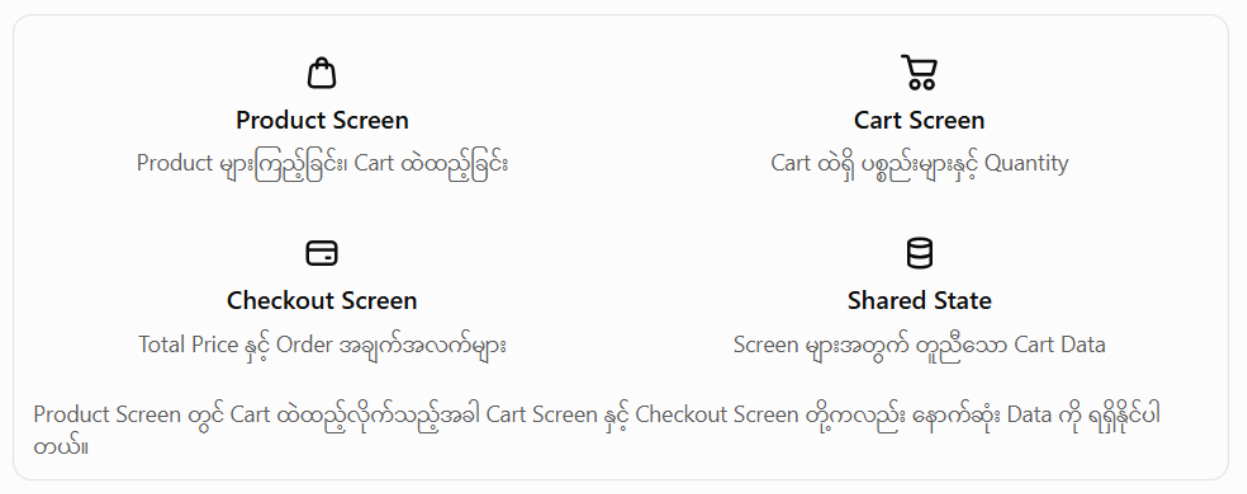


Flutter Shopping Application တွင် Screen တစ်ခုမှ ပြောင်းလဲလိုက်သော Data ကို အခြား Screen များတွင်ပါ အလိုအလျောက် Update ဖြစ်စေရန် Shared State Management ကို အသုံးပြုမှာ ဖြစ်ပါတယ်။ ဥပမာ Product ကို Cart ထဲထည့်ခြင်း၊ Cart ထဲရှိ ပစ္စည်းအရေအတွက် ပြောင်းလဲခြင်း၊ Total Price တွက်ချက်ခြင်းနှင့် App ရှိ Screen များအကြား Data မျှဝေခြင်းတို့ကို လေ့လာနိုင်ပါတယ်။

ဒီChapter မှာ Riverpod ကို အသုံးပြုပြီး Shopping Cart အတွက် လက်တွေ့ Project တစ်ခု တည်ဆောက်သွားပါမယ်။ Riverpod သည် Widget တစ်ခုနှင့်တစ်ခု တိုက်ရိုက်ချိတ်ဆက်ထားစရာမလိုဘဲ Application State ကို စီမံခန့်ခွဲနိုင်စေတဲ့ နည်းလမ်းတစ်ခု ဖြစ်ပါတယ်။

10.1. State Management ဆိုတာဘာလဲ?

ဥပမာ Shopping Application တစ်ခုတွင် အောက်ပါ Screen များ ရှိတယ်လို့ ယူဆကြည့်ပါ။



State Management မသုံးဘဲ Screen တစ်ခုမှတစ်ခုသို့ Data များကို အဆင့်ဆင့် ပို့နေရပါက Application ကြီးလာတဲ့အခါ Code ကို ထိန်းသိမ်းရခက်နိုင်ပါတယ်။ Riverpod ကဲ့သို့သော နည်းလမ်းကို အသုံးပြုခြင်းဖြင့် Shared State ကို သီးခြားစီ စီမံနိုင်ပါတယ်။

10.2. Provider နှင့် Riverpod ကွာခြားချက်

Provider	Riverpod
Flutter Widget Tree နှင့် `BuildContext` ကို အဓိကအသုံးပြုပါတယ်	Widget အပြင်ဘက်မှ State ကို စီမံနိုင်ပါတယ်
`context.watch()` ၊ `context.read()` ကို အသုံးပြုပါတယ်	`ref.watch()` ၊ `ref.read()` ကို အသုံးပြုပါတယ်
`ChangeNotifierProvider` ကို အသုံးပြုနိုင်ပါတယ်	`NotifierProvider` ၊ `Provider` စသည်တို့ကို အသုံးပြုနိုင်ပါတယ်
Beginner များအတွက် လေ့လာရလွယ်ပါတယ်	State Management နှင့် Testing အတွက် စနစ်တကျ ဖွဲ့စည်းနိုင်ပါတယ်

ဒီသင်ခန်းစာတွင် Riverpod ကို ရွေးချယ်ထားပြီး Product, Cart နှင့် Checkout Screen များအတွက် State တစ်ခုတည်းကို မျှဝေသုံးစွဲသွားမှာ ဖြစ်ပါတယ်။

10.3. Flutter Project တွင် Riverpod တပ်ဆင်ခြင်း

Terminal ကိုဖွင့်ပြီး Shopping Application Project Folder ထဲဝင်ပါ။

```
flutter pub add flutter_riverpod
```

ပြီးလျှင် Application ကို Run လုပ်နိုင်ရန် အောက်ပါ Command ကို အသုံးပြုပါ။

```
flutter pub get
```

ဒီသင်ခန်းစာမှာ အသုံးပြုမယ့် အဓိက Library က `flutter_riverpod` ဖြစ်ပါတယ်။

10.4. Project Structure တည်ဆောက်ခြင်း

Code များကို သီးခြားဖိုင်ခွဲထားခြင်းဖြင့် နောက်ပိုင်း Product CRUD၊ Authentication နှင့် Database ချိတ်ဆက်ရာတွင် ပိုမိုလွယ်ကူပါတယ်။

```
lib/
├── main.dart
├── models/
│   └── cart_item.dart
├── providers/
│   └── cart_provider.dart
└── screens/
    ├── product_screen.dart
    ├── cart_screen.dart
    └── checkout_screen.dart
```

ဒီသင်ခန်းစာမှာ နားလည်ရလွယ်စေရန် ပထမဆုံး `main.dart` ဖိုင်တစ်ခုတည်းဖြင့် လက်တွေ့အသုံးပြုနိုင်တဲ့ ဥပမာကို တည်ဆောက်ပြီးနောက် ဖိုင်ခွဲနည်းကို လေ့လာပါမယ်။

10.5. Riverpod ဖြင့် Shared Cart State တည်ဆောက်ခြင်း

`lib/main.dart` ဖိုင်ထဲတွင် အောက်ပါ Code အပြည့်အစုံကို ထည့်ပါ။

အောက်ပါဥပမာတွင် Product List၊ Cart Quantity၊ Remove၊ Total Price နှင့် Checkout Screen တို့ကို ထည့်သွင်းထားပါတယ်။

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

// 1. Product Model
class Product {
  final int id;
  final String name;
  final double price;

  const Product({
    required this.id,
    required this.name,
    required this.price,
  });
}

// 2. Cart Item Model
class CartItem {
  final Product product;
  final int quantity;

  const CartItem({
    required this.product,
    required this.quantity,
  });

  CartItem copyWith({int? quantity}) {
    return CartItem(
      product: product,
      quantity: quantity ?? this.quantity,
    );
  }
}

// 3. Shared State Provider
final cartProvider =
    NotifierProvider<CartNotifier, List<CartItem>>(
  CartNotifier.new,
);

// 4. Cart State Logic
class CartNotifier extends Notifier<List<CartItem>> {
  @override
  List<CartItem> build() => [];

  void addProduct(Product product) {
    final index = state.indexWhere(
      (item) => item.product.id == product.id,
    );

    if (index >= 0) {
      state = [
        for (int i = 0; i < state.length; i++)
          if (i == index)
            state[i].copyWith(
              quantity: state[i].quantity + 1,
            )
```

```
          else
            state[i],
      ];
    } else {
      state = [
        ...state,
        CartItem(product: product, quantity: 1),
      ];
    }
  }

  void decreaseQuantity(int productId) {
    state = [
      for (final item in state)
        if (item.product.id == productId &&
            item.quantity > 1)
          item.copyWith(quantity: item.quantity - 1)
        else
          if (item.product.id != productId) item,
    ];
  }

  void removeProduct(int productId) {
    state = state
        .where((item) => item.product.id != productId)
        .toList();
  }

  void clearCart() {
    state = [];
  }

  int get totalQuantity => state.fold(
        0,
        (total, item) => total + item.quantity,
      );

  double get totalPrice => state.fold(
        0.0,
        (total, item) =>
            total + item.product.price * item.quantity,
      );
}

// 5. Sample Products
const products = [
  Product(id: 1, name: 'Laptop', price: 850.00),
  Product(id: 2, name: 'Smartphone', price: 450.00),
  Product(id: 3, name: 'Headphones', price: 50.00),
  Product(id: 4, name: 'Keyboard', price: 35.00),
];

// 6. Application Entry Point
void main() {
  runApp(
    const ProviderScope(
      child: ShoppingApp(),
```

```
    ),
  );
}

class ShoppingApp extends StatelessWidget {
  const ShoppingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Riverpod Shopping',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      home: const ProductScreen(),
    );
  }
}

// 7. Product Screen
class ProductScreen extends ConsumerWidget {
  const ProductScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cart = ref.watch(cartProvider);

    final count = cart.fold<int>(
      0,
      (sum, item) => sum + item.quantity,
    );

    return Scaffold(
      appBar: AppBar(
        title: const Text('Products'),
        actions: [
          IconButton(
            icon: Badge(
              label: Text('$count'),
              child: const Icon(Icons.shopping_cart),
            ),
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (_) => const CartScreen(),
                ),
              );
            },
          ),
        ],
      ),
      body: ListView.builder(
        itemCount: products.length,
        itemBuilder: (context, index) {
```

```
          final product = products[index];

          return Card(
            margin: const EdgeInsets.all(8),
            child: ListTile(
              leading: const Icon(
                Icons.inventory_2,
                size: 36,
              ),
              title: Text(product.name),
              subtitle: Text(
                '\$${product.price.toStringAsFixed(2)}',
              ),
              trailing: IconButton(
                icon: const Icon(
                  Icons.add_shopping_cart,
                ),
                onPressed: () {
                  ref
                      .read(cartProvider.notifier)
                      .addProduct(product);

                  ScaffoldMessenger.of(context)
                    ..hideCurrentSnackBar()
                    ..showSnackBar(
                      SnackBar(
                        content: Text(
                          '${product.name} added to cart',
                        ),
                      ),
                    );
                },
              ),
            ),
          );
        },
      ),
    );
  }
}

// 8. Cart Screen
class CartScreen extends ConsumerWidget {
  const CartScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cart = ref.watch(cartProvider);
    final notifier = ref.read(cartProvider.notifier);

    return Scaffold(
      appBar: AppBar(title: const Text('Shopping Cart')),
      body: cart.isEmpty
          ? const Center(
              child: Text('Your cart is empty'),
            )
          : Column(
```

```
children: [
  Expanded(
    child: ListView.builder(
      itemCount: cart.length,
      itemBuilder: (context, index) {
        final item = cart[index];

        return ListTile(
          title: Text(item.product.name),
          subtitle: Text(
            '\$${item.product.price.toStringAsFixed(2)}'
            ' x ${item.quantity}',
          ),
          leading: IconButton(
            icon: const Icon(
              Icons.remove_circle_outline,
            ),
            onPressed: () {
              notifier.decreaseQuantity(
                item.product.id,
              );
            },
          ),
          trailing: Row(
            mainAxisSize: MainAxisSize.min,
            children: [
              Text('${item.quantity}'),
              IconButton(
                icon: const Icon(
                  Icons.add_circle_outline,
                ),
                onPressed: () {
                  notifier.addProduct(
                    item.product,
                  );
                },
              ),
              IconButton(
                icon: const Icon(
                  Icons.delete_outline,
                  color: Colors.red,
                ),
                onPressed: () {
                  notifier.removeProduct(
                    item.product.id,
                  );
                },
              ),
            ],
          ),
        );
      },
    ),
  ),
  Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
```

```
                    crossAxisAlignment:
                        CrossAxisAlignment.stretch,
                    children: [
                      Text(
                        'Total: \$${notifier.totalPrice.toStringAsFixed(2)}',
                        style: Theme.of(context)
                            .textTheme
                            .titleLarge,
                      ),
                      const SizedBox(height: 12),
                      FilledButton(
                        onPressed: () {
                          Navigator.push(
                            context,
                            MaterialPageRoute(
                              builder: (_) =>
                                  const CheckoutScreen(),
                            ),
                          );
                        },
                        child: const Text('Checkout'),
                      ),
                    ],
                  ),
                ),
              ],
            ),
          ),
        );
      }
    }

    // 9. Checkout Screen
    class CheckoutScreen extends ConsumerWidget {
      const CheckoutScreen({super.key});

      @override
      Widget build(BuildContext context, WidgetRef ref) {
        final cart = ref.watch(cartProvider);

        final total = cart.fold<double>(
          0.0,
          (sum, item) =>
              sum + item.product.price * item.quantity,
        );

        return Scaffold(
          appBar: AppBar(title: const Text('Checkout')),
          body: Padding(
            padding: const EdgeInsets.all(20),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                const Text(
                  'Order Summary',
                  style: TextStyle(
                    fontSize: 22,
                    fontWeight: FontWeight.bold,
```

```
              ),
            ),
            const SizedBox(height: 16),
            Expanded(
              child: ListView(
                children: [
                  for (final item in cart)
                    ListTile(
                      title: Text(item.product.name),
                      subtitle: Text(
                        'Quantity: ${item.quantity}',
                      ),
                      trailing: Text(
                        '\$${(item.product.price *
item.quantity).toStringAsFixed(2)}',
                      ),
                    ),
                ],
              ),
            ),
            Text(
              'Grand Total: \$${total.toStringAsFixed(2)}',
              style: const TextStyle(
                fontSize: 20,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 16),
            FilledButton(
              onPressed: cart.isEmpty
                  ? null
                  : () {
                      // Demo only: no real payment or order API.
                      ref
                          .read(cartProvider.notifier)
                          .clearCart();

                      ScaffoldMessenger.of(context)
                          .showSnackBar(
                        const SnackBar(
                          content: Text(
                            'Demo order completed',
                          ),
                        ),
                      );

                      Navigator.popUntil(
                        context,
                        (route) => route.isFirst,
                      );
                    },
              child: const Text('Place Demo Order'),
            ),
          ],
        ),
      ),
    );
```

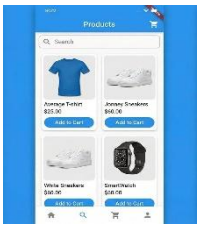
```
    }
}
```

10.6. Application ကို Run လုပ်ခြင်း

Terminal မှာ အောက်ပါ Command ကို အသုံးပြုပါ။

```
flutter run
```

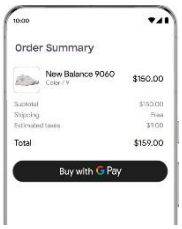
Application ထဲမှာ အောက်ပါအတိုင်း စမ်းသပ်ကြည့်ပါ။



Step 1 — Product Screen
Laptop ကို Cart ထဲထည့်ပါ။ Smartphone နှင့် Headphones တို့ကိုလည်း ထည့်ကြည့်ပါ။ AppBar ပေါ်က Cart Icon ဘေးတွင် Quantity ပြောင်းလဲလာပါမယ်။



Step 2 — Cart Screen
Cart ထဲဝင်ပြီး + ဖြင့် Quantity တိုးခြင်း၊ − ဖြင့် Quantity လျှော့ခြင်းနှင့် Delete Icon ဖြင့် Product ဖျက်ခြင်းတို့ကို စမ်းသပ်ပါ။



Step 3 — Checkout Screen
Checkout ကိုနှိပ်ပြီး Order Summary နှင့် Grand Total ကို ကြည့်ပါ။ Place Demo Order ကိုနှိပ်ပါက ဤဥပမာတွင် Cart ကို ရှင်းလင်းပေးပါမယ်။ တကယ့် Payment သို့မဟုတ် Database သိမ်းဆည်းမှု မပါဝင်သေးပါ။

10.7. Riverpod ရှိ အရေးကြီးသော Functions များ

ProviderScope

Application တစ်ခုလုံးက Riverpod Provider များကို အသုံးပြုနိုင်ရန် `runApp()` အတွင်း ထည့်ပေးရပါတယ်။

```
ProviderScope(
  child: ShoppingApp(),
)
```

ConsumerWidget

Provider မှ Data ကို ဖတ်ပြီး State ပြောင်းလဲသည့်အခါ UI ကို ပြန်တည်ဆောက်ပေးနိုင်တဲ့ Widget ဖြစ်ပါတယ်။

```
class CartScreen extends ConsumerWidget {
  const CartScreen({super.key});

  @override
  Widget build(
    BuildContext context,
    WidgetRef ref,
  ) {
    final cart = ref.watch(cartProvider);
    return Text('Items: ${cart.length}');
  }
}
```

ref.watch()

Provider ရဲ့ State ကို စောင့်ကြည့်ပြီး State ပြောင်းလဲတဲ့အခါ သက်ဆိုင်ရာ UI ကို ပြန်လည်တည်ဆောက်စေပါတယ်။

```
final cart = ref.watch(cartProvider);
```

ref.read()

State ကို ပြောင်းလဲစေမယ့် Action များ ခေါ်ယူတဲ့အခါ အသုံးပြုပါတယ်။

```
ref.read(cartProvider.notifier)
    .addProduct(product);
```

NotifierProvider

State နှင့် State ကို ပြောင်းလဲပေးမယ့် Business Logic ကို သီးခြားစီမံခန့်ခွဲနိုင်စေပါတယ်။

```
class CartNotifier
    extends Notifier<List<CartItem>> {
  @override
  List<CartItem> build() => [];
}
```

10.8. လက်တွေ့လေ့ကျင့်ခန်းများ

Exercise Checklist

1. Cart Badge
   Product Screen ရှိ Cart Icon တွင် စုစုပေါင်း Quantity ကို ပြပါ။
2. Increase / Decrease
   Quantity ကို တိုး၊ လျှော့နိုင်ပြီး အနည်းဆုံး 1 ဖြစ်အောင် ထိန်းပါ။
3. Remove Product
   Cart ထဲမှ Product တစ်ခုကို ဖျက်နိုင်အောင် လုပ်ပါ။
4. Total Price
   Price × Quantity ဖြင့် စုစုပေါင်းတန်ဖိုး တွက်ချက်ပါ။
5. Empty Cart
   Cart အလွတ်ဖြစ်သွားလျှင် Empty Cart Message ပြပါ။

10.9. Summary

- ဒီဥပမာရှိ State သည် App လည်ပတ်နေစဉ်အတွင်းသာ မှတ်ထားနိုင်ပါတယ်။ App ပိတ်ပြီး ပြန်ဖွင့်ပါက Cart Data ပျောက်သွားပါမယ်။
- App ကို ပိတ်ပြီးနောက်လည်း Data မပျောက်စေရန် SQLite၊ SharedPreferences သို့မဟုတ် Backend API နှင့် ချိတ်ဆက်ရန် လိုအပ်ပါတယ်။
- လက်တွေ့ Shopping Application တွင် Stock Validation၊ User Authentication၊ Order Persistence နှင့် Payment Verification တို့ကိုလည်း သီးခြားတည်ဆောက်ရပါမယ်။

နောက်သင်ခန်းစာ — Lesson 11: REST API Integration with Flutter

`http` သို့မဟုတ် `Dio` အသုံးပြုပြီး Backend API မှ Product Data ရယူခြင်း၊ JSON ကို Model အဖြစ် ပြောင်းလဲခြင်း၊ Loading / Error State များ စီမံခြင်းနှင့် Server မှ Product CRUD လုပ်ဆောင်ချက်များကို ချိတ်ဆက်သွားနိုင်ပါတယ်။

CHAPTER 11

Local Database with SQLite



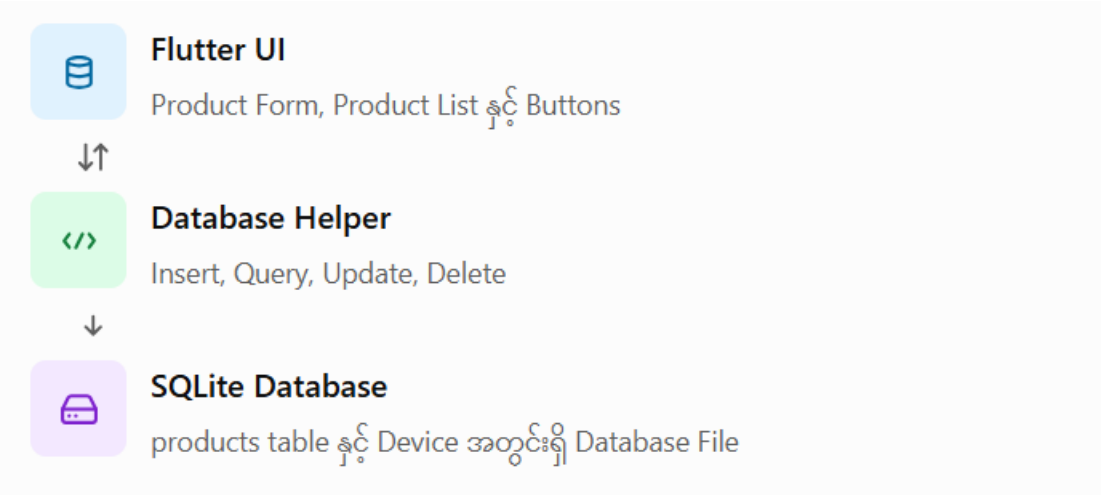
ဒီ Chapter မှာ Flutter Application အတွင်းရှိ Product Data များကို ဖုန်းရဲ့ Local Database ထဲမှာ သိမ်းဆည်းနိုင်ရန် `sqflite` Package ကို အသုံးပြုမှာဖြစ်ပါတယ်။ Database တည်ဆောက်ခြင်း၊ Table ဖန်တီးခြင်း၊ Product အသစ်ထည့်ခြင်း၊ Data ပြန်ဖတ်ခြင်း၊ ပြင်ဆင်ခြင်းနှင့် ဖျက်ခြင်းတို့ကို CRUD ပုံစံဖြင့် လက်တွေ့ရေးသားသွားမှာဖြစ်ပါတယ်။

သင်ခန်းစာပြီးဆုံးတဲ့အခါ Internet မလိုဘဲ ဖုန်းထဲမှာ Product Data များကို သိမ်းဆည်းပြီး App ကို ပိတ်ဖွင့်လုပ်သည့်အခါတွင်လည်း Data များကို ပြန်လည်ရယူအသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

11.1. SQLite နှင့် sqflite ကို နားလည်ခြင်း

- SQLite ဆိုတာ Application ရဲ့ Device ထဲမှာ Database File အဖြစ် Data များကို သိမ်းဆည်းပေးတဲ့ Database System ဖြစ်ပါတယ်။
- sqflite ဆိုတာ Flutter/Dart ကနေ SQLite Database ကို အသုံးပြုနိုင်အောင် ချိတ်ဆက်ပေးတဲ့ Package ဖြစ်ပါတယ်။
- CRUD ဆိုတာ Create, Read, Update, Delete လုပ်ဆောင်ချက်လေးမျိုးကို ဆိုလိုပါတယ်။

- Local Storage ဖြစ်တဲ့အတွက် သိမ်းဆည်းထားတဲ့ Data များကို Server မရှိဘဲ Device ပေါ်မှာပဲ ပြန်ဖတ်နိုင်ပါတယ်။
- ဒီသင်ခန်းစာမှာ Shopping Application ရဲ့ Product များကို နမူနာအဖြစ် အသုံးပြုသွားမှာ ဖြစ်ပါတယ်။



11.2. Package များ တပ်ဆင်ခြင်း

Flutter Project Folder ထဲမှာ Terminal ဖွင့်ပြီး အောက်ပါ Command ကို Run လုပ်ပါ။

```
flutter pub add sqflite
flutter pub add path
```

- `sqflite` က SQLite Database ကို အသုံးပြုရန် ဖြစ်ပါတယ်။
- `path` က Database File ရဲ့ Path ကို စနစ်တကျ တည်ဆောက်ရန် ဖြစ်ပါတယ်။

ဒီသင်ခန်းစာအတွက် Android Emulator သို့မဟုတ် Android Device ကို အသုံးပြုနိုင်ပါတယ်။ `sqflite` ရဲ့ Platform Support ကို သတိပြုပါ။ Flutter Web အတွက်တော့ ဒီ SQLite Mobile Implementation ကို တိုက်ရိုက်အသုံးမပြုနိုင်ပါဘူး။

11.3. Project Folder Structure

Project ကို အောက်ပါအတိုင်း စီစဉ်ထားပါမယ်။

```
lib/
├── main.dart
├── models/
│   └── product.dart
└── database/
    └── database_helper.dart
```

- `product.dart` — Product Model Class ကို သတ်မှတ်ပေးပါတယ်။
- `database_helper.dart` — Database ဖန်တီးခြင်းနှင့် CRUD Operations များကို ကိုင်တွယ်ပေးပါတယ်။
- `main.dart` — User Interface၊ Product Form နှင့် Product List ကို တည်ဆောက်ပေးပါတယ်။

11.4. Product Model Class တည်ဆောက်ခြင်း

`lib/models/product.dart` ဖိုင်ကို ဖန်တီးပါ။

```
class Product {
  final int? id;
  final String name;
  final double price;
  final String description;

  Product({
    this.id,
    required this.name,
    required this.price,
    required this.description,
  });

  // Product Object ကို Map အဖြစ် ပြောင်းခြင်း
  Map<String, dynamic> toMap() {
    return {
      'id': id,
      'name': name,
      'price': price,
      'description': description,
    };
```

```
  }

  // Database Map ကို Product Object အဖြစ် ပြောင်းခြင်း
  factory Product.fromMap(Map<String, dynamic> map) {
    return Product(
      id: map['id'] as int?,
      name: map['name'] as String,
      price: (map['price'] as num).toDouble(),
      description: map['description'] as String,
    );
  }
}
```

ရှင်းလင်းချက်

`Product` Class မှာ `id`, `name`, `price`, `description` ဆိုတဲ့ Properties လေးခု ပါဝင်ပါတယ်။
Database ထဲသို့ Data ထည့်တဲ့အခါ `toMap()` ကို အသုံးပြုပြီး Map အဖြစ် ပြောင်းပေးရပါတယ်။
Database ကနေ Data ပြန်ဖတ်တဲ့အခါ `fromMap()` နဲ့ Product Object ပြန်တည်ဆောက်နိုင်ပါတယ်။

`id` ကို `int?` သတ်မှတ်ထားတာက Product အသစ်ထည့်တဲ့အချိန်မှာ ID ကို Database က အလိုအလျောက် ထုတ်ပေးမှာဖြစ်လို့ပါ။

11.5. Database Helper တည်ဆောက်ခြင်း

`lib/database/database_helper.dart` ဖိုင်ကို ဖန်တီးပါ။

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

import '../models/product.dart';

class DatabaseHelper {
  DatabaseHelper._();

  static final DatabaseHelper instance =
      DatabaseHelper._();

  static Database? _database;

  Future<Database> get database async {
    if (_database != null) {
      return _database!;
```

```
    }

    _database = await _initDatabase();
    return _database!;
  }

  Future<Database> _initDatabase() async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, 'shopping.db');

    return await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE products (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            price REAL NOT NULL,
            description TEXT NOT NULL
          )
        ''');
      },
    );
  }

  // CREATE: Product အသစ်ထည့်ခြင်း
  Future<int> insertProduct(Product product) async {
    final db = await database;

    return await db.insert(
      'products',
      product.toMap(),
      conflictAlgorithm: ConflictAlgorithm.replace,
    );
  }

  // READ: Product အားလုံးဖတ်ခြင်း
  Future<List<Product>> getProducts() async {
    final db = await database;

    final maps = await db.query(
      'products',
      orderBy: 'id DESC',
    );

    return maps.map(Product.fromMap).toList();
  }

  // UPDATE: Product ပြင်ဆင်ခြင်း
  Future<int> updateProduct(Product product) async {
    final db = await database;

    return await db.update(
      'products',
```

```
      product.toMap(),
      where: 'id = ?',
      whereArgs: [product.id],
    );
  }

  // DELETE: Product ဖျက်ခြင်း
  Future<int> deleteProduct(int id) async {
    final db = await database;

    return await db.delete(
      'products',
      where: 'id = ?',
      whereArgs: [id],
    );
  }
}
```

Database Helper ထဲက အရေးကြီးတဲ့ အပိုင်းများ

(က) Database ဖန်တီးခြင်း

```
final path = join(dbPath, 'shopping.db');
```

ဒီ Code က `shopping.db` ဆိုတဲ့ Database File ရဲ့ Path ကို တည်ဆောက်ပေးပါတယ်။ Database ကို ပထမဆုံးဖွင့်တဲ့အခါ `onCreate` ကို ခေါ် ပြီး `products` Table ကို ဖန်တီးပေးပါတယ်။

(ခ) Table Structure

Column	SQLite Type	ရည်ရွယ်ချက်
`id`	INTEGER	အလိုအလျောက်တိုးလာမည့် Primary Key
`name`	TEXT	Product အမည်
`price`	REAL	Product ဈေးနှုန်း
`description`	TEXT	Product အကြောင်းအရာ

(ဂ) `whereArgs` အသုံးပြုခြင်း

```
where: 'id = ?',
whereArgs: [product.id],
```

ဒီပုံစံက သတ်မှတ်ထားတဲ့ Product ID နဲ့ ကိုက်ညီတဲ့ Record ကိုပဲ Update သို့မဟုတ် Delete လုပ်ပေးပါတယ်။ SQL Query ထဲမှာ User Input ကို String အဖြစ် တိုက်ရိုက်ပေါင်းထည့်ခြင်းထက် Parameterized Query ကို အသုံးပြုတာ ပိုမိုလုံခြုံပါတယ်။

(ဃ) Singleton Pattern

`DatabaseHelper.instance` ကို အသုံးပြုထားတဲ့အတွက် Application အတွင်းမှာ Database Helper Instance တစ်ခုတည်းကို ပြန်လည်အသုံးပြုနိုင်ပါတယ်။ Database Connection ကိုလည်း `_database` ထဲမှာ Cache လုပ်ထားပါတယ်။

11.6. Flutter UI နှင့် CRUD Application တည်ဆောက်ခြင်း

ယခု `lib/main.dart` ဖိုင်ထဲမှာ Product အသစ်ထည့်ခြင်း၊ Product List ပြသခြင်း၊ Edit လုပ်ခြင်းနှင့် Delete လုပ်ခြင်းတို့ကို လုပ်ဆောင်နိုင်တဲ့ Application တစ်ခု တည်ဆောက်ပါမယ်။

အောက်ပါ Code က အထက်က `Product` Model နဲ့ `DatabaseHelper` ကို အသုံးပြုထားတဲ့ ပြည့်စုံသော နမူနာဖြစ်ပါတယ်။

```
lib/main.dart

import 'package:flutter/material.dart';

import 'models/product.dart';
import 'database/database_helper.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const ShoppingApp());
}

class ShoppingApp extends StatelessWidget {
  const ShoppingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'SQLite Shopping App',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
```

```
      home: const ProductPage(),
    );
  }
}

class ProductPage extends StatefulWidget {
  const ProductPage({super.key});

  @override
  State<ProductPage> createState() => _ProductPageState();
}

class _ProductPageState extends State<ProductPage> {
  final db = DatabaseHelper.instance;

  List<Product> products = [];
  bool isLoading = true;

  @override
  void initState() {
    super.initState();
    loadProducts();
  }

  // READ
  Future<void> loadProducts() async {
    try {
      final result = await db.getProducts();

      if (!mounted) return;

      setState(() {
        products = result;
        isLoading = false;
      });
    } catch (e) {
      if (!mounted) return;

      setState(() => isLoading = false);

      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Load Error: $e')),
      );
    }
  }

  // CREATE / UPDATE Form
  Future<void> showProductForm({
    Product? product,
  }) async {
    final formKey = GlobalKey<FormState>();

    final nameController = TextEditingController(
      text: product?.name ?? '',
    );

    final priceController = TextEditingController(
```

```
  text: product?.price.toString() ?? '',
);

final descriptionController = TextEditingController(
  text: product?.description ?? '',
);

await showDialog<void>(
  context: context,
  builder: (dialogContext) {
    bool saving = false;

    return StatefulBuilder(
      builder: (context, setDialogState) {
        return AlertDialog(
          title: Text(
            product == null
                ? 'Add Product'
                : 'Edit Product',
          ),
          content: SingleChildScrollView(
            child: Form(
              key: formKey,
              child: Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  TextFormField(
                    controller: nameController,
                    decoration: const InputDecoration(
                      labelText: 'Product Name',
                      border: OutlineInputBorder(),
                    ),
                    validator: (value) {
                      if (value == null ||
                          value.trim().isEmpty) {
                        return 'Enter product name';
                      }
                      return null;
                    },
                  ),
                  const SizedBox(height: 12),
                  TextFormField(
                    controller: priceController,
                    keyboardType:
                        const TextInputType.numberWithOptions(
                      decimal: true,
                    ),
                    decoration: const InputDecoration(
                      labelText: 'Price',
                      border: OutlineInputBorder(),
                    ),
                    validator: (value) {
                      final price =
                          double.tryParse(value ?? '');

                      if (price == null || price < 0) {
                        return 'Enter a valid price';
```

```
                }

                return null;
              },
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: descriptionController,
              maxLines: 3,
              decoration: const InputDecoration(
                labelText: 'Description',
                border: OutlineInputBorder(),
              ),
              validator: (value) {
                if (value == null ||
                    value.trim().isEmpty) {
                  return 'Enter description';
                }
                return null;
              },
            ),
          ],
        ),
      ),
    ),
  ),
  actions: [
    TextButton(
      onPressed: saving
          ? null
          : () => Navigator.pop(dialogContext),
      child: const Text('Cancel'),
    ),
    FilledButton(
      onPressed: saving
          ? null
          : () async {
              if (!formKey.currentState!
                  .validate()) {
                return;
              }

              setDialogState(() => saving = true);

              try {
                final item = Product(
                  id: product?.id,
                  name: nameController.text.trim(),
                  price: double.parse(
                    priceController.text.trim(),
                  ),
                  description:
                      descriptionController.text.trim(),
                );

                if (product == null) {
                  await db.insertProduct(item);
                } else {
```

```
                            await db.updateProduct(item);
                          }

                          if (!mounted) return;

                          Navigator.pop(dialogContext);
                          await loadProducts();

                          if (!mounted) return;

                          ScaffoldMessenger.of(context)
                              .showSnackBar(
                            SnackBar(
                              content: Text(
                                product == null
                                    ? 'Product added successfully'
                                    : 'Product updated successfully',
                              ),
                            ),
                          );
                        } catch (e) {
                          setDialogState(() => saving = false);

                          ScaffoldMessenger.of(context)
                              .showSnackBar(
                            SnackBar(
                              content: Text('Save Error: $e'),
                            ),
                          );
                        }
                      },
                child: Text(
                  saving ? 'Saving...' : 'Save',
                ),
              ),
            ],
          );
        },
      );
    },
  );

  nameController.dispose();
  priceController.dispose();
  descriptionController.dispose();
}

// DELETE
Future<void> deleteProduct(Product product) async {
  final confirm = await showDialog<bool>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Delete Product?'),
      content: Text(
        'Delete "${product.name}" permanently?',
      ),
      actions: [
```

```
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('Cancel'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('Delete'),
          ),
        ],
      ),
    );

    if (confirm != true) return;

    try {
      await db.deleteProduct(product.id!);
      await loadProducts();

      if (!mounted) return;

      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Product deleted successfully'),
        ),
      );
    } catch (e) {
      if (!mounted) return;

      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Delete Error: $e')),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('SQLite Product Manager'),
        actions: [
          IconButton(
            onPressed: loadProducts,
            icon: const Icon(Icons.refresh),
            tooltip: 'Refresh',
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () => showProductForm(),
        icon: const Icon(Icons.add),
        label: const Text('Add Product'),
      ),
      body: isLoading
          ? const Center(
              child: CircularProgressIndicator(),
            )
          : products.isEmpty
```

```
            ? const Center(
                child: Text(
                  'No products found.\nTap Add Product to begin.',
                  textAlign: TextAlign.center,
                ),
              )
            : ListView.builder(
                padding: const EdgeInsets.only(
                  left: 12,
                  right: 12,
                  top: 12,
                  bottom: 90,
                ),
                itemCount: products.length,
                itemBuilder: (context, index) {
                  final product = products[index];

                  return Card(
                    child: ListTile(
                      leading: CircleAvatar(
                        child: Text('${product.id}'),
                      ),
                      title: Text(product.name),
                      subtitle: Text(
                        '\$${product.price.toStringAsFixed(2)}\n'
                        '${product.description}',
                        maxLines: 3,
                        overflow: TextOverflow.ellipsis,
                      ),
                      isThreeLine: true,
                      trailing: PopupMenuButton<String>(
                        onSelected: (value) {
                          if (value == 'edit') {
                            showProductForm(
                              product: product,
                            );
                          } else if (value == 'delete') {
                            deleteProduct(product);
                          }
                        },
                        itemBuilder: (context) => const [
                          PopupMenuItem(
                            value: 'edit',
                            child: Text('Edit'),
                          ),
                          PopupMenuItem(
                            value: 'delete',
                            child: Text('Delete'),
                          ),
                        ],
                      ),
                    ),
                  );
                },
              ),
    );
  }
```

```
}
```

11.7. Application ကို Run လုပ်ခြင်း

Terminal မှာ အောက်ပါ Command များကို အစဉ်လိုက် Run လုပ်ပါ။

```
flutter pub get
flutter analyze
flutter run
```

ပထမဆုံး Run လုပ်တဲ့အခါ `shopping.db` ကို ဖန်တီးပြီး `products` Table ကို အလိုအလျောက် တည်ဆောက်ပေးပါမယ်။

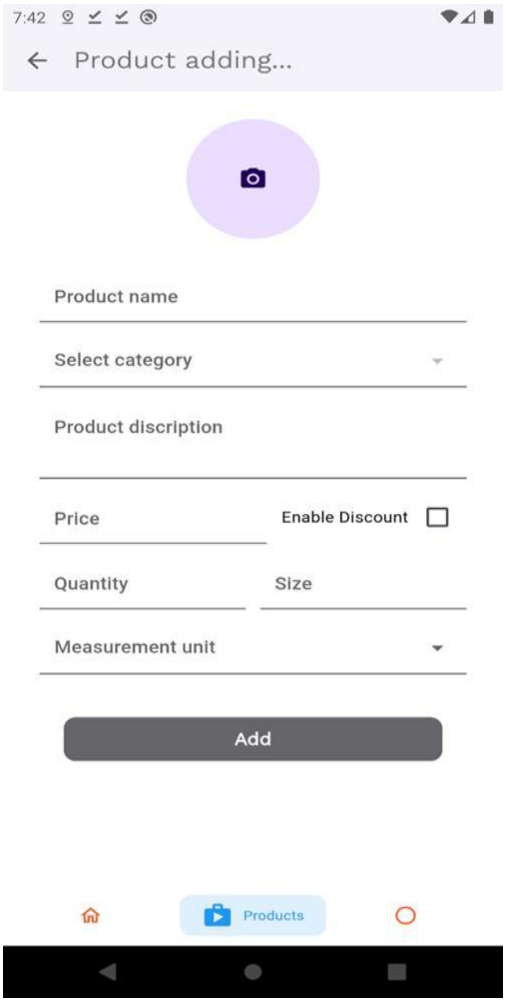


Figure 11.1. Add New Product Form

Add Product

Product Name, Price နဲ့ Description ဖြည့်ပြီး Add နှိပ်ပါ။

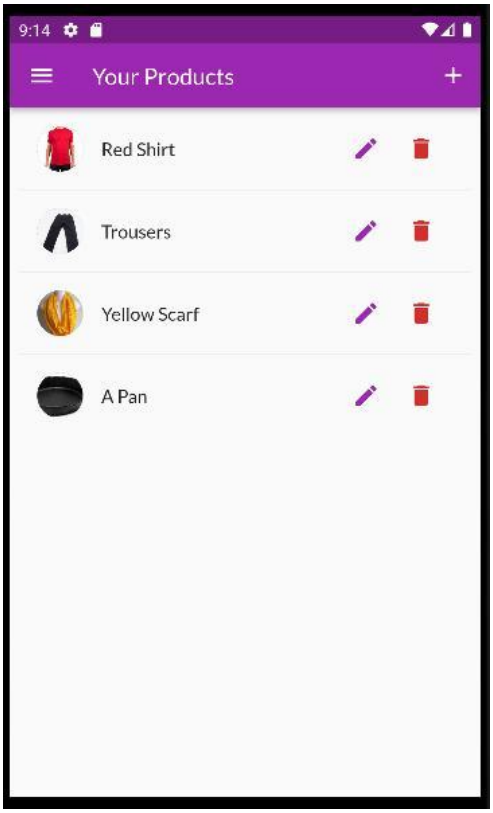


Figure 11.2. Show Product List

Product List

သိမ်းထားတဲ့ Product များကို List အဖြစ် ကြည့်နိုင်ပြီး Edit၊ Delete လုပ်နိုင်ပါတယ်။

အထက်ပါပုံများသည် UI နမူနာပုံများသာဖြစ်ပြီး သင့် Project မှ တကယ် Run ထားသော Screenshot များ မဟုတ်ပါဘူး။

11.8. CRUD Operations ကို နားလည်ခြင်း

အခုရေးထားတဲ့ Application မှာ CRUD လုပ်ဆောင်ချက်လေးမျိုးကို အောက်ပါအတိုင်း ချိတ်ဆက်အသုံးပြုထားပါတယ်။

Operation	Method	လုပ်ဆောင်ချက်
Create	`insertProduct()`	Product အသစ်ထည့်ခြင်း
Read	`getProducts()`	Product အားလုံး ဖတ်ခြင်း
Update	`updateProduct()`	ရှိပြီးသား Product ပြင်ဆင်ခြင်း
Delete	`deleteProduct()`	Product ဖျက်ခြင်း

ဥပမာ Database Helper ကို UI မပါဘဲ သီးခြားစမ်းသပ်ချင်ရင် အောက်ပါပုံစံကို အသုံးပြုနိုင်ပါတယ်။

```
final db = DatabaseHelper.instance;

// 1. CREATE
final id = await db.insertProduct(
  Product(
    name: 'Keyboard',
    price: 25.50,
    description: 'Wireless keyboard',
  ),
);

// 2. READ
final products = await db.getProducts();

for (final product in products) {
  print('${product.id}: ${product.name}');
}

// 3. UPDATE
await db.updateProduct(
  Product(
    id: id,
    name: 'Gaming Keyboard',
    price: 35.00,
    description: 'RGB wireless keyboard',
  ),
);

// 4. DELETE
await db.deleteProduct(id);
```

ဒီ Code ကို စမ်းသပ်ရန် `main()` အတွင်းမှာ တိုက်ရိုက်ထည့်မယ့်အစား သီးခြား Test Function တစ်ခုထဲမှာ ထည့်ပြီး `WidgetsFlutterBinding.ensureInitialized()` နဲ့ Database Initialization ပြီးမှ ခေါ်သုံးသင့်ပါတယ်။

11.9. Database Data များ မပျောက်စေရန် သိထားရမည့်အချက်များ

- App ကို ပိတ်ပြီး ပြန်ဖွင့်တဲ့အခါ ပုံမှန်အားဖြင့် Database File နဲ့ သိမ်းထားတဲ့ Data များ ကျန်ရှိနေပါတယ်။
- Android App ကို Uninstall လုပ်ခြင်း၊ App Data ကို Clear လုပ်ခြင်းတို့ကြောင့် Local Database ပျောက်သွားနိုင်ပါတယ်။
- Database Structure ပြောင်းလဲတဲ့အခါ `version` နှင့် `onUpgrade` ကို စနစ်တကျ အသုံးပြုရပါတယ်။
- `sqflite` က Local Database အတွက် သင့်တော်ပေမယ့် Device များအကြား Data Sync လုပ်ရန် Server သို့မဟုတ် Cloud Database ထပ်မံလိုအပ်ပါတယ်။
- Product အများအပြားရှိလာတဲ့အခါ Pagination၊ Search Query နဲ့ Database Index များကိုလည်း ထည့်သွင်းစဉ်းစားနိုင်ပါတယ်။

11.10. လက်တွေ့လေ့ကျင့်ခန်းများ

လေ့ကျင့်ခန်း Checklist

1. Product အသစ် ၅ ခု ထည့်ပါ
   Product Name၊ Price နှင့် Description မတူညီအောင် ဖြည့်ပါ။
2. Product စျေးနှုန်း ပြင်ပါ
   ရှိပြီးသား Product တစ်ခုရဲ့ Price ကို Update လုပ်ပါ။
3. Product တစ်ခု ဖျက်ပါ
   Delete Confirmation Dialog ကို အသုံးပြုပြီး ဖျက်ပါ။
4. Search Feature ထည့်ပါ
   Product Name အလိုက် ရှာဖွေနိုင်ရန် Database Query အသစ်တစ်ခု ရေးပါ။
5. Category Table ထပ်တည်ဆောက်ပါ
   categories Table ဖန်တီးပြီး Product များနဲ့ ချိတ်ဆက်နိုင်အောင် လေ့လာပါ။

11.11. Summary

ဒီသင်ခန်းစာမှာ `sqflite` နဲ့ `path` Package များကို အသုံးပြုပြီး Database ဖန်တီးခြင်း၊ Table တည်ဆောက်ခြင်း၊ Model Class ပြုလုပ်ခြင်းနှင့် CRUD Operations များကို Flutter UI နဲ့ ချိတ်ဆက်အသုံးပြုခြင်းတို့ကို လေ့လာခဲ့ပါတယ်။

နောက်သင်ခန်းစာမှာ Lesson 12: REST API Integration with HTTP and JSON ကို ဆက်လက်လေ့လာနိုင်ပါတယ်။ အဲဒီသင်ခန်းစာမှာ Flutter Application ကနေ API ခေါ်ယူခြင်း၊ JSON Data ပြောင်းလဲဖတ်ရှုခြင်း၊ Loading/Error State များကို စီမံခြင်းနှင့် Server မှ Product Data များကို ရယူပြသခြင်းတို့ကို တည်ဆောက်သွားနိုင်ပါတယ်။

CHAPTER 12

REST API Integration with HTTP and JSON



ဒီ Chapter မှာ Flutter Application ကို REST API နဲ့ ချိတ်ဆက်ပြီး Server မှ Product Data များ ရယူခြင်း၊ Product အသစ်ထည့်ခြင်း၊ Product ပြင်ဆင်ခြင်း၊ Product ဖျက်ခြင်းနဲ့ JSON Data များကို Dart Model Object များအဖြစ် ပြောင်းလဲခြင်းတို့ကို လေ့လာသွားမှာ ဖြစ်ပါတယ်။

12.1. Chapter ၏ ရည်ရွယ်ချက်များ

သင်ခန်းစာပြီးဆုံးတဲ့အခါ အောက်ပါလုပ်ဆောင်ချက်တွေကို တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။

- HTTP GET ဖြင့် Product List ရယူခြင်း
- HTTP POST ဖြင့် Product အသစ်ထည့်ခြင်း
- HTTP PUT ဖြင့် Product Data ပြင်ဆင်ခြင်း
- HTTP DELETE ဖြင့် Product ဖျက်ခြင်း
- JSON Parsing နှင့် Model Conversion
- Loading Indicator ပြသခြင်း
- API Error များကို ကိုင်တွယ်ခြင်း
- API မှရရှိတဲ့ Data များကို Flutter UI တွင် ပြသခြင်း

12.2. REST API ဆိုတာဘာလဲ?

REST API ဆိုတာ Flutter Application နဲ့ Server တို့အကြား Data ပေးပို့ခြင်း၊ ရယူခြင်းတို့အတွက် အသုံးပြုတဲ့ Interface တစ်မျိုးဖြစ်ပါတယ်။

ဥပမာ Shopping Application တစ်ခုမှာ Product List ကို Server ကနေ ရယူမယ်ဆိုရင် Flutter က HTTP GET Request ပို့ပြီး Server က JSON Data ပြန်ပေးမှာ ဖြစ်ပါတယ်။

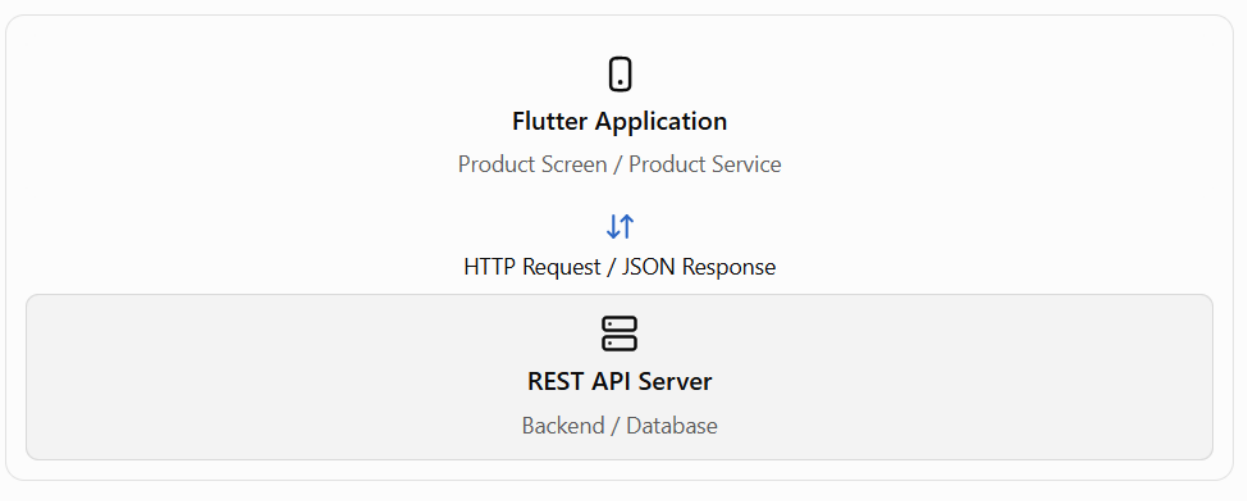


Figure 12.1. Rest API Data Flow

HTTP Methods

Method	လုပ်ဆောင်ချက်	ဥပမာ
GET	Data ရယူခြင်း	Product List
POST	Data အသစ်ထည့်ခြင်း	Add Product
PUT	Data ပြင်ဆင်ခြင်း	Update Product
DELETE	Data ဖျက်ခြင်း	Delete Product

ဒီသင်ခန်းစာမှာ လေ့ကျင့်ရန် `https://fakestoreapi.com/products` ကို အသုံးပြုပါမယ်။ ဒီ API ဟာ Demo အတွက်ဖြစ်ပြီး Data ပြောင်းလဲမှုအားလုံးကို အမြဲတမ်း Database ထဲသိမ်းဆည်းပေးမယ်လို့ မယူဆသင့်ပါဘူး။

12.3. HTTP Package တပ်ဆင်ခြင်း

Terminal ကိုဖွင့်ပြီး Flutter Project Folder ထဲမှာ အောက်ပါ Command ကို Run ပါ။

```
flutter pub add http
```

ပြီးရင် `lib` Folder အောက်မှာ အောက်ပါ File Structure ကို တည်ဆောက်ပါ။

```
lib/
├── main.dart
├── models/
│   └── product.dart
├── services/
│   └── product_service.dart
└── screens/
    └── product_screen.dart
```

- `models/product.dart` — Product Data Model
- `services/product_service.dart` — REST API Request များ
- `screens/product_screen.dart` — Product List UI
- `main.dart` — Application Entry Point

12.4. JSON Parsing နှင့် Model Conversion

Server ကနေ အောက်ပါ JSON Data ရရှိတယ်လို့ ယူဆပါမယ်။

```
{
  "id": 1,
  "title": "Wireless Headphones",
  "price": 49.99,
  "description": "Bluetooth headphones",
  "category": "electronics",
  "image": "https://example.com/headphones.jpg"
}
```

ဒီ JSON Data ကို Dart Object အဖြစ် ပြောင်းလဲနိုင်ဖို့ `lib/models/product.dart` File ကို တည်ဆောက်ပါ။

```
class Product {
  final int id;
  final String title;
  final double price;
  final String description;
  final String category;
  final String image;

  Product({
    required this.id,
    required this.title,
    required this.price,
    required this.description,
    required this.category,
    required this.image,
  });

  factory Product.fromJson(Map<String, dynamic> json) {
    return Product(
      id: (json['id'] as num).toInt(),
      title: json['title'] as String,
      price: (json['price'] as num).toDouble(),
      description: json['description'] as String,
      category: json['category'] as String,
      image: json['image'] as String,
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'title': title,
      'price': price,
      'description': description,
      'category': category,
      'image': image,
    };
  }
}
```

ရှင်းလင်းချက်

- `Product.fromJson()` က JSON Map ကို Product Object အဖြစ် ပြောင်းပေးပါတယ်။
- `toJson()` က Product Object ကို JSON ပေးပို့နိုင်တဲ့ Map အဖြစ် ပြောင်းပေးပါတယ်။

- `num` ကနေ `double` ပြောင်းထားတာကြောင့် API မှာ Price ကို Integer သို့မဟုတ် Decimal ပုံစံနဲ့ ပေးပို့လာတဲ့အခါ ကိုင်တွယ်နိုင်ပါတယ်။

12.5. REST API Service တည်ဆောက်ခြင်း

`lib/services/product_service.dart` File ထဲမှာ GET, POST, PUT, DELETE လုပ်ဆောင်ချက်တွေကို စုစည်းရေးသားပါမယ်။

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/product.dart';

class ProductService {
  static const String baseUrl =
      'https://fakestoreapi.com/products';

  Future<List<Product>> getProducts() async {
    final response = await http.get(
      Uri.parse(baseUrl),
    );

    if (response.statusCode == 200) {
      final List<dynamic> data =
          jsonDecode(response.body) as List<dynamic>;

      return data
          .map((item) => Product.fromJson(
              item as Map<String, dynamic>))
          .toList();
    }

    throw Exception(
      'Product loading failed: ${response.statusCode}',
    );
  }

  Future<Product> createProduct(Product product) async {
    final response = await http.post(
      Uri.parse(baseUrl),
      headers: {
        'Content-Type': 'application/json',
      },
      body: jsonEncode({
        'title': product.title,
        'price': product.price,
        'description': product.description,
        'category': product.category,
        'image': product.image,
      }),
```

```
    );

    if (response.statusCode == 200 ||
        response.statusCode == 201) {
      return Product.fromJson(
        jsonDecode(response.body)
            as Map<String, dynamic>,
      );
    }

    throw Exception(
      'Product creation failed: ${response.statusCode}',
    );
  }

  Future<Product> updateProduct(Product product) async {
    final response = await http.put(
      Uri.parse('$baseUrl/${product.id}'),
      headers: {
        'Content-Type': 'application/json',
      },
      body: jsonEncode(product.toJson()),
    );

    if (response.statusCode == 200) {
      return Product.fromJson(
        jsonDecode(response.body)
            as Map<String, dynamic>,
      );
    }

    throw Exception(
      'Product update failed: ${response.statusCode}',
    );
  }

  Future<void> deleteProduct(int id) async {
    final response = await http.delete(
      Uri.parse('$baseUrl/$id'),
    );

    if (response.statusCode != 200 &&
        response.statusCode != 204) {
      throw Exception(
        'Product deletion failed: ${response.statusCode}',
      );
    }
  }
}
```

အရေးကြီးသော API Methods များ

- `http.get()` — Server မှ Data ရယူပါတယ်။
- `http.post()` — Server သို့ Data အသစ် ပေးပို့ပါတယ်။
- `http.put()` — ရှိပြီးသား Data ကို ပြင်ဆင်ဖို့ Request ပို့ပါတယ်။
- `http.delete()` — သတ်မှတ်ထားတဲ့ ID အလိုက် Data ဖျက်ဖို့ Request ပို့ပါတယ်။
- `jsonDecode()` — JSON String ကို Dart Object အဖြစ် ပြောင်းပေးပါတယ်။
- `jsonEncode()` — Dart Map ကို JSON String အဖြစ် ပြောင်းပေးပါတယ်။

`statusCode` ကို စစ်ဆေးခြင်းက API Request အောင်မြင်မှုကို သိရှိနိုင်စေပါတယ်။ HTTP Error များကိုသာမက Network Connection ပြတ်တောက်မှုကိုလည်း UI ဘက်မှာ `try-catch` ဖြင့် ကိုင်တွယ်ရပါမယ်။

12.6. Product List UI, Loading Indicator နှင့် Error Handling

`lib/screens/product_screen.dart` File ကို အောက်ပါအတိုင်း ရေးသားပါ။

```
import 'package:flutter/material.dart';
import '../models/product.dart';
import '../services/product_service.dart';

class ProductScreen extends StatefulWidget {
  const ProductScreen({super.key});

  @override
  State<ProductScreen> createState() =>
      _ProductScreenState();
}

class _ProductScreenState extends State<ProductScreen> {
  final ProductService service = ProductService();

  List<Product> products = [];
  bool isLoading = true;
  String? errorMessage;

  @override
  void initState() {
```

```
    super.initState();
    loadProducts();
  }

  Future<void> loadProducts() async {
    setState(() {
      isLoading = true;
      errorMessage = null;
    });

    try {
      final result = await service.getProducts();

      if (!mounted) return;

      setState(() {
        products = result;
        isLoading = false;
      });
    } catch (e) {
      if (!mounted) return;

      setState(() {
        errorMessage = 'Data ရယူရာတွင် အမှားဖြစ်နေပါတယ်။';
        isLoading = false;
      });
    }
  }

  Future<void> removeProduct(int id) async {
    try {
      await service.deleteProduct(id);

      if (!mounted) return;

      setState(() {
        products.removeWhere((p) => p.id == id);
      });

      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Product ဖျက်ပြီးပါပြီ။'),
        ),
      );
    } catch (e) {
      if (!mounted) return;

      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Product ဖျက်၍မရပါ။'),
        ),
      );
    }
  }
```

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Shopping Products'),
      actions: [
        IconButton(
          onPressed: loadProducts,
          icon: const Icon(Icons.refresh),
        ),
      ],
    ),
    body: Builder(
      builder: (context) {
        if (isLoading) {
          return const Center(
            child: CircularProgressIndicator(),
          );
        }

        if (errorMessage != null) {
          return Center(
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                Text(errorMessage!),
                const SizedBox(height: 12),
                ElevatedButton(
                  onPressed: loadProducts,
                  child: const Text('Retry'),
                ),
              ],
            ),
          );
        }

        if (products.isEmpty) {
          return const Center(
            child: Text('Product မရှိသေးပါ။'),
          );
        }

        return RefreshIndicator(
          onRefresh: loadProducts,
          child: ListView.builder(
            physics: const AlwaysScrollableScrollPhysics(),
            itemCount: products.length,
            itemBuilder: (context, index) {
              final product = products[index];

              return Card(
                margin: const EdgeInsets.symmetric(
                  horizontal: 12,
                  vertical: 6,
                ),
                child: ListTile(
```

```
                    leading: Image.network(
                      product.image,
                      width: 48,
                      height: 48,
                      fit: BoxFit.contain,
                      errorBuilder: (_, __, ___) =>
                          const Icon(Icons.image_not_supported),
                    ),
                    title: Text(
                      product.title,
                      maxLines: 2,
                      overflow: TextOverflow.ellipsis,
                    ),
                    subtitle: Text(
                      '\$${product.price.toStringAsFixed(2)}',
                    ),
                    trailing: IconButton(
                      icon: const Icon(
                        Icons.delete,
                        color: Colors.red,
                      ),
                      onPressed: () =>
                          removeProduct(product.id),
                    ),
                  ),
                );
              },
            ),
          );
        },
      ),
    );
  }
}
```

ဒီ Screen မှာ Application စဖွင့်ချိန် `initState()` ကနေ API ကို ခေါ်ပါတယ်။ Data ရယူနေစဉ် `CircularProgressIndicator` ပြသပြီး Error ဖြစ်လာတဲ့အခါ Retry Button ကို ပြသပေးပါတယ်။ ထို့အပြင် အောက်ကိုဆွဲချပြီး Data Refresh လုပ်နိုင်သလို Product တစ်ခုချင်းစီကို Delete လုပ်နိုင်ပါတယ်။

12.7. Main Application ကို ချိတ်ဆက်ခြင်း

`lib/main.dart` File ကို အောက်ပါအတိုင်း ပြင်ဆင်ပါ။

```
import 'package:flutter/material.dart';
import 'screens/product_screen.dart';

void main() {
```

```
  runApp(const ShoppingApp());
}

class ShoppingApp extends StatelessWidget {
  const ShoppingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Flutter Shopping App',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      home: const ProductScreen(),
    );
  }
}
```

ပြီးရင် Terminal မှာ အောက်ပါ Command ကို Run ပါ။

```
flutter pub get
flutter run
```

Internet Connection ရှိနေပါက Product List၊ ပုံများနဲ့ ဈေးနှုန်းများကို မြင်တွေ့ရမှာ ဖြစ်ပါတယ်။

12.8. POST, PUT, DELETE ကို အသုံးပြုပုံ

အထက်က Service မှာ CRUD API Methods တွေ ရေးပြီးပါပြီ။ UI မှာ အသုံးပြုဖို့ ဥပမာအနေနဲ့ အောက်ပါ Code ကို `ProductScreen` ရဲ့ State Class ထဲမှာ ထည့်နိုင်ပါတယ်။

Product အသစ်ထည့်ခြင်း

```
Future<void> addSampleProduct() async {
  final product = Product(
    id: 0,
    title: 'New T-Shirt',
    price: 19.99,
    description: 'Cotton T-Shirt',
    category: 'clothing',
    image: 'https://placehold.co/200x200',
  );

  try {
```

```
    final created =
        await service.createProduct(product);

    if (!mounted) return;

    setState(() {
      products.insert(0, created);
    });
  } catch (e) {
    if (!mounted) return;

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Product ထည့်၍မရပါ။'),
      ),
    );
  }
}
```

Product ပြင်ဆင်ခြင်း

```
Future<void> editSampleProduct(Product oldProduct) async {
  final updated = Product(
    id: oldProduct.id,
    title: '${oldProduct.title} - Updated',
    price: oldProduct.price + 1,
    description: oldProduct.description,
    category: oldProduct.category,
    image: oldProduct.image,
  );

  try {
    final result =
        await service.updateProduct(updated);

    if (!mounted) return;

    setState(() {
      final index = products.indexWhere(
        (p) => p.id == result.id,
      );

      if (index != -1) {
        products[index] = result;
      }
    });
  } catch (e) {
    if (!mounted) return;

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Product ပြင်ဆင်၍မရပါ။'),
```

```
        ),
      );
    }
}
```

`addSampleProduct()` နဲ့ `editSampleProduct()` ကို Button သို့မဟုတ် Form Submit မှာ ချိတ်ဆက်အသုံးပြုနိုင်ပါတယ်။

သတိပြုရန်: Fake Store API သည် လေ့ကျင့်ရေး Demo API ဖြစ်တဲ့အတွက် POST, PUT, DELETE Request များ အောင်မြင်တဲ့ Response ပြန်ပေးသော်လည်း Server Data ကို အမြဲတမ်း ပြောင်းလဲသိမ်းဆည်းထားမည်မဟုတ်ပါ။ ထို့ကြောင့် ဒီသင်ခန်းစာက HTTP CRUD Flow ကို လေ့လာရန်အတွက် ဖြစ်ပြီး အမှန်တကယ် Shopping Application အတွက် ကိုယ်ပိုင် Backend နှင့် Database လိုအပ်ပါတယ်။

12.9. Error Handling အတွက် အကြံပြုချက်များ

အခြေအနေ	ကိုင်တွယ်ပုံ
Internet မရှိခြင်း	`try-catch` ဖြင့် ဖမ်းယူပြီး Retry ပြသပါ
HTTP 404	Resource မတွေ့ကြောင်း ပြသပါ
HTTP 500	Server Error Message ပြသပါ
JSON Format မှားခြင်း	Parsing Error ကို ကိုင်တွယ်ပါ
Loading ကြာခြင်း	Timeout သတ်မှတ်ပြီး Retry ပေးပါ
Product Image မရခြင်း	`errorBuilder` ဖြင့် Placeholder ပြပါ

အမှန်တကယ် Production Application တွင် Request Timeout၊ Authentication Token၊ Pagination၊ API Response Validation နဲ့ User-Friendly Error Message များကိုလည်း ထည့်သွင်းစဉ်းစားသင့်ပါတယ်။

12.10. လက်တွေ့လေ့ကျင့်ခန်းများ

Exercises Checklist

1. API မှ Product List ကို ရယူပြီး GridView ဖြင့် ပြသပါ။
2. Product အသစ်ထည့်ရန် TextField နှင့် Form Validation တည်ဆောက်ပါ။
3. Product တစ်ခုကို Edit Screen သို့ ပို့ပြီး PUT Request ဖြင့် ပြင်ဆင်ပါ။
4. Delete မလုပ်မီ Confirmation Dialog ပြသပါ။
5. Loading Indicator အစား Shimmer Loading UI တည်ဆောက်ပါ။
6. Network Error ဖြစ်သောအခါ Retry Button နှင့် သက်ဆိုင်ရာ Error Message ပြပါ။

11.Summary

ဒီသင်ခန်းစာမှာ Flutter ကို REST API နဲ့ ချိတ်ဆက်ပြီး HTTP GET, POST, PUT, DELETE Request များ ပို့ခြင်း၊ JSON ကို Model Object အဖြစ် ပြောင်းလဲခြင်း၊ Loading Indicator နဲ့ Error Handling တို့ကို တည်ဆောက်ခဲ့ပါတယ်။

နောက်သင်ခန်းစာ Lesson 13: User Authentication and Login System မှာ Login Form၊ Register Form၊ API Authentication၊ Token Storage နဲ့ Protected Screens များကို တည်ဆောက်သွားနိုင်ပါတယ်။

CHAPTER 13
Firebase Authentication and User Login



ဒီ Chapter မှာ Flutter Shopping Application အတွက် Firebase Authentication ကို အသုံးပြုပြီး User Registration, Login, Logout, Password Reset နဲ့ Authentication State Management တို့ကို အဆင့်ဆင့် တည်ဆောက်သွားမှာ ဖြစ်ပါတယ်။

သင်ခန်းစာပြီးဆုံးတဲ့အခါ User တစ်ယောက်က Account အသစ်ဖွင့်ခြင်း၊ Login ဝင်ခြင်း၊ Password မေ့သွားတဲ့အခါ ပြန်လည်သတ်မှတ်ခြင်းနဲ့ Login ဝင်ထားခြင်း ရှိ၊ မရှိအလိုက် သင့်တော်တဲ့ Screen ကို ပြသခြင်းတို့ကို လုပ်ဆောင်နိုင်မှာ ဖြစ်ပါတယ်။

13.1. Firebase Authentication ဆိုတာဘာလဲ?

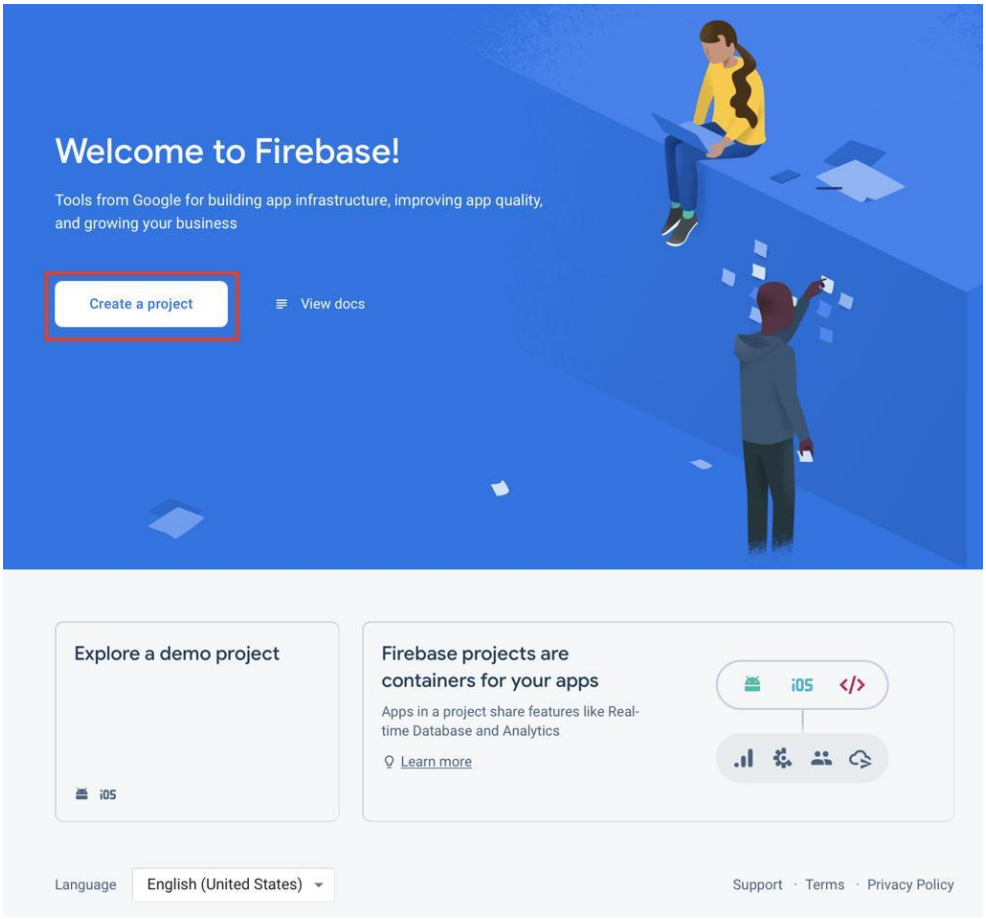
Firebase Authentication ဆိုတာ Google Firebase က ပေးထားတဲ့ User Identity Management Service တစ်ခုဖြစ်ပါတယ်။ ကိုယ်ပိုင် Backend Authentication System ကို အစအဆုံး ရေးသားစရာမလိုဘဲ User Account များကို စီမံခန့်ခွဲနိုင်ပါတယ်။

ဒီသင်ခန်းစာမှာ အသုံးပြုမယ့် Features တွေကတော့ -

- Register — Email နဲ့ Password အသုံးပြုပြီး Account အသစ်ဖန်တီးခြင်း
- Login — ရှိပြီးသား Account ဖြင့် ဝင်ရောက်ခြင်း

- Logout — လက်ရှိ Account မှ ထွက်ခြင်း
- Password Reset — Email မှတစ်ဆင့် Password ပြန်သတ်မှတ်ခြင်း
- Authentication State — Login ဝင်ထားခြင်း ရှိ၊ မရှိကို အလိုအလျောက် စစ်ဆေးခြင်း

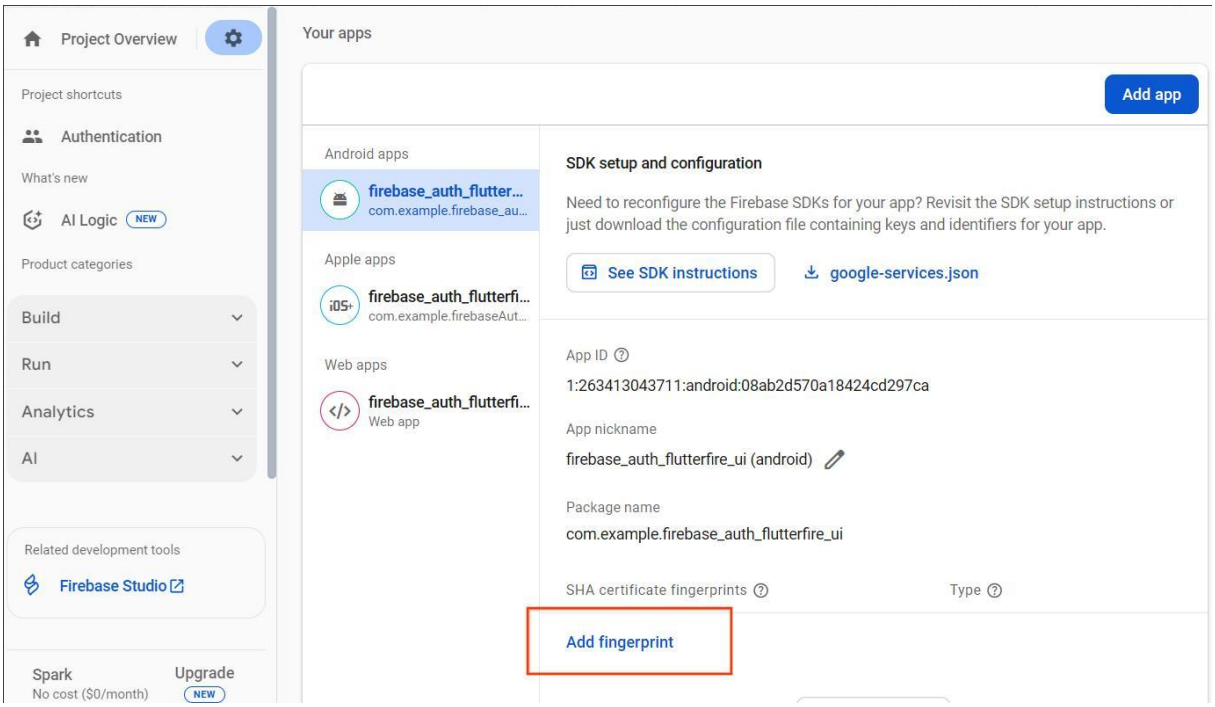
13.2. Firebase Project တည်ဆောက်ခြင်း



အဆင့် 1 — Firebase Console ဖွင့်ပါ

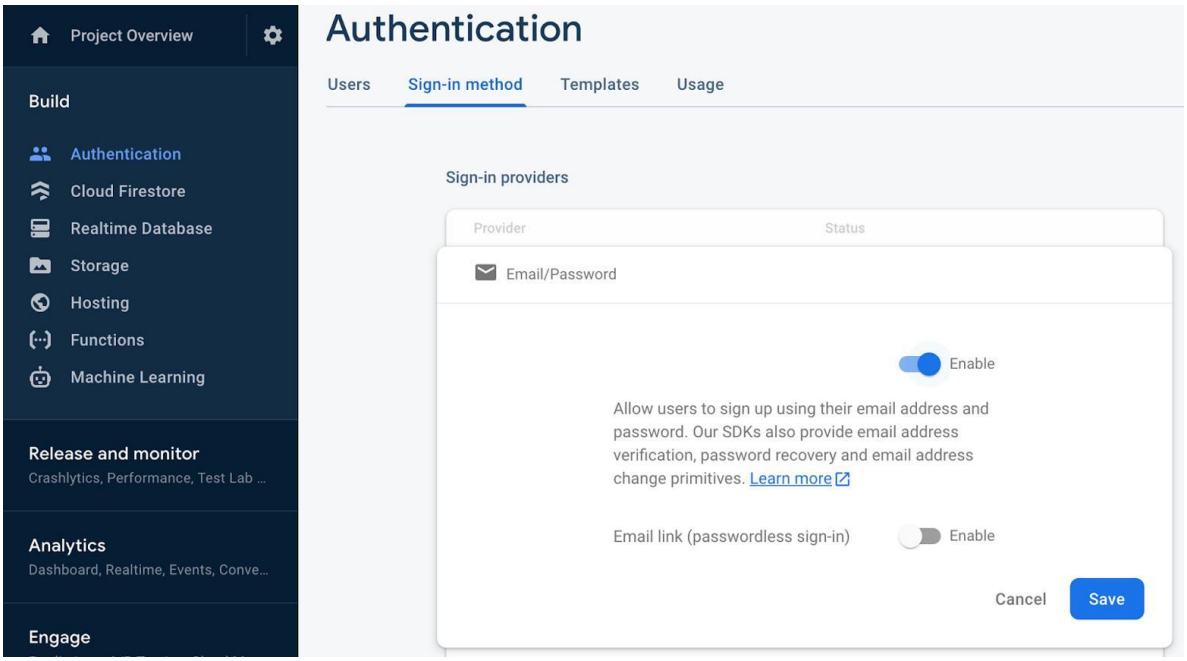
Firebase Console သို့သွားပြီး Google Account ဖြင့် Login ဝင်ပါ။

Firebase Console



အဆင့် 2 — Project အသစ်ဖန်တီးပါ

Project name ကို `flutter_shopping_app` လို့ ပေးနိုင်ပါတယ်။ ပြီးရင် Project ကို Create လုပ်ပါ။



အဆင့် 3 — Email/Password Authentication ဖွင့်ပါ

Firebase Console → Build → Authentication → Get started → Sign-in method → Email/Password ကို ရွေးပြီး Enable လုပ်ပါ။

13.3. Flutter Project ကို Firebase နှင့် ချိတ်ဆက်ခြင်း

Flutter Project ရဲ့ Root Folder ကို Terminal ဖြင့် ဖွင့်ပါ။

```
flutter pub add firebase_core
flutter pub add firebase_auth
```

Firebase နဲ့ Flutter Project ကို ချိတ်ဆက်ရန် Firebase CLI နဲ့ FlutterFire CLI ကို အသုံးပြုနိုင်ပါတယ်။

```
npm install -g firebase-tools
firebase login
dart pub global activate flutterfire_cli
flutterfire configure
```

`flutterfire configure` ကို Run လုပ်တဲ့အခါ Firebase Project ကို ရွေးချယ်ပြီး Android App အပါအဝင် အသုံးပြုမယ့် Platform များကို Configure လုပ်ပါ။

ဒီ Command ပြီးသွားတဲ့အခါ ပုံမှန်အားဖြင့် `lib/firebase_options.dart` ဖိုင်ကို ဖန်တီးပေးမှာ ဖြစ်ပါတယ်။

မှတ်ချက် — FlutterFire CLI ကို Terminal မှာ မတွေ့ပါက Dart Pub Cache ရဲ့ executable directory ကို PATH ထဲ ထည့်ထားကြောင်း စစ်ဆေးပါ။

13.4. Firebase Initialization — main.dart

ဖိုင် `lib/main.dart` ကို အောက်ပါအတိုင်း ပြင်ဆင်ပါ။ Firebase ကို Application မစတင်ခင် Initialize လုပ်ထားရပါမယ်။

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

import 'auth_gate.dart';

Future<void> main() async {
```

```
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  runApp(const ShoppingApp());
}

class ShoppingApp extends StatelessWidget {
  const ShoppingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Flutter Shopping App',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.indigo,
        ),
        useMaterial3: true,
      ),
      home: const AuthGate(),
    );
  }
}
```

`AuthGate` က User Login ဝင်ထားခြင်း ရှိ၊ မရှိ စစ်ဆေးပေးမယ့် Widget ဖြစ်ပါတယ်။ နောက်အပိုင်းမှာ ဒီဖိုင်ကို တည်ဆောက်ပါမယ်။

13.5. Authentication State စစ်ဆေးခြင်း — auth_gate.dart

Firebase ရဲ့ `authStateChanges()` က User ရဲ့ Authentication State ပြောင်းလဲမှုကို Stream အဖြစ် ပေးပါတယ်။ User Login ဝင်တဲ့အခါ Home Screen ကို ပြသပြီး Logout လုပ်တဲ့အခါ Login Screen ကို ပြန်ပြသနိုင်ပါတယ်။

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

import 'auth_page.dart';
import 'home_page.dart';

class AuthGate extends StatelessWidget {
  const AuthGate({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return StreamBuilder<User?>(
      stream: FirebaseAuth.instance.authStateChanges(),
      builder: (context, snapshot) {
        if (snapshot.connectionState ==
            ConnectionState.waiting) {
          return const Scaffold(
            body: Center(
              child: CircularProgressIndicator(),
            ),
          );
        }

        if (snapshot.hasData) {
          return const HomePage();
        }

        return const AuthPage();
      },
    );
  }
}
```

အလုပ်လုပ်ပုံ

- `snapshot.connectionState` က စောင့်ဆိုင်းနေစဉ် Loading ပြပါတယ်။
- `snapshot.hasData` က `true` ဖြစ်ရင် Login ဝင်ထားတဲ့ User ရှိပါတယ်။
- User မရှိရင် `AuthPage` ကို ပြပါတယ်။
- Login/Logout လုပ်တိုင်း Stream က Update ဖြစ်တဲ့အတွက် Screen ကို ကိုယ်တိုင် Navigate လုပ်စရာ မလိုပါဘူး။

13.6. Register, Login နှင့် Password Reset — auth_page.dart

ဒီဖိုင်တစ်ခုတည်းမှာ Login Form၊ Registration Form နဲ့ Forgot Password Form တို့ကို ထည့်သွင်းတည်ဆောက်ပါမယ်။

Firebase Authentication မှာ Registration အတွက် `createUserWithEmailAndPassword()`၊ Login အတွက် `signInWithEmailAndPassword()` နဲ့ Password Reset အတွက် `sendPasswordResetEmail()` တို့ကို အသုံးပြုပါတယ်။

အောက်ပါ Code က Email/Password Authentication အတွက် အလုပ်လုပ်နိုင်တဲ့ UI နှင့် Error Handling ပါဝင်သော ဥပမာဖြစ်ပါတယ်။

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

class AuthPage extends StatefulWidget {
  const AuthPage({super.key});

  @override
  State<AuthPage> createState() => _AuthPageState();
}

class _AuthPageState extends State<AuthPage> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();

  bool isLogin = true;
  bool isLoading = false;
  bool hidePassword = true;

  final FirebaseAuth _auth = FirebaseAuth.instance;

  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  void showMessage(String message) {
    if (!mounted) return;

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(message)),
    );
  }

  String getAuthError(FirebaseAuthException e) {
    switch (e.code) {
      case 'email-already-in-use':
        return 'ဒီ Email ဖြင့် Account ရှိပြီးသားပါ။';
      case 'invalid-email':
        return 'Email လိပ်စာ မှားယွင်းနေပါတယ်။';
      case 'weak-password':
        return 'Password ပိုမိုခိုင်မာအောင် ပြုလုပ်ပါ။';
      case 'invalid-credential':
      case 'wrong-password':
      case 'user-not-found':
        return 'Email သို့မဟုတ် Password မှားယွင်းနေပါတယ်။';
```

```
      case 'network-request-failed':
        return 'Internet Connection ကို စစ်ဆေးပါ။';
      case 'too-many-requests':
        return 'ကြိုးစားမှုများလွန်းပါတယ်။ ခဏစောင့်ပြီး ပြန်လုပ်ပါ။';
      case 'operation-not-allowed':
        return 'Firebase Console တွင် Email/Password ကို Enable လုပ်ပါ။';
      default:
        return e.message ?? 'လုပ်ဆောင်မှု မအောင်မြင်ပါ။';
    }
  }

  Future<void> submitAuth() async {
    if (!_formKey.currentState!.validate()) return;

    setState(() => isLoading = true);

    try {
      final email = _emailController.text.trim();
      final password = _passwordController.text;

      if (isLogin) {
        await _auth.signInWithEmailAndPassword(
          email: email,
          password: password,
        );
      } else {
        await _auth.createUserWithEmailAndPassword(
          email: email,
          password: password,
        );
      }

      // AuthGate က Login State ကို အလိုအလျောက် စစ်ဆေးပါမယ်။
    } on FirebaseAuthException catch (e) {
      showMessage(getAuthError(e));
    } catch (_) {
      showMessage('မမျှော်လင့်ထားသော Error ဖြစ်ပွားခဲ့ပါတယ်။');
    } finally {
      if (mounted) {
        setState(() => isLoading = false);
      }
    }
  }

  Future<void> resetPassword() async {
    final email = _emailController.text.trim();

    if (email.isEmpty || !email.contains('@')) {
      showMessage('Password Reset အတွက် Email မှန်ကန်စွာ ထည့်ပါ။');
      return;
    }
```

```
    setState(() => isLoading = true);

    try {
      await _auth.sendPasswordResetEmail(email: email);
      showMessage(
        'Password Reset Email ပို့ပြီးပါပြီ။ Inbox ကို စစ်ဆေးပါ။',
      );
    } on FirebaseAuthException catch (e) {
      showMessage(getAuthError(e));
    } catch (_) {
      showMessage('Email ပို့ရာတွင် Error ဖြစ်ပွားခဲ့ပါတယ်။');
    } finally {
      if (mounted) {
        setState(() => isLoading = false);
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Shopping App Authentication'),
        centerTitle: true,
      ),
      body: Center(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: ConstrainedBox(
            constraints: const BoxConstraints(maxWidth: 420),
            child: Form(
              key: _formKey,
              child: Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  const Icon(
                    Icons.shopping_bag,
                    size: 80,
                    color: Colors.indigo,
                  ),
                  const SizedBox(height: 20),
                  Text(
                    isLogin ? 'User Login' : 'Create Account',
                    style: Theme.of(context)
                        .textTheme.headlineMedium,
                  ),
                  const SizedBox(height: 24),
                  TextFormField(
                    controller: _emailController,
                    keyboardType: TextInputType.emailAddress,
                    decoration: const InputDecoration(
                      labelText: 'Email Address',
                      prefixIcon: Icon(Icons.email),
                      border: OutlineInputBorder(),
                    ),
```

```
  validator: (value) {
    final email = value?.trim() ?? '';
    if (email.isEmpty || !email.contains('@')) {
      return 'မှန်ကန်သော Email ထည့်ပါ။';
    }
    return null;
  },
),
const SizedBox(height: 16),
TextFormField(
  controller: _passwordController,
  obscureText: hidePassword,
  decoration: InputDecoration(
    labelText: 'Password',
    prefixIcon: const Icon(Icons.lock),
    border: const OutlineInputBorder(),
    suffixIcon: IconButton(
      icon: Icon(
        hidePassword
            ? Icons.visibility
            : Icons.visibility_off,
      ),
      onPressed: () {
        setState(() {
          hidePassword = !hidePassword;
        });
      },
    ),
  ),
  validator: (value) {
    if (value == null || value.isEmpty) {
      return 'Password ထည့်ပါ။';
    }

    if (!isLogin && value.length < 6) {
      return 'Password အနည်းဆုံး 6 လုံး ထည့်ပါ။';
    }

    return null;
  },
),
const SizedBox(height: 20),
SizedBox(
  width: double.infinity,
  height: 48,
  child: ElevatedButton(
    onPressed: isLoading ? null : submitAuth,
    child: isLoading
        ? const CircularProgressIndicator()
        : Text(
            isLogin ? 'Login' : 'Register',
          ),
  ),
),
if (isLogin)
```

```
                    TextButton(
                      onPressed:
                          isLoading ? null : resetPassword,
                      child: const Text('Forgot Password?'),
                    ),
                  TextButton(
                    onPressed: isLoading
                        ? null
                        : () {
                            setState(() {
                              isLogin = !isLogin;
                            });
                          },
                    child: Text(
                      isLogin
                          ? 'Create a new account'
                          : 'Already have an account? Login',
                    ),
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

Code ရှင်းလင်းချက်

1. `GlobalKey<FormState>` ကို အသုံးပြုပြီး Form Validation စစ်ဆေးပါတယ်။
2. `TextEditingController` က Email နဲ့ Password တို့ရဲ့ Input Value ကို ဖတ်ယူပေးပါတယ်။
3. `isLogin` က `true` ဖြစ်ရင် Login၊ `false` ဖြစ်ရင် Register လုပ်ဆောင်ပါတယ်။
4. `FirebaseAuthException` က Firebase မှ ပြန်လာတဲ့ Error များကို ဖမ်းယူပြီး သက်ဆိုင်ရာ Message ပြသပါတယ်။
5. `resetPassword()` က Email သို့ Password Reset Link ပို့ပေးပါတယ်။
6. `dispose()` က Controller များ အသုံးမပြုတော့တဲ့အခါ Memory ကို သင့်တော်စွာ ရှင်းလင်းပေးပါတယ်။

မှတ်ချက် - အနည်းဆုံး Password 6 လုံးဆိုတဲ့ Validation ဟာ ဒီဥပမာရဲ့သတ်မှတ်ချက်ဖြစ်ပါတယ်။ Firebase Console မှာ Password Policy ပိုမိုတင်းကျပ်စွာ သတ်မှတ်ထားရင် အဲဒီ Policy နဲ့လည်း ကိုက်ညီဖို့ လိုပါတယ်။

13.7. Login ဝင်ပြီးနောက် Home Screen — home_page.dart

User Login အောင်မြင်သွားတဲ့အခါ `AuthGate` က ဒီ Screen ကို အလိုအလျောက် ပြပေးမှာ ဖြစ်ပါတယ်။

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

class HomePage extends StatelessWidget {
  const HomePage({super.key});

  Future<void> logout(BuildContext context) async {
    try {
      await FirebaseAuth.instance.signOut();
    } on FirebaseAuthException catch (e) {
      if (!context.mounted) return;

      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text(e.message ?? 'Logout မအောင်မြင်ပါ။'),
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    final user = FirebaseAuth.instance.currentUser;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Shopping Home'),
        actions: [
          IconButton(
            tooltip: 'Logout',
            onPressed: () => logout(context),
            icon: const Icon(Icons.logout),
          ),
        ],
      ),
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(24),
```

```
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                const Icon(
                  Icons.account_circle,
                  size: 90,
                  color: Colors.indigo,
                ),
                const SizedBox(height: 20),
                const Text(
                  'Welcome to Shopping App!',
                  style: TextStyle(
                    fontSize: 22,
                    fontWeight: FontWeight.bold,
                  ),
                ),
                const SizedBox(height: 12),
                Text('Email: ${user?.email ?? "Unknown"}'),
                const SizedBox(height: 8),
                Text('User ID: ${user?.uid ?? "Unknown"}'),
                const SizedBox(height: 24),
                ElevatedButton.icon(
                  onPressed: () => logout(context),
                  icon: const Icon(Icons.logout),
                  label: const Text('Logout'),
                ),
              ],
            ),
          ),
        ),
      );
  }
}
```

ဒီနေရာမှာ `FirebaseAuth.instance.currentUser` က လက်ရှိ Login ဝင်ထားတဲ့ User ရဲ့ အချက်အလက်ကို ရယူပေးပါတယ်။ `uid` က User တစ်ယောက်ချင်းစီအတွက် သီးခြား ID ဖြစ်ပြီး နောက်ပိုင်း Shopping Cart၊ Order History နဲ့ User Profile တို့ကို ဆက်စပ်သိမ်းဆည်းတဲ့အခါ အသုံးဝင်ပါတယ်။

13.8. Project Folder Structure

```
flutter_shopping_app/
├── lib/
│   ├── main.dart
│   ├── firebase_options.dart
│   ├── auth_gate.dart
│   ├── auth_page.dart
│   └── home_page.dart
├── android/
├── ios/
├── web/
```

```
└── pubspec.yaml
```

ဒီသင်ခန်းစာအတွက် လိုအပ်တဲ့ ဖိုင်တွေကို အထက်ပါအတိုင်း စီစဉ်ထားနိုင်ပါတယ်။
`firebase_options.dart` ကို ကိုယ်တိုင်ရေးစရာမလိုဘဲ FlutterFire CLI က ဖန်တီးပေးမှာ ဖြစ်ပါတယ်။

13.9. Application ကို Run ပြီး စမ်းသပ်ခြင်း

Terminal မှာ အောက်ပါ Command ကို အသုံးပြုပါ။

```
flutter pub get
flutter run
```

Testing Checklist

1. Register
   Email အသစ်နှင့် Password အနည်းဆုံး 6 လုံးထည့်ပြီး Account ဖန်တီးပါ။
2. Login
   Register လုပ်ထားတဲ့ Email နှင့် Password ဖြင့် Login ဝင်ပါ။
3. Authentication State
   App ကို ပိတ်ပြီး ပြန်ဖွင့်ပါ။ Login Session ရှိနေပါက Home Screen ပြသသင့်ပါတယ်။
4. Password Reset
   Login Screen မှ Email ထည့်ပြီး Forgot Password ကို နှိပ်ပါ။ Inbox ထဲက Link ကို စစ်ဆေးပါ။
5. Logout
   Logout ကို နှိပ်ပြီး Login Screen ကို ပြန်ရောက်ကြောင်း စစ်ဆေးပါ။
6. Invalid Input
   Email မှားခြင်း၊ Password မမှန်ခြင်းနှင့် Internet မရှိခြင်းတို့ကို စမ်းသပ်ပါ။

Firebase Console → Authentication → Users ထဲမှာ Register လုပ်ထားတဲ့ User တွေကိုလည်း စစ်ဆေးနိုင်ပါတယ်။

13.10. မကြာခဏ ကြုံတွေ့ရတဲ့ Error များ

Error	ဖြေရှင်းနည်း
`operation-not-allowed`	Firebase Console မှာ Email/Password ကို Enable လုပ်ပါ။
`email-already-in-use`	အခြား Email အသစ်ဖြင့် Register လုပ်ပါ။
`weak-password`	Firebase Password Policy နှင့် ကိုက်ညီအောင် ပြင်ပါ။
`invalid-credential`	Email နှင့် Password မှန်ကန်မှုကို စစ်ဆေးပါ။
`network-request-failed`	Internet Connection ကို စစ်ဆေးပါ။
`No Firebase App '[DEFAULT]'`	`Firebase.initializeApp()` ကို `runApp()` မတိုင်ခင် ခေါ်ထားကြောင်း စစ်ဆေးပါ။

13.11. လုံခြုံရေးအတွက် သတိပြုရန်

- Password ကို Firestore သို့မဟုတ် အခြား Database ထဲမှာ ကိုယ်တိုင် Plain Text အနေနဲ့ မသိမ်းပါနဲ့။
- User Login ဝင်ထားခြင်းတစ်ခုတည်းနဲ့ Database ထဲက User အားလုံးရဲ့ Data ကို ဖတ်ခွင့်မပေးသင့်ပါဘူး။
- နောက်ပိုင်း Firestore အသုံးပြုတဲ့အခါ Firebase Security Rules မှာ `request.auth.uid` နဲ့ ပိုင်ရှင်ဖြစ်ကြောင်း စစ်ဆေးပါ။
- Shopping Order၊ Payment နဲ့ Admin လုပ်ဆောင်ချက်များကို Client UI တစ်ခုတည်းအပေါ် မူတည်ပြီး ခွင့်ပြုမထားသင့်ပါဘူး။

13.12. လေ့ကျင့်ခန်း

1. Register Form ထဲမှာ Confirm Password Field ထပ်ထည့်ပါ။
2. Password နှစ်ခု မတူရင် Error Message ပြပါ။

3. Login အောင်မြင်ပြီးနောက် User ရဲ့ Email ကို Home Screen မှာ ပြပါ။
4. Logout Button ကို AppBar မှာသာထားပြီး ထပ်မံစမ်းသပ်ပါ။
5. User တစ်ယောက် Login ဝင်ထားချိန်မှာ App ကို ပိတ်ပြီး ပြန်ဖွင့်ကြည့်ပါ။

13.13. Summary

ဒီသင်ခန်းစာမှာ Firebase Authentication ကို Flutter Shopping Application နဲ့ ချိတ်ဆက်ပြီး Email/Password Registration, Login, Logout, Password Reset နဲ့ Authentication State Management တို့ကို တည်ဆောက်ခဲ့ပါတယ်။

နောက်သင်ခန်းစာမှာ Lesson 14: Firestore Cloud Database Integration ကို ဆက်လက်လေ့လာပြီး Product များကို Cloud Database ထဲမှာ သိမ်းဆည်းခြင်း၊ ဖတ်ယူခြင်း၊ Update လုပ်ခြင်း၊ Delete လုပ်ခြင်းနဲ့ User ID အလိုက် Order Data များကို ခွဲခြားစီမံခြင်းတို့ကို တည်ဆောက်နိုင်ပါတယ်။

CHAPTER 14

Building a Complete Flutter Application



Shopping App Project တွင် Authentication, Navigation, CRUD, Database နှင့် State Management တို့ကို ပေါင်းစပ်တည်ဆောက်ခြင်း

ဒီ Chapter မှာ ယခင် Chapter 1 မှ Chapter 13 အထိ လေ့လာခဲ့တဲ့ Flutter Widgets, Navigation, Form Validation, CRUD Operations, Provider, SQLite Database, REST API နှင့် Firebase Authentication တို့ကို Shopping Application Project တစ်ခုထဲမှာ ပေါင်းစပ်အသုံးပြုသွားမှာ ဖြစ်ပါတယ်။

ဒီ Project ရဲ့ရည်ရွယ်ချက်က Product များကို ကြည့်ရှုနိုင်ခြင်း၊ Login ဝင်နိုင်ခြင်း၊ Product အသစ်ထည့်ခြင်း၊ ပြင်ဆင်ခြင်း၊ ဖျက်ခြင်း၊ Shopping Cart ထဲထည့်ခြင်းနှင့် Order ပြုလုပ်ခြင်းတို့ကို လေ့ကျင့်တည်ဆောက်နိုင်ဖို့ ဖြစ်ပါတယ်။

14.1. Shopping App ရဲ့အဓိကလုပ်ဆောင်ချက်များ

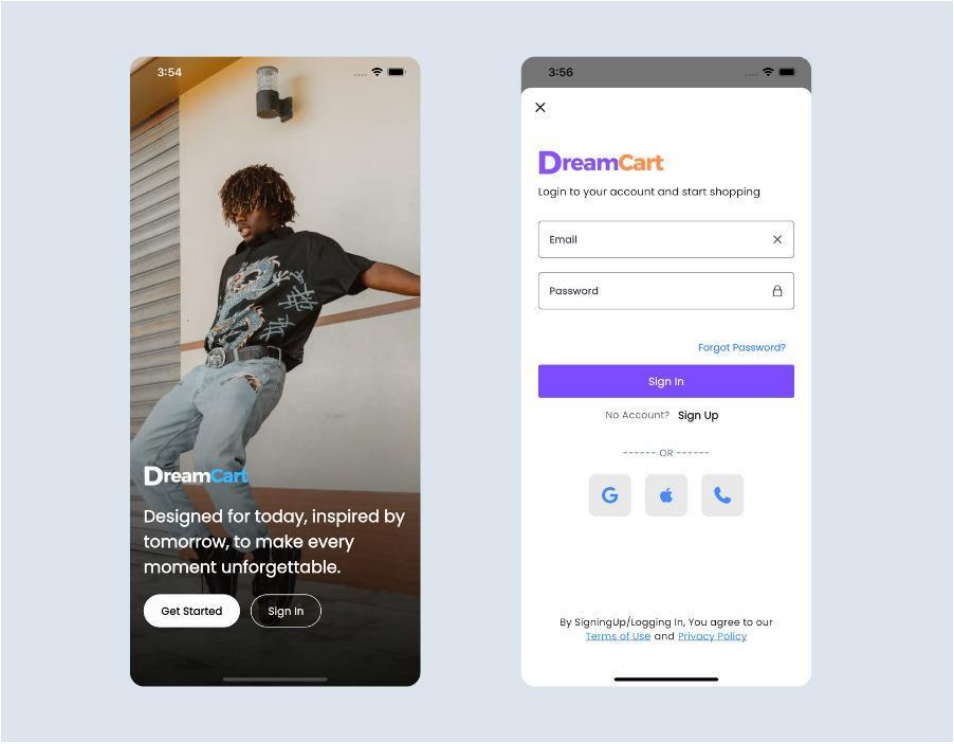


Figure 14.1. Login Page

1. Authentication

- User Registration နှင့် Login
- Logout နှင့် Password Reset
- Firebase Authentication ဖြင့် User စစ်ဆေးခြင်း

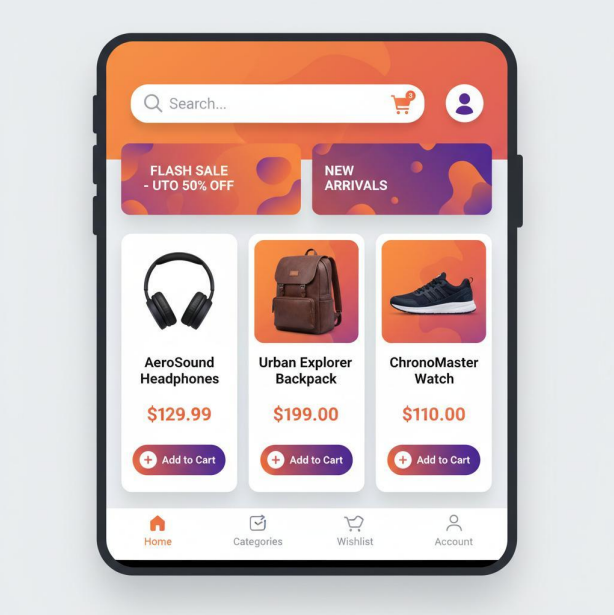


Figure 14.2. Home Page

2. Product Management

- Product List နှင့် Product Details
- Create, Read, Update, Delete (CRUD)
- Product Name, Price, Description, Image

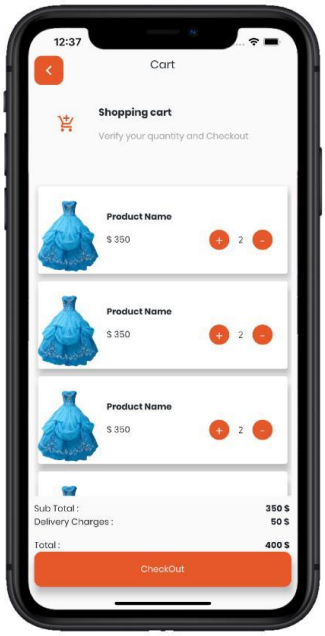


Figure 14.3. Shopping Cart page

3. Shopping Cart နှင့် Orders

- Cart ထဲ Product ထည့်ခြင်း
- Quantity တိုးခြင်း၊ လျှော့ခြင်း
- Total Price တွက်ချက်ခြင်း
- Order တင်ခြင်းနှင့် Order History

14.2. အသုံးပြုမည့် Technologies

Technology	အသုံးပြုမည့်နေရာ
Flutter + Dart	Mobile UI နှင့် Application Logic
Provider	Shared State Management
SQLite (`sqflite`)	Local Product နှင့် Order Data
Firebase Auth	Register, Login, Logout
REST API (`http`)	Server Data နှင့် ချိတ်ဆက်ခြင်း
`image_picker`	Product ဓာတ်ပုံရွေးချယ်ခြင်း

မှတ်ချက်။ SQLite သည် Device ပေါ် ရှိ Local Database ဖြစ်ပြီး Firebase Authentication သည် User Identity ကို စစ်ဆေးပေးတာဖြစ်ပါတယ်။ Product Data ကို Device အများအပြားအကြား မျှဝေလိုပါက Backend API သို့မဟုတ် Cloud Database ကို ထပ်မံချိတ်ဆက်ဖို့ လိုအပ်ပါတယ်။

14.3. Project Folder Structure

```
shopping_app/
├── lib/
│   ├── main.dart
│   ├── models/
│   │   ├── product.dart
│   │   ├── cart_item.dart
│   │   └── order.dart
│   ├── database/
│   │   └── app_database.dart
│   ├── providers/
│   │   ├── auth_provider.dart
│   │   ├── product_provider.dart
│   │   └── cart_provider.dart
│   ├── services/
```

```
│   │   └── api_service.dart
│   ├── screens/
│   │   ├── login_screen.dart
│   │   ├── register_screen.dart
│   │   ├── home_screen.dart
│   │   ├── product_detail_screen.dart
│   │   ├── product_form_screen.dart
│   │   ├── cart_screen.dart
│   │   ├── checkout_screen.dart
│   │   └── order_history_screen.dart
│   └── widgets/
│       ├── product_card.dart
│       └── loading_widget.dart
├── android/
├── test/
└── pubspec.yaml
```

ဒီ Folder Structure က Model, Database, State Management, Services, Screens နှင့် Reusable Widgets တို့ကို ခွဲခြားထားတာကြောင့် Project ကြီးလာတဲ့အခါ Source Code ကို ပြင်ဆင်ထိန်းသိမ်းရတာ ပိုမိုလွယ်ကူစေပါတယ်။

14.4. Project စတင်တည်ဆောက်ခြင်း

Terminal သို့မဟုတ် VS Code Terminal မှာ အောက်ပါ Command များကို Run ပါ။

```
flutter create shopping_app
cd shopping_app
flutter pub add provider
flutter pub add sqflite
flutter pub add path
flutter pub add path_provider
flutter pub add http
flutter pub add firebase_core
flutter pub add firebase_auth
flutter pub add image_picker
flutter pub add intl
```

Firebase အတွက် FlutterFire CLI ဖြင့် Project Configuration ပြုလုပ်ရန် လိုအပ်ပါတယ်။

```
dart pub global activate flutterfire_cli
flutterfire configure
```

Firebase Console ထဲမှာလည်း Authentication အတွက် Email/Password Sign-in Method ကို Enable လုပ်ထားရပါမယ်။

14.5. Product Model တည်ဆောက်ခြင်း

lib/models/product.dart

```
class Product {
  final int? id;
  final String name;
  final double price;
  final String description;
  final String image;

  Product({
    this.id,
    required this.name,
    required this.price,
    required this.description,
    required this.image,
  });

  Map<String, dynamic> toMap() {
    return {
      'id': id,
      'name': name,
      'price': price,
      'description': description,
      'image': image,
    };
  }

  factory Product.fromMap(Map<String, dynamic> map) {
    return Product(
      id: map['id'] as int?,
      name: map['name'] as String,
      price: (map['price'] as num).toDouble(),
      description: map['description'] as String,
      image: map['image'] as String,
    );
  }
}
```

ဒီ Model က Product တစ်ခုချင်းစီရဲ့ Data Structure ကို သတ်မှတ်ပေးပါတယ်။ `toMap()` ကို Database ထဲသိမ်းဖို့ အသုံးပြုပြီး `fromMap()` ကို Database က Data ပြန်ဖတ်တဲ့အခါ Object အဖြစ် ပြောင်းလဲဖို့ အသုံးပြုပါတယ်။

14.6. SQLite Database တည်ဆောက်ခြင်း

```
lib/database/app_database.dart

import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class AppDatabase {
  static Database? _database;

  Future<Database> get database async {
    if (_database != null) return _database!;

    final dbPath = await getDatabasesPath();

    _database = await openDatabase(
      join(dbPath, 'shopping.db'),
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE products (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            price REAL NOT NULL,
            description TEXT NOT NULL,
            image TEXT NOT NULL
          )
        ''');

        await db.execute('''
          CREATE TABLE orders (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            userId TEXT NOT NULL,
            total REAL NOT NULL,
            createdAt TEXT NOT NULL
          )
        ''');
      },
    );

    return _database!;
  }
}
```

ဒီနေရာမှာ Product နဲ့ Order အတွက် Table နှစ်ခု တည်ဆောက်ထားပါတယ်။ နောက်ပိုင်းမှာ Database Class ထဲကို Product Insert, Select, Update, Delete နဲ့ Order Insert လုပ်ဆောင်ချက်တွေကို ထပ်ထည့်သွားနိုင်ပါတယ်။

14.7. Product CRUD နှင့် State Management

lib/providers/product_provider.dart

```
import 'package:flutter/foundation.dart';
import '../database/app_database.dart';
import '../models/product.dart';

class ProductProvider extends ChangeNotifier {
  final AppDatabase _appDatabase = AppDatabase();

  List<Product> _products = [];
  bool _isLoading = false;

  List<Product> get products => List.unmodifiable(_products);
  bool get isLoading => _isLoading;

  Future<void> loadProducts() async {
    _isLoading = true;
    notifyListeners();

    try {
      final db = await _appDatabase.database;
      final rows = await db.query('products', orderBy: 'id DESC');

      _products = rows.map(Product.fromMap).toList();
    } finally {
      _isLoading = false;
      notifyListeners();
    }
  }

  Future<void> addProduct(Product product) async {
    final db = await _appDatabase.database;

    await db.insert('products', product.toMap());

    await loadProducts();
  }

  Future<void> updateProduct(Product product) async {
    if (product.id == null) {
      throw ArgumentError('Product ID is required');
    }

    final db = await _appDatabase.database;

    await db.update(
      'products',
      product.toMap(),
      where: 'id = ?',
      whereArgs: [product.id],
    );

    await loadProducts();
  }

  Future<void> deleteProduct(int id) async {
```

```
    final db = await _appDatabase.database;

    await db.delete(
      'products',
      where: 'id = ?',
      whereArgs: [id],
    );

    await loadProducts();
  }
}
```

Provider က Database နဲ့ UI ကြားမှာ အလုပ်လုပ်ပေးပါတယ်။ Product အသစ်ထည့်ခြင်း သို့မဟုတ် ပြင်ဆင်ခြင်း၊ ဖျက်ခြင်း ပြုလုပ်ပြီးတဲ့အခါ `loadProducts()` နဲ့ Data ပြန်ဖတ်ကာ `notifyListeners()` ဖြင့် UI ကို Update လုပ်ပေးပါတယ်။

14.8. Firebase Authentication ပေါင်းစပ်ခြင်း

```
lib/providers/auth_provider.dart

import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/foundation.dart';

class AuthProvider extends ChangeNotifier {
  final FirebaseAuth _auth = FirebaseAuth.instance;

  User? get user => _auth.currentUser;
  bool get isLoggedIn => user != null;

  Future<void> register(
    String email,
    String password,
  ) async {
    await _auth.createUserWithEmailAndPassword(
      email: email.trim(),
      password: password,
    );

    notifyListeners();
  }

  Future<void> login(
    String email,
    String password,
  ) async {
    await _auth.signInWithEmailAndPassword(
      email: email.trim(),
      password: password,
    );

    notifyListeners();
```

```
  }

  Future<void> logout() async {
    await _auth.signOut();
    notifyListeners();
  }

  Future<void> resetPassword(String email) async {
    await _auth.sendPasswordResetEmail(
      email: email.trim(),
    );
  }
}
```

`main.dart` မှာ Firebase ကို Initialize လုပ်ပြီး Provider များကို Application တစ်ခုလုံးမှာ အသုံးပြုနိုင်အောင် သတ်မှတ်ရပါမယ်။

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:provider/provider.dart';

import 'firebase_options.dart';
import 'providers/auth_provider.dart';
import 'providers/product_provider.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(
          create: (_) => AuthProvider(),
        ),
        ChangeNotifierProvider(
          create: (_) => ProductProvider(),
        ),
      ],
      child: const ShoppingApp(),
    ),
  );
}

class ShoppingApp extends StatelessWidget {
  const ShoppingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
```

```
      title: 'Shopping App',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      home: const AuthGate(),
    );
  }
}

class AuthGate extends StatelessWidget {
  const AuthGate({super.key});

  @override
  Widget build(BuildContext context) {
    return StreamBuilder(
      stream: AuthProvider().user == null
          ? null
          : null,
      builder: (context, snapshot) {
        final auth = context.watch<AuthProvider>();

        if (!auth.isLoggedIn) {
          return const LoginPlaceholder();
        }

        return const HomePlaceholder();
      },
    );
  }
}

class LoginPlaceholder extends StatelessWidget {
  const LoginPlaceholder({super.key});

  @override
  Widget build(BuildContext context) {
    return const Scaffold(
      body: Center(child: Text('Login Screen')),
    );
  }
}

class HomePlaceholder extends StatelessWidget {
  const HomePlaceholder({super.key});

  @override
  Widget build(BuildContext context) {
    return const Scaffold(
      body: Center(child: Text('Shopping Home')),
    );
  }
}
```

အရေးကြီးသော ပြင်ဆင်ချက်: အထက်ပါ `AuthGate` ကို လက်တွေ့အသုံးပြုရာမှာ `StreamBuilder` ကို မလိုအပ်ဘဲ ထည့်ထားခြင်း မဖြစ်သင့်ပါဘူး။ Firebase Auth အခြေအနေကို စနစ်တကျ စောင့်ကြည့်ဖို့ `AuthProvider` ထဲမှာ `authStateChanges()` ကို expose လုပ်ပြီး `StreamBuilder<User?>` ဖြင့် Login နှင့် Home Screen ကို ခွဲပြသင့်ပါတယ်။ အောက်ပါပုံစံက ပိုသင့်တော်ပါတယ်။

```
// Add to AuthProvider:
Stream<User?> get authStateChanges =>
    _auth.authStateChanges();

// Use in AuthGate:
StreamBuilder<User?>(
  stream: context.read<AuthProvider>().authStateChanges,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Scaffold(
        body: Center(child: CircularProgressIndicator()),
      );
    }

    if (snapshot.data == null) {
      return const LoginPlaceholder();
    }

    return const HomePlaceholder();
  },
)
```

`LoginPlaceholder` နဲ့ `HomePlaceholder` တို့ကို နောက်ပိုင်းမှာ အမှန်တကယ် Login Form နဲ့ Shopping Home Screen များဖြင့် အစားထိုးရပါမယ်။

14.9. Navigation နှင့် Product List UI

Product များကို `ListView` သို့မဟုတ် `GridView` ဖြင့် ပြသပြီး Product တစ်ခုကို နှိပ်တဲ့အခါ Detail Screen သို့ သွားနိုင်အောင် တည်ဆောက်ပါ။

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/product_provider.dart';
import 'product_detail_screen.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});
```

```
  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  @override
  void initState() {
    super.initState();

    WidgetsBinding.instance.addPostFrameCallback((_) {
      context.read<ProductProvider>().loadProducts();
    });
  }

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<ProductProvider>();

    return Scaffold(
      appBar: AppBar(
        title: const Text('Shopping App'),
        actions: [
          IconButton(
            icon: const Icon(Icons.shopping_cart),
            onPressed: () {
              Navigator.pushNamed(context, '/cart');
            },
          ),
        ],
      ),
      body: provider.isLoading
          ? const Center(child: CircularProgressIndicator())
          : GridView.builder(
              padding: const EdgeInsets.all(12),
              gridDelegate:
                  const SliverGridDelegateWithFixedCrossAxisCount(
                crossAxisCount: 2,
                childAspectRatio: 0.72,
                crossAxisSpacing: 12,
                mainAxisSpacing: 12,
              ),
              itemCount: provider.products.length,
              itemBuilder: (context, index) {
                final product = provider.products[index];

                return Card(
                  child: InkWell(
                    onTap: () {
                      Navigator.push(
                        context,
                        MaterialPageRoute(
                          builder: (_) => ProductDetailScreen(
                            product: product,
                          ),
                        ),
                      );
                    },
```

```
                    child: Padding(
                      padding: const EdgeInsets.all(12),
                      child: Column(
                        crossAxisAlignment:
                            CrossAxisAlignment.start,
                        children: [
                          Expanded(
                            child: Center(
                              child: Icon(
                                Icons.shopping_bag,
                                size: 64,
                                color: Colors.blue.shade300,
                              ),
                            ),
                          ),
                          Text(
                            product.name,
                            maxLines: 2,
                            overflow: TextOverflow.ellipsis,
                          ),
                          const SizedBox(height: 6),
                          Text(
                            '\$${product.price.toStringAsFixed(2)}',
                            style: const TextStyle(
                              fontWeight: FontWeight.bold,
                            ),
                          ),
                        ],
                      ),
                    ),
                  ),
                );
              },
            ),
    );
  }
}
```

ဒီဥပမာမှာ Product များကို Grid ပုံစံဖြင့် ပြသထားပြီး Product ကို နှိပ်တဲ့အခါ `ProductDetailScreen` သို့ Navigation ပြုလုပ်ပါတယ်။ Cart Screen အတွက် `/cart` Route ကို `MaterialApp.routes` ထဲမှာ ထပ်မံသတ်မှတ်ရပါမယ်။

14.10. Shopping Cart State Management

Cart ထဲက Data ကို Screen အများအပြားက အသုံးပြုနိုင်ဖို့ `CartProvider` တည်ဆောက်ပါ။

```
import 'package:flutter/foundation.dart';
import '../models/product.dart';

class CartItem {
  final Product product;
```

```
  int quantity;

  CartItem({
    required this.product,
    this.quantity = 1,
  });

  double get subtotal => product.price * quantity;
}

class CartProvider extends ChangeNotifier {
  final Map<int, CartItem> _items = {};

  List<CartItem> get items => _items.values.toList();

  int get itemCount =>
      _items.values.fold(0, (sum, item) => sum + item.quantity);

  double get totalPrice =>
      _items.values.fold(0, (sum, item) => sum + item.subtotal);

  void addToCart(Product product) {
    final id = product.id;

    if (id == null) {
      throw ArgumentError('Product must have an ID');
    }

    if (_items.containsKey(id)) {
      _items[id]!.quantity++;
    } else {
      _items[id] = CartItem(product: product);
    }

    notifyListeners();
  }

  void removeFromCart(int productId) {
    _items.remove(productId);
    notifyListeners();
  }

  void clearCart() {
    _items.clear();
    notifyListeners();
  }
}
```

`MultiProvider` ထဲမှာ `CartProvider` ကိုလည်း ထည့်ပါ။

```
ChangeNotifierProvider(
  create: (_) => CartProvider(),
),
```

Cart Screen မှာ `context.watch<CartProvider>()` ကို အသုံးပြုပြီး Product Quantity, Subtotal နဲ့ Total Price ကို ပြသနိုင်ပါတယ်။ လက်တွေ့ Project မှာ Quantity ကို 0 အောက်မကျအောင် စစ်ဆေးခြင်းနှင့် Cart Data ကို App ပိတ်ပြီးနောက်လည်း ကျန်ရှိစေဖို့ Local Storage ထဲ သိမ်းခြင်းတို့ကို ထည့်သွင်းသင့်ပါတယ်။

14.11. Project အားလုံးကို ပေါင်းစပ်စမ်းသပ်ခြင်း

Testing Checklist

1. Firebase ကို Initialize လုပ်ပြီး Register / Login စမ်းသပ်ခြင်း
2. Login မဝင်ထားသူကို Home Screen မပြစေရန် စစ်ဆေးခြင်း
3. SQLite ထဲ Product အသစ်ထည့်ပြီး List တွင် ပြသခြင်း
4. Product Edit နှင့် Delete လုပ်ဆောင်ချက်များ စမ်းသပ်ခြင်း
5. Product Detail မှ Cart ထဲထည့်ပြီး Quantity ပြောင်းလဲခြင်း
6. Cart Total မှန်ကန်မှုနှင့် Checkout လုပ်ဆောင်ချက် စမ်းသပ်ခြင်း
7. Logout ပြုလုပ်ပြီး Login Screen သို့ ပြန်ရောက်မှု စစ်ဆေးခြင်း
8. App ကို Android Emulator နှင့် Physical Device တွင် စမ်းသပ်ခြင်း

14.12. Android APK နှင့် Google Play Store အတွက် Build ပြုလုပ်ခြင်း

စမ်းသပ်ရန် APK ထုတ်လိုပါက -

```
flutter analyze
flutter test
flutter build apk --release
```

APK ဖိုင်ကို ပုံမှန်အားဖြင့် အောက်ပါနေရာမှာ ရရှိနိုင်ပါတယ်။

```
build/app/outputs/flutter-apk/app-release.apk
```

Google Play Store သို့ တင်ရန် Android App Bundle ထုတ်နိုင်ပါတယ်။

```
flutter build appbundle --release
```

ထုတ်လုပ်မည့် App အတွက် Release Signing, Application ID, App Icon, Version Number, Privacy Policy နှင့် Google Play Console ရဲ့လက်ရှိလိုအပ်ချက်များကို စစ်ဆေးပြီးမှ Publish လုပ်သင့်ပါတယ်။

14.13. Summary

ဒီသင်ခန်းစာမှာ Shopping App တစ်ခုအတွက် အဓိက Architecture နဲ့ နမူနာ Source Code အစိတ်အပိုင်းများကို ပေါင်းစပ်လေ့လာခဲ့ပါတယ်။ အထက်ပါ Code များသည် သင်ခန်းစာအလိုက် တည်ဆောက်ရန် အခြေခံနမူနာများဖြစ်ပြီး ချက်ချင်း Run လို့ရတဲ့ Complete Application တစ်ခု မဟုတ်သေးပါဘူး။ Screen များ၊ Routes, Form Validation, CRUD UI, Checkout, Order History နဲ့ Error Handling တို့ကို အပြည့်အစုံ ချိတ်ဆက်ပြီး စမ်းသပ်ရန် ကျန်ရှိပါတယ်။

နောက်ဆုံး Lesson 15 မှာ Flutter Application ကို Production အတွက် ပြင်ဆင်ခြင်း၊ Testing, Performance Optimization, Release Build နှင့် Google Play Store Deployment တို့ကို လေ့လာနိုင်ပါတယ်။

CHAPTER 15

Publishing Flutter Apps to Google Play Store



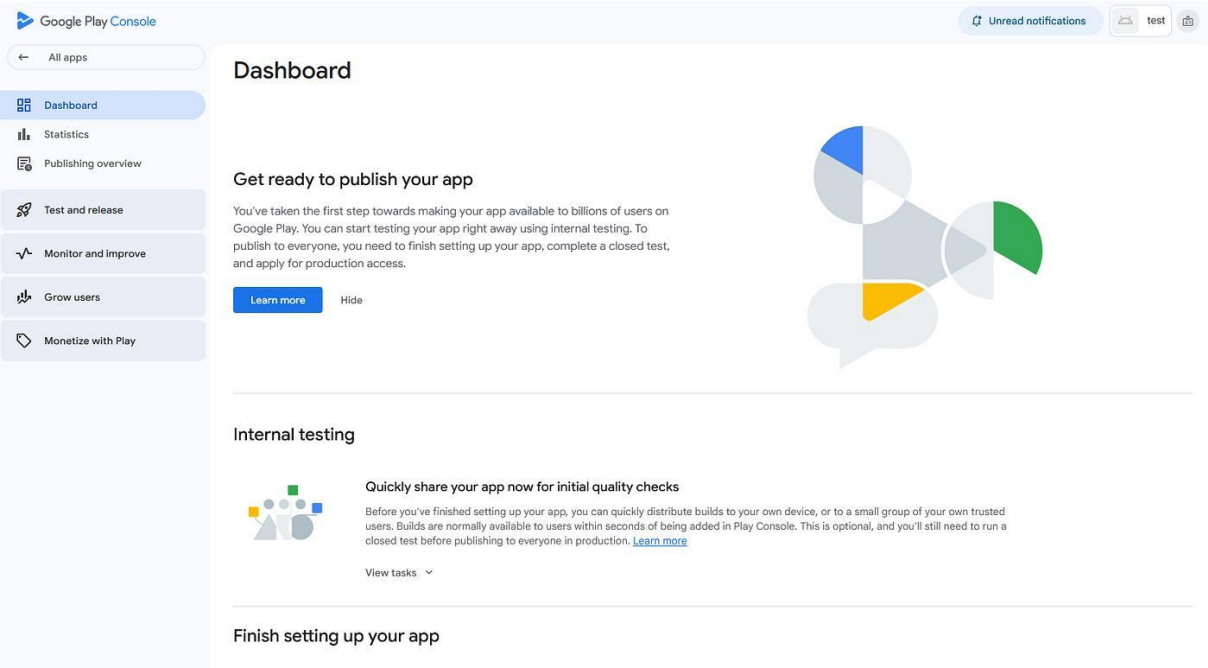
ဒီ Chapter မှာ Flutter Shopping Application ကို Development အဆင့်မှစတင်ပြီး Android Release App အဖြစ် Build လုပ်ခြင်း၊ Google Play Console တွင် Developer Account ဖွင့်ခြင်း၊ Store Listing ပြင်ဆင်ခြင်း၊ Privacy Policy နှင့် Data Safety ဖြည့်သွင်းခြင်း၊ Testing ပြုလုပ်ခြင်းနှင့် Production Release တင်ခြင်းအထိ လေ့လာသွားမှာ ဖြစ်ပါတယ်။

15.1. Chapter ၏ရည်ရွယ်ချက်များ

ဒီ Chapter ပြီးဆုံးတဲ့အခါ အောက်ပါလုပ်ဆောင်ချက်တွေကို ကိုယ်တိုင်ပြုလုပ်နိုင်မှာ ဖြစ်ပါတယ်။

- Flutter App ကို Release အတွက် ပြင်ဆင်နိုင်ခြင်း။
- Android App Bundle (`.aab`) ထုတ်လုပ်နိုင်ခြင်း။
- Google Play Console မှာ App အသစ်ဖန်တီးနိုင်ခြင်း။
- App Name, Description, Icon, Screenshots များ ပြင်ဆင်နိုင်ခြင်း။
- Privacy Policy, Data Safety နှင့် Content Rating ဖြည့်သွင်းနိုင်ခြင်း။
- Internal Testing၊ Closed Testing နှင့် Production Release ပြုလုပ်နိုင်ခြင်း။
- နောက်ပိုင်း App Version အသစ်များ Update တင်နိုင်ခြင်း။

15.2. Google Play Console ဆိုတာဘာလဲ?



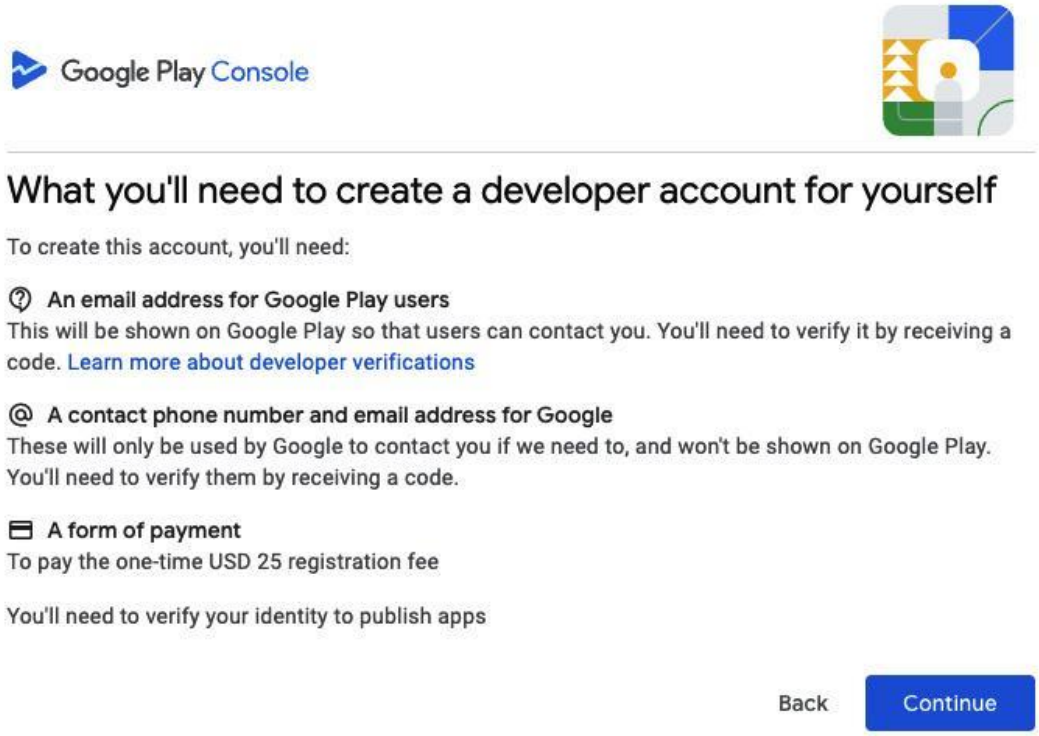
Google Play Console ဆိုတာ Android Application များကို Google Play Store ပေါ်သို့ ဖြန့်ချိဖို့၊ Testing ပြုလုပ်ဖို့၊ User Feedback ကြည့်ဖို့နဲ့ App Update များ စီမံခန့်ခွဲဖို့ အသုံးပြုတဲ့ Platform ဖြစ်ပါတယ်။

Google Play Store ပေါ်မှာ App တင်ဖို့အတွက် အဓိကအားဖြင့် အောက်ပါအဆင့်တွေ လိုအပ်ပါတယ်။

1. Google Play Console Developer Account ဖွင့်ခြင်း။
2. App အသစ် Create လုပ်ခြင်း။
3. Flutter Project ကို Release အတွက် ပြင်ဆင်ခြင်း။
4. Signed Android App Bundle (`.aab`) တည်ဆောက်ခြင်း။
5. Store Listing နှင့် App Policy အချက်အလက်များ ဖြည့်သွင်းခြင်း။
6. Testing ပြုလုပ်ပြီး Review အတွက် Submit လုပ်ခြင်း။
7. Google ၏ Review နှင့် ခွင့်ပြုချက်ရရှိပါက Release လုပ်ခြင်း။

Google Play Console ဝင်ရန် — https://play.google.com/console

15.3. Developer Account ဖွင့်ခြင်း



Google Play Console မှာ Developer Account ဖွင့်ဖို့ Google Account တစ်ခု၊ ဆက်သွယ်ရန်အချက်အလက်များနှင့် Google က တောင်းဆိုနိုင်တဲ့ Identity Verification အချက်အလက်များ လိုအပ်ပါတယ်။

လုပ်ဆောင်ရမည့်အဆင့်များ

1. Google Play Console ကို ဝင်ပါ။
2. Developer Registration ကို စတင်ပါ။
3. Personal သို့မဟုတ် Organization Account ကို ရွေးပါ။
4. Developer Information နှင့် ဆက်သွယ်ရန်အချက်အလက်များ ဖြည့်ပါ။
5. လိုအပ်သည့် Verification အဆင့်များ ပြီးစီးအောင် ဆောင်ရွက်ပါ။
6. သတ်မှတ်ထားသည့် Registration Fee ကို ပေးချေပါ။

သတိပြုရန်။ Personal Account အသစ်များအတွက် Google က သတ်မှတ်ထားသော Testing Requirement များ ရှိပါတယ်။ Account အမျိုးအစားအလိုက် လိုအပ်ချက်ကွာခြားနိုင်တာကြောင့် Registration ပြုလုပ်နေစဉ် Play Console မှာ ပြသထားတဲ့ သတ်မှတ်ချက်တွေကို လိုက်နာပါ။

15.4. Flutter Project ကို Release အတွက် ပြင်ဆင်ခြင်း

ကျွန်တော်တို့ရဲ့ Shopping Application Project ကို အသုံးပြုမယ်ဆိုရင် အရင်ဆုံး Project အတွင်းရှိ Error တွေကို ရှင်းလင်းပြီး App ကို စမ်းသပ်ရပါမယ်။

Terminal ကိုဖွင့်ပြီး အောက်ပါ Command များကို Run ပါ။

```
flutter doctor
flutter clean
flutter pub get
flutter analyze
flutter test
flutter run
```

Command တစ်ခုချင်းစီရဲ့ရည်ရွယ်ချက်ကတော့ —

- `flutter doctor` — Development Environment ကို စစ်ဆေးခြင်း။
- `flutter clean` — ယခင် Build Files များကို ရှင်းလင်းခြင်း။
- `flutter pub get` — လိုအပ်သော Packages များ ရယူခြင်း။
- `flutter analyze` — Dart Code အတွင်းရှိ ပြဿနာများကို စစ်ဆေးခြင်း။
- `flutter test` — Automated Tests များ လုပ်ဆောင်ခြင်း။
- `flutter run` — Application ကို Device ပေါ်မှာ စမ်းသပ်ခြင်း။

Application Name နှင့် Version သတ်မှတ်ခြင်း

Project ရဲ့ `pubspec.yaml` ကိုဖွင့်ပါ။

```
name: flutter_shopping_app
description: A Flutter shopping application

version: 1.0.0+1

environment:
```

```
  sdk: ">=3.0.0 <4.0.0"
```

ဒီနေရာမှာ `version: 1.0.0+1` ဆိုတာ `1.0.0` က User တွေမြင်ရမယ့် Version Name ဖြစ်ပြီး `+1` က Build Number ဖြစ်ပါတယ်။

နောက်ပိုင်း Update ထုတ်တဲ့အခါ ဥပမာ `1.0.1+2`၊ `1.1.0+3` လိုမျိုး Version ကို သင့်တော်သလို တိုးပေးရပါမယ်။ Dart SDK သတ်မှတ်ချက်ကိုတော့ ကိုယ်အသုံးပြုနေတဲ့ Flutter Version နဲ့ ကိုက်ညီအောင် ပြင်ဆင်ပါ။

Application ID သတ်မှတ်ခြင်း

Android App တစ်ခုချင်းစီမှာ မတူညီတဲ့ Application ID ရှိဖို့ လိုအပ်ပါတယ်။

ဥပမာ —

```
com.northerncity.shopping
```

Flutter Project ရဲ့ Android configuration ထဲမှာ `applicationId` ကို စစ်ဆေးပါ။ Project Template အလိုက် `android/app/build.gradle` သို့မဟုတ် `android/app/build.gradle.kts` ဖိုင်မှာ ရှိနိုင်ပါတယ်။

Production အတွက် အသုံးပြုမယ့် ID ကို သေချာရွေးပါ။ Play Store ပေါ်မှာ App ထုတ်ဝေပြီးနောက် အဲဒီ App ကို အခြား Application ID နဲ့ အစားထိုး Update တင်လို့ မရပါဘူး။

15.5. Upload Keystore နှင့် App Signing ပြုလုပ်ခြင်း

Google Play Store သို့ Release တင်တဲ့အခါ App ကို Digital Certificate ဖြင့် Sign လုပ်ထားရပါတယ်။ Flutter က Android App Bundle ကို Build လုပ်နိုင်ပေမယ့် Signing Configuration ကို မှန်ကန်စွာ ပြင်ဆင်ထားဖို့ လိုပါတယ်။

Step 1 — Keystore ဖန်တီးခြင်း

Windows Command Prompt သို့မဟုတ် PowerShell မှာ အောက်ပါ Command ကို Run ပါ။

```
keytool -genkey -v `
  -keystore upload-keystore.jks `
  -keyalg RSA `
  -keysize 2048 `
  -validity 10000 `
  -alias upload
```

Terminal က မေးလာတဲ့ Password နဲ့ အခြားအချက်အလက်တွေကို ဖြည့်ပါ။

အရေးကြီးချက်များ

- `upload-keystore.jks` ဖိုင်ကို လုံခြုံစွာ သိမ်းထားပါ။
- Password များကို Source Code ထဲ တိုက်ရိုက်မရေးပါနှင့်။
- Keystore နှင့် Password ကို GitHub ကဲ့သို့ Public Repository ပေါ်မတင်ပါနှင့်။
- လုံခြုံသော Backup တစ်ခုထားပါ။

Step 2 — key.properties ဖန်တီးခြင်း

Project ရဲ့ `android/key.properties` ဖိုင်ကို ဖန်တီးပြီး အောက်ပါအချက်အလက်များ ထည့်ပါ။

```
storePassword=YOUR_STORE_PASSWORD
keyPassword=YOUR_KEY_PASSWORD
keyAlias=upload
storeFile=../upload-keystore.jks
```

ဒီဥပမာမှာ Keystore ဖိုင်ကို Project ရဲ့ Root Folder မှာ ထားထားတယ်လို့ ယူဆထားပါတယ်။
သင့်ဖိုင်တည်နေရာနဲ့ ကိုက်ညီအောင် `storeFile` Path ကို ပြင်ဆင်ပါ။

Step 3 — Gradle Signing Configuration

Project က Kotlin DSL (`build.gradle.kts`) အသုံးပြုထားတယ်ဆိုရင်
`android/app/build.gradle.kts` ထဲမှာ Signing Configuration ကို ပြင်ဆင်ရပါမယ်။
အောက်ပါအပိုင်းက နမူနာဖြစ်ပြီး လက်ရှိ Project ရဲ့ Gradle Configuration နဲ့ ပေါင်းစပ်ရပါမယ်။

```
import java.util.Properties
import java.io.FileInputStream
```

```
val keystoreProperties = Properties()
val keystorePropertiesFile = rootProject.file("key.properties")

if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(
        FileInputStream(keystorePropertiesFile)
    )
}

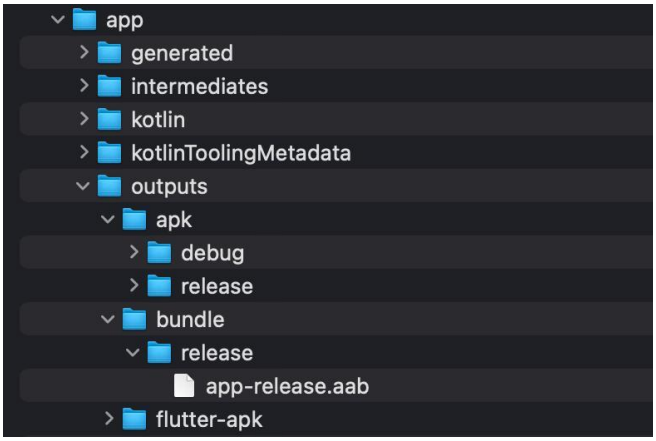
android {
    signingConfigs {
        create("release") {
            keyAlias = keystoreProperties["keyAlias"] as String
            keyPassword =
                keystoreProperties["keyPassword"] as String
            storeFile = file(
                keystoreProperties["storeFile"] as String
            )
            storePassword =
                keystoreProperties["storePassword"] as String
        }
    }

    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
        }
    }
}
```

Project ထဲမှာ `release` Signing Configuration ရှိပြီးသားဆိုရင် ထပ်မံဖန်တီးမယ့်အစား ရှိပြီးသား Configuration ကို ပြင်ဆင်ပါ။ `key.properties` နဲ့ Keystore ဖိုင်ကို Version Control မှာ မတင်မိစေရန် `.gitignore` မှာလည်း ထည့်ထားပါ။

```
android/key.properties
*.jks
*.keystore
```

15.6. Android App Bundle (.aab) Build လုပ်ခြင်း



Google Play Store သို့ တင်ဖို့ Android App Bundle (`.aab`) ကို အသုံးပြုပါမယ်။

Project Root Folder မှာ Terminal ဖွင့်ပြီး —

```
flutter clean
flutter pub get
flutter analyze
flutter build appbundle --release
```

Build အောင်မြင်သွားရင် အောက်ပါ Path မှာ File ထွက်လာပါမယ်။

```
build/app/outputs/bundle/release/app.aab
```

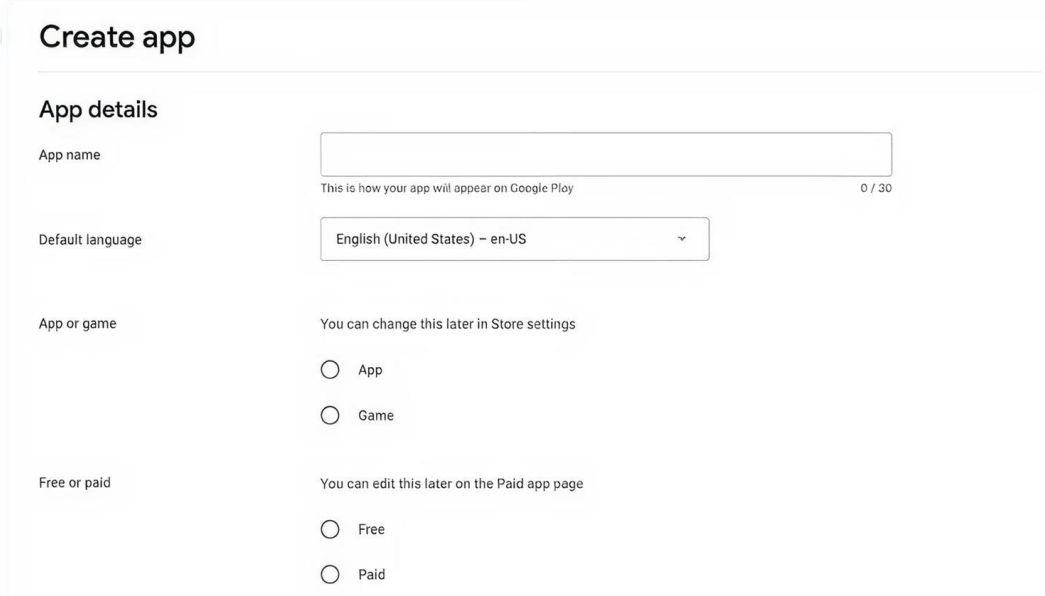
Play Console မှာ Upload လုပ်မယ့် File က ဒီ `app.aab` ဖြစ်ပါတယ်။ `.apk` က Device မှာ တိုက်ရိုက် Install လုပ်ပြီး စမ်းသပ်ဖို့ အသုံးဝင်ပေမယ့် Play Store ဖြန့်ချိမှုအတွက် App Bundle ကို ဦးစားပေးအသုံးပြုနိုင်ပါတယ်။

Target Android API Level ကို စစ်ဆေးခြင်း

အရေးကြီးတဲ့အချက်တစ်ခုက 2026 ခုနှစ် ဩဂုတ်လ 31 ရက်မှစပြီး Google Play သို့ တင်သွင်းမယ့် App အသစ်များနှင့် App Updates များသည် Android 16 (API Level 36) သို့မဟုတ် ပိုမြင့်တဲ့ API Level ကို Target လုပ်ရမှာ ဖြစ်ပါတယ်။ အခြား Device အမျိုးအစားအချို့မှာ သီးခြားသတ်မှတ်ချက်တွေ ရှိပါတယ်။

Flutter Project ရဲ့ Android SDK Configuration ကို စစ်ဆေးပြီး လိုအပ်ပါက `targetSdk` ကို 36 သို့မဟုတ် သတ်မှတ်ချက်နဲ့ ကိုက်ညီတဲ့ Version သို့ မြှင့်တင်ပါ။ Flutter SDK နဲ့ Android Gradle Plugin တို့လည်း သက်ဆိုင်ရာ API Level ကို Support လုပ်ရပါမယ်။

15.7. Google Play Console တွင် App အသစ် Create လုပ်ခြင်း



Google Play Console ကို ဝင်ပြီး အောက်ပါအဆင့်များကို လုပ်ဆောင်ပါ။

1. Create app ကို နှိပ်ပါ။
2. App Name — `Northern City Shopping` ဟု ထည့်ပါ။
3. Default Language ကို ရွေးပါ။
4. App သို့မဟုတ် Game ကို ရွေးပါ။
5. Free သို့မဟုတ် Paid ကို သတ်မှတ်ပါ။
6. Developer Declarations များကို ဖတ်ရှုပြီး မှန်ကန်စွာ ဖြည့်သွင်းပါ။
7. Create app ကို နှိပ်ပါ။

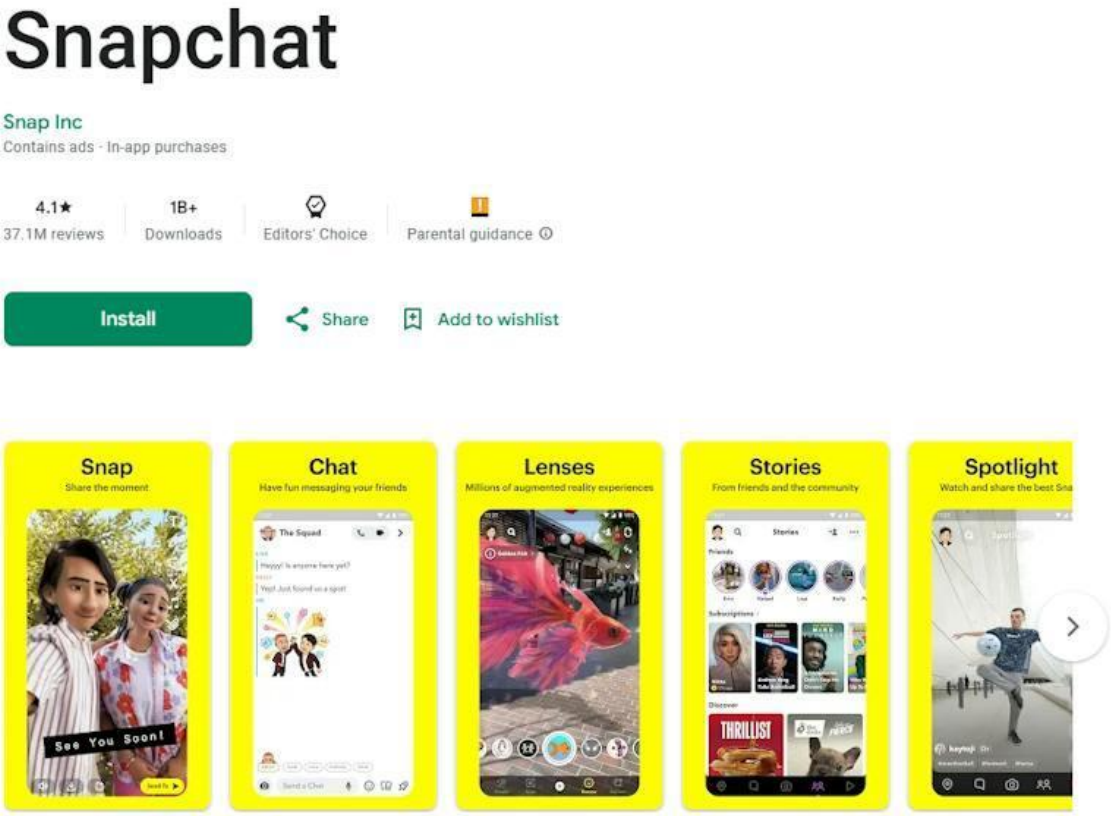
App အသစ်တစ်ခုအတွက် အောက်ပါအချက်အလက်တွေကို ဖြည့်သွင်းရနိုင်ပါတယ်။

- App access
- Ads declaration
- Content rating
- Target audience
- Data Safety
- Privacy Policy
- Store Listing
- Testing and Release

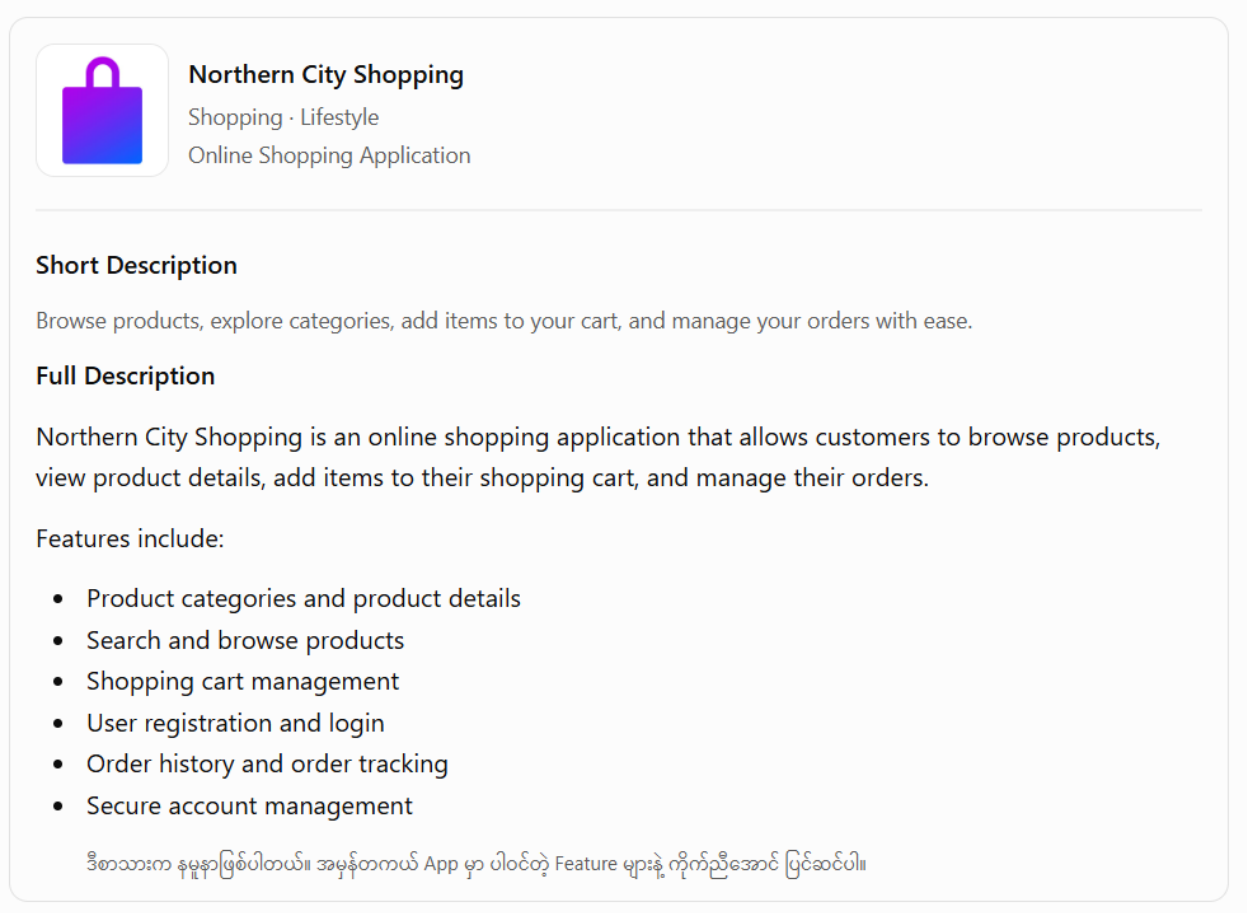
Play Console ရဲ့ Dashboard မှာ လုပ်ဆောင်ရန်ကျန်ရှိတဲ့ Task တွေကို စစ်ဆေးနိုင်ပါတယ်။

15.8. Store Listing ပြင်ဆင်ခြင်း

Store Listing ဆိုတာ User တွေ Play Store မှာ App ကို ရှာဖွေတဲ့အခါ မြင်တွေ့ရမယ့် အချက်အလက်တွေ ဖြစ်ပါတယ်။



App Listing နမူနာ



Northern City Shopping

Shopping · Lifestyle

Online Shopping Application

Short Description

Browse products, explore categories, add items to your cart, and manage your orders with ease.

Full Description

Northern City Shopping is an online shopping application that allows customers to browse products, view product details, add items to their shopping cart, and manage their orders.

Features include:

- Product categories and product details
- Search and browse products
- Shopping cart management
- User registration and login
- Order history and order tracking
- Secure account management

ဒီစာသားက နမူနာဖြစ်ပါတယ်။ အမှန်တကယ် App မှာ ပါဝင်တဲ့ Feature များနဲ့ ကိုက်ညီအောင် ပြင်ဆင်ပါ။

လိုအပ်သော Graphic Assets

Asset	ပြင်ဆင်ရန်
App Icon	512 × 512 px
Feature Graphic	1024 × 500 px
Phone Screenshots	App Screen အမှန်များကို Capture လုပ်ပါ
App Name	ရှင်းလင်းပြီး မှတ်မိလွယ်သောအမည်
Description	App ရဲ့ Feature များကို တိကျစွာ ဖော်ပြရန်

Screenshot များမှာ Login Screen၊ Home Screen၊ Product Detail၊ Shopping Cart နှင့် Order History တို့ကို ပြသနိုင်ပါတယ်။ Play Console ရဲ့ လက်ရှိ Asset Specification နဲ့ သင့်တော်တဲ့ Device အမျိုးအစားများကို စစ်ဆေးပါ။

15.9. Privacy Policy ပြင်ဆင်ခြင်း

Shopping App မှာ User Registration၊ Email၊ Profile၊ Order History သို့မဟုတ် Firebase Authentication အသုံးပြုထားရင် Privacy Policy ကို သေချာပြင်ဆင်ဖို့ လိုအပ်ပါတယ်။

Privacy Policy ထဲမှာ အောက်ပါအချက်တွေ ပါဝင်သင့်ပါတယ်။

- ဘယ်လို User Data တွေ စုဆောင်းသလဲ။
- Data တွေကို ဘာအတွက် အသုံးပြုသလဲ။
- ဘယ် Third-party Services တွေနဲ့ မျှဝေသလဲ။
- Data တွေကို ဘယ်လောက်ကြာ သိမ်းဆည်းသလဲ။
- User က Data ဖျက်ပေးရန် ဘယ်လိုတောင်းဆိုနိုင်သလဲ။
- Developer ကို ဘယ်လိုဆက်သွယ်နိုင်သလဲ။

Privacy Policy HTML နမူနာ

အောက်ပါစာသားကို ကိုယ့် App အတွက် မှန်ကန်အောင် ပြင်ဆင်ပြီး Website တစ်ခုမှာ HTTPS ဖြင့် လွှင့်တင်နိုင်ပါတယ်။

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Privacy Policy - Shopping App</title>
</head>
<body>
  <h1>Privacy Policy</h1>
  <p>Last updated: October 12, 2026</p>

  <h2>Information We Collect</h2>
  <p>
```

```
    Our app may process account information, contact details,
    and order information required to provide shopping services.
    Update this section to describe actual data collection.
  </p>

  <h2>How We Use Information</h2>
  <p>
    We use relevant information to manage accounts,
    process orders, and provide customer support.
  </p>

  <h2>Third-Party Services</h2>
  <p>
    Disclose the actual services used by the app, such as
    authentication providers, analytics, or payment providers.
  </p>

  <h2>Data Deletion</h2>
  <p>
    Explain how users can request account and data deletion,
    where applicable.
  </p>

  <h2>Contact</h2>
  <p>Email: your-support@example.com</p>
</body>
</html>
```

သတိပြုရန်။ ဒီ HTML က အခြေခံ Template သာ ဖြစ်ပါတယ်။ ကိုယ့် App က စုဆောင်းတဲ့ Data၊ Third-party SDK များ၊ Account Deletion နည်းလမ်းနှင့် Developer Contact တို့ကို အမှန်အတိုင်း ထည့်သွင်းရပါမယ်။ နမူနာစာသားအတိုင်းသာ ထုတ်ဝေခြင်းက Policy လိုအပ်ချက်များကို ပြည့်မီကြောင်း အာမမခံပါဘူး။

15.10. Data Safety Form ဖြည့်သွင်းခြင်း

Data Safety ဆိုတာ App က User Data များကို ဘယ်လို စုဆောင်းခြင်း၊ အသုံးပြုခြင်း၊ မျှဝေခြင်းနှင့် ကာကွယ်ခြင်း ပြုလုပ်သလဲဆိုတာကို User တွေ သိရှိနိုင်စေဖို့ ဖော်ပြတဲ့ အချက်အလက် ဖြစ်ပါတယ်။

ဥပမာ Shopping App တစ်ခုမှာ —

Data အမျိုးအစား	ဥပမာ
Personal Information	Name, Email
Financial Information	Payment Data ရှိပါက သက်ဆိုင်ရာအချက်အလက်

App Activity	App Interaction၊ Order History
Device or Other IDs	သက်ဆိုင်ရာ SDK များက စုဆောင်းပါက

အထက်ပါအချက်တွေဟာ စဉ်းစားကြည့်ဖို့ ဥပမာများသာ ဖြစ်ပါတယ်။ App တိုင်းမှာ အားလုံးကို ရွေးရမယ်လို့ မဆိုလိုပါဘူး။

Data Safety Form ကို ဖြည့်သွင်းတဲ့အခါ —

1. App ရဲ့ကိုယ်ပိုင် Code ကို စစ်ဆေးပါ။
2. Firebase၊ Analytics၊ Ads နှင့် Payment SDK များကို စစ်ဆေးပါ။
3. စုဆောင်းသည့် Data အမျိုးအစားများကို သတ်မှတ်ပါ။
4. Data ကို မျှဝေသလားဆိုတာ စစ်ဆေးပါ။
5. Data ကို Encryption ဖြင့် ကာကွယ်ထားသလားဆိုတာ မှန်ကန်စွာ ဖြည့်ပါ။
6. Privacy Policy နဲ့ Data Safety ကြေညာချက်များ ကိုက်ညီမှုရှိစေပါ။

Third-party SDK တစ်ခုက Data စုဆောင်းနေပါကလည်း Developer အနေနဲ့ သက်ဆိုင်ရာ Policy များကို လိုက်နာရပါတယ်။

15.11. Content Rating နှင့် Target Audience

Content Rating က App ကို ဘယ်အသက်အရွယ်ရှိသူတွေ အသုံးပြုဖို့ သင့်တော်သလဲဆိုတာ သတ်မှတ်ပေးပါတယ်။

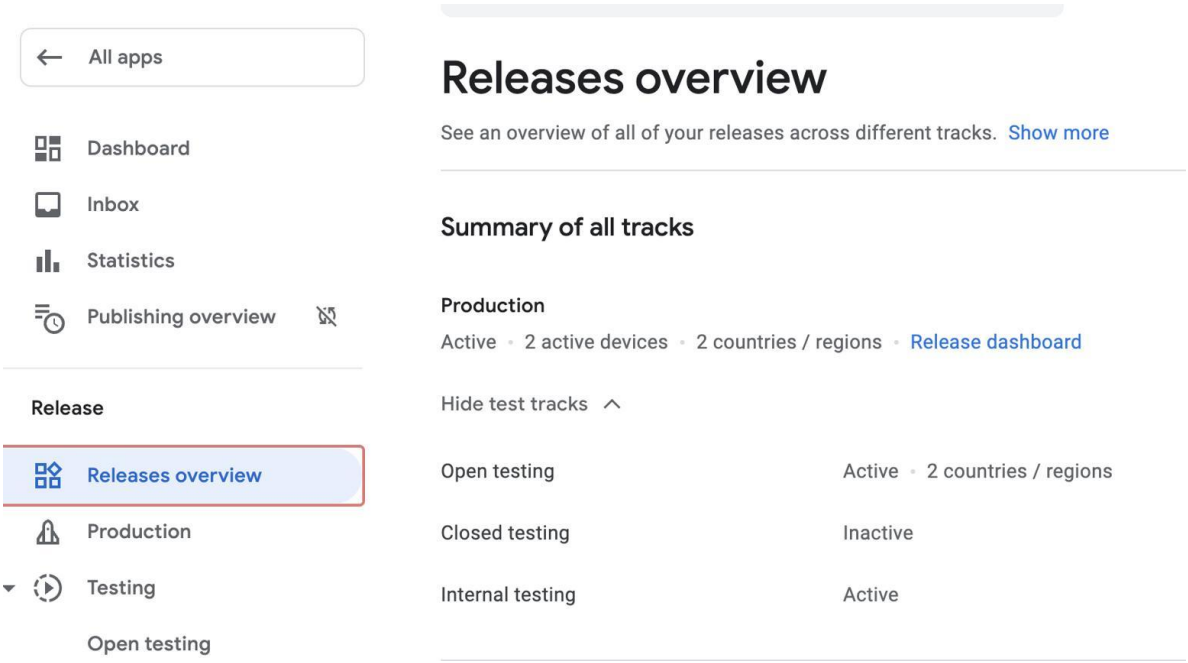
Play Console ရဲ့ App content အပိုင်းကို ဝင်ပြီး —

- Content Rating Questionnaire ဖြည့်ပါ။
- Target Audience သတ်မှတ်ပါ။
- App မှာ Ads ပါဝင်သလားဆိုတာ ကြေညာပါ။
- App Access လိုအပ်ချက်များကို ဖြည့်ပါ။

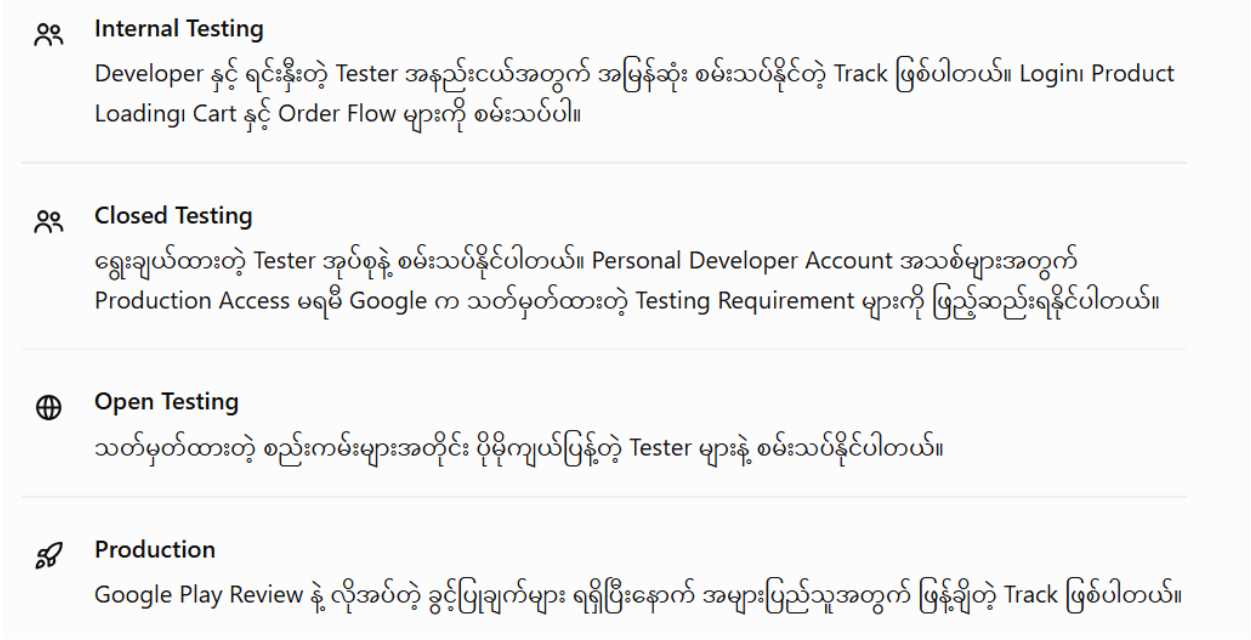
- Login မဖြစ်မနေလိုအပ်တဲ့ App ဖြစ်ပါက Review ပြုလုပ်နိုင်ရန် လိုအပ်သော Access Instructions များ ပေးပါ။

Content Rating ကို ခန့်မှန်းပြီး မရွေးချယ်ဘဲ Questionnaire ထဲက မေးခွန်းတွေကို အမှန်အတိုင်း ဖြေဆိုပါ။ Google Play မှာ Rating မရှိတဲ့ App များကို ထုတ်ဝေခွင့်မပြုတဲ့ Policy လည်း ရှိပါတယ်။

15.12. Testing Tracks များ



App ကို User အားလုံးဆီ မဖြန့်ချိမီ Testing ပြုလုပ်ဖို့ လိုအပ်ပါတယ်။



Google ရဲ့လက်ရှိသတ်မှတ်ချက်အရ 2023 ခုနှစ် နိုဝင်ဘာလ 13 ရက်နောက်ပိုင်း ဖန်တီးထားတဲ့ Personal Developer Account များအတွက် Closed Testing လုပ်ငန်းစဉ်တွင် Tester အနည်းဆုံး 12 ဦးက 14 ရက်ဆက်တိုက် Opt-in လုပ်ထားရန် လိုအပ်ပါတယ်။ ထို့နောက် Production Access ကို လျှောက်ထားရပြီး Google ရဲ့သတ်မှတ်ချက်များအတိုင်း ဆောင်ရွက်ရပါတယ်။ Account အလိုက် လက်ရှိ Play Console မှာ ပြသထားတဲ့ Requirement ကို အတည်ပြုပါ။

Testing လုပ်ဆောင်ချက်များ

- User Registration နှင့် Login
- Password Reset
- Product List နှင့် Product Detail
- Add to Cart / Remove from Cart
- Order Submission
- Database နှင့် API Error Handling
- Network မရှိတဲ့အချိန် စမ်းသပ်ခြင်း
- Screen Size မတူတဲ့ Device များတွင် စမ်းသပ်ခြင်း
- App Crash နှင့် Performance စစ်ဆေးခြင်း

15.13. Production Release တင်ခြင်း

Testing အောင်မြင်ပြီး Store Listing နဲ့ Policy Form များ ပြည့်စုံသွားရင် Production Release ပြုလုပ်နိုင်ပါတယ်။

1. Google Play Console ထဲက ကိုယ့် App ကို ဖွင့်ပါ။
2. Test and release → Production အပိုင်းကို ဝင်ပါ။ Console ရဲ့လက်ရှိ UI အလိုက် Menu အမည် အနည်းငယ်ကွာနိုင်ပါတယ်။
3. Create new release ကို ရွေးပါ။
4. `app.aab` ဖိုင်ကို Upload လုပ်ပါ။
5. Release Notes ရေးပါ။ ဥပမာ — `Initial release of Northern City Shopping.`
6. Play Console က ပြသတဲ့ Error၊ Warning နှင့် Policy Issue များကို ဖြေရှင်းပါ။
7. လိုအပ်တဲ့ Declarations နှင့် Release Information များ ပြည့်စုံကြောင်း စစ်ဆေးပြီး Review အတွက် Submit လုပ်ပါ။

Google က Review ပြုလုပ်ပြီး ခွင့်ပြုမှသာ App ကို ဖြန့်ချိနိုင်မှာ ဖြစ်ပါတယ်။ Review ကြာချိန်ဟာ App၊ Account နှင့် Policy စစ်ဆေးမှုအပေါ် မူတည်နိုင်ပါတယ်။

15.14. App Update အသစ် ထုတ်ဝေခြင်း

App ကို Play Store ပေါ်မှာ ထုတ်ဝေပြီးနောက် Feature အသစ်တွေ ထည့်ခြင်း၊ Bug Fix ပြုလုပ်ခြင်းတို့အတွက် Update Version ထုတ်ပေးနိုင်ပါတယ်။

ဥပမာ `pubspec.yaml` မှာ —

```
version: 1.0.1+2
```

ပြီးနောက် Code ကို ပြင်ဆင်ပြီး စမ်းသပ်ပါ။

```
flutter pub get
flutter analyze
flutter test
flutter build appbundle --release
```

ထွက်လာတဲ့ `.aab` ဖိုင်ကို Play Console မှာ Release အသစ်အဖြစ် Upload လုပ်ပါ။

သတိပြုရန်။ Update တင်တဲ့အခါ Version Code/Build Number ကို ယခင် Release ထက် မြှင့်ပေးရပါမယ်။ App ရဲ့ Application ID ကို မပြောင်းပါနှင့်။ ယခင် App နဲ့ ကိုက်ညီတဲ့ Signing Setup ကိုလည်း အသုံးပြုရပါမယ်။

15.15. လက်တွေ့ Project Assignment

သင်ခန်းစာအဆုံးမှာ `Northern City Shopping` App ကို Google Play Store ပေါ်သို့ တင်ဖို့ အောက်ပါလုပ်ငန်းများကို ပြီးစီးအောင် လုပ်ဆောင်ပါ။

Publishing Checklist

1. Flutter App ကို Error မရှိအောင် စမ်းသပ်ပြီး Release Build ပြင်ဆင်ရန်
2. Application ID၊ Version Name နှင့် Build Number သတ်မှတ်ရန်
3. Upload Keystore နှင့် Signing Configuration ပြင်ဆင်ရန်
4. Release app.aab ကို Build လုပ်ရန်
5. Developer Account နှင့် Play Console App Setup ပြီးစီးရန်
6. App Icon၊ Feature Graphic နှင့် Screenshots ပြင်ဆင်ရန်
7. Privacy Policy၊ Data Safety နှင့် Content Rating ဖြည့်သွင်းရန်
8. သတ်မှတ်ထားတဲ့ Testing Track မှာ စမ်းသပ်ရန်
9. Production Access နှင့် Release လိုအပ်ချက်များ ပြည့်မီအောင် ဆောင်ရွက်ရန်
10. Review အတွက် Submit လုပ်ပြီး App Update လုပ်နည်းကို လေ့ကျင့်ရန်

15.16. Summary

ဒီသင်ခန်းစာမှာ Flutter Application ကို Release အတွက် ပြင်ဆင်ခြင်း၊ `.aab` Build လုပ်ခြင်း၊ Signing Key စီမံခန့်ခွဲခြင်း၊ Google Play Console Account ဖွင့်ခြင်း၊ Store Listing နှင့် Privacy Policy ပြင်ဆင်ခြင်း၊ Data Safety၊ Content Rating၊ Testing Tracks နှင့် Production Release တင်ခြင်းတို့ကို လေ့လာခဲ့ပါတယ်။

အထူးသဖြင့် Keystore ကို လုံခြုံစွာ ထိန်းသိမ်းခြင်း၊ Data Safety ကို အမှန်အတိုင်း ကြေညာခြင်း၊ Android Target API Requirement ကို လိုက်နာခြင်းနဲ့ Update Version Number ကို မြှင့်ပေးခြင်း တို့ဟာ App တစ်ခုကို ရေရှည်ထိန်းသိမ်းနိုင်ဖို့ အရေးကြီးပါတယ်။

Official Documentation

- Flutter — Android App Build and Release
- Google Play Console — Developer Console
- Testing Requirements for New Personal Accounts
- Target API Level Requirements

Flutter Course — Lesson 1 မှ Lesson 15 အထိ ပြီးဆုံးပါပြီ။ ယခုအချိန်မှာ Development Environment Setup ကနေ Shopping Application တည်ဆောက်ခြင်း၊ Authentication၊ SQLite၊ REST API၊ Testing နှင့် Google Play Store Publishing အထိ အခြေခံလုပ်ငန်းစဉ်များကို လေ့လာပြီးဖြစ်ပါတယ်။

References

I. Beginning Flutter: A Hands-On Guide to App Development
Author: Marco L. Napoli
Covers Flutter fundamentals, widgets, layouts, navigation, and application development.
Publisher information: Wiley

II. Flutter Complete Reference
Author: Alberto Miola
A practical reference for Flutter widgets, layouts, navigation, and app features.
Author's website: Alberto Miola

III. Flutter Apprentice
Authors: Michael Katz, Kevin Moore, Vincent Ngo, and others
Project-based learning covering Flutter fundamentals and building practical mobile applications.
Learning resource: Kodeco

IV. Flutter in Action
Author: Eric Windmill
Explains Flutter's programming model, widgets, state, and application architecture.
Publisher: Manning Publications

V. Dart Apprentice: Beginning Programming with Dart
Authors: Jonathan Sande and Matt Galloway
Covers Dart syntax, variables, functions, collections, classes, and object-oriented programming.
Learning resource: Kodeco

VI. Programming Flutter
Author: Carmine Zaccagnino
A practical book for understanding Flutter application development and cross-platform mobile programming.
Book search: Google Books

VII. Flutter Official Documentation
Google
https://docs.flutter.dev/Official guides for installation, widgets, navigation, networking, state management, testing, and deployment.

VIII. Dart Language Documentation
Dart Team
https://dart.dev/Learn variables, data types, functions, classes, collections, asynchronous programming, and null safety.

IX. Flutter Cookbook
Flutter Team
https://docs.flutter.dev/cookbookPractical examples for forms, navigation, networking, persistence, animations, and common app development tasks.

X. Firebase Documentation for Flutter
Google Firebase
https://firebase.google.com/docs/flutterCovers Firebase Authentication, Cloud Firestore, Realtime Database, cloud storage, and other backend services

XI. pub.dev – Dart and Flutter Packages
Dart Package Repository
https://pub.dev/Find packages for SQLite, HTTP requests, Provider, Riverpod, image handling, and other application features.

XII. Android Developers – Publish Your App
Google
https://developer.android.com/studio/publishLearn Android app signing, release preparation, application packaging, and publishing workflows for Android applications.

About Author

He is an experienced IT professional and educator with over 20 years of comprehensive experience in software development and teaching. His career includes significant international roles, such as serving as a Senior Software Developer at Zepto Microsoft Solutions in Kuala Lumpur, Malaysia, and leadership positions at home country as the IT Director at ValueStar Institute in Yangon. Building on this extensive expertise, he is now the Founder of the Northern City Computer Center in Kamayut Township, Yangon, where he channels his passion for technology into training the next generation of IT talent in Myanmar.



Empowering Future Innovators